

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

(найменування кафедри)

КУРСОВИЙ ПРОЄКТ
(РОБОТА)

з дисципліни «Безпека та захист програм і даних»

(назва дисципліни)

на тему: Розробка програмного засобу стеганографічного захисту інформації
в цифрових зображеннях

Студента 4 курсу КНТ-113сп групи
спеціальності 121 Інженерія
програмного забезпечення
освітня програма (спеціалізація) інженерія
програмного забезпечення
Щедровського І. А.

(прізвище та ініціали)

Керівник доцент, Зайко Т. А.

(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Національна шкала

Кількість балів: Оцінка: ECTS

Члени комісії

(підпис)

Зайко Т. А.

(прізвище та ініціали)

(підпис)

(прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Запорізька політехніка»
 (повне найменування закладу вищої освіти)

Інститут, факультет Запорізький національний університет НУ «Запорізька політехніка». Факультет комп'ютерних наук і технологій ;
 Кафедра програмних засобів
 Ступінь вищої освіти бакалавр
 Спеціальність 121 Інженерія програмного забезпечення
 (код і найменування)
 Освітня програма (спеціалізація) Інженерія програмного забезпечення
 (назва освітньої програми (спеціалізації))

ЗАТВЕРДЖУЮ

Завідувач кафедри ПЗ, д.т.н, проф.
С.О. Субботін
 “ ” 2025 року

З А В Д А Н Н Я

НА КУРСОВИЙ ПРОЄКТ (РОБОТУ) СТУДЕНТА

Щедровського Івана Андрійовича
 (прізвище, ім'я, по батькові)

1. Тема проєкту (роботи) Розробка програмного засобу стеганографічного захисту інформації в цифрових зображеннях
 керівник проєкту (роботи) Зайко Тетяна Анатоліївна, доцент,
 (прізвище, ім'я, по батькові, науковий ступінь, вчене звання)
 затверджені наказом закладу вищої освіти від _____
2. Строк подання студентом проєкту (роботи) 21 грудня 2025 року
3. Вихідні дані до проєкту (роботи) застосунок згідно теми курсової роботи; технічне завдання; опис програми; специфікація.
4. Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити) 1. Аналіз предметної області; 2. Вибір та обґрунтування структури системи, яка проектується; 3. Основні рішення щодо реалізації системи; 4. Керівництво програміста; 5. Керівництво оператора; 6. Тестування і результати; 7. Технічне завдання; 8. Опис програми; 9. Специфікація; 10. Текст програми.
5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень) Слайди презентації

6. Консультанти розділів проєкту (роботи)

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	прийняв виконане завдання
1-5 Основна частина	Зайко Т.А., доцент		

7. Дата видачі завдання 06 жовтня 2025 р.

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів курсового проєкту (роботи)	Строк виконання етапів проєкту (роботи)	Примітка
1.	Аналіз індивідуального завдання.	1 тиждень	
2.	Аналіз програмних засобів, що будуть використовуватись в роботі.	2 тиждень	
3.	Аналіз структур даних, що необхідно використати в курсовій роботі.	3 тиждень	
4.	Затвердження завдання	4 тиждень	
5.	Вивчення можливостей програмної реалізації структур даних та інтерфейсу користувача.	5-9 тиждень	
6.	Аналіз вимог до апаратних засобів	9 тиждень	
7.	Розробка програмного забезпечення	9-13 тиждень	
8.	Проміжний контроль	10 тиждень	Розділи 1-5 ПЗ
9.	Оформлення, відповідних пунктів пояснювальної записки.	10-14 тиждень	Розділи 1-2 ПЗ
10.	Захист курсової роботи.	15 тиждень	

Студент _____ Щедровський І.А.
(підпис) (прізвище та ініціали)

Керівник проєкту (роботи) _____ Зайко Т.А.
(підпис) (прізвище та ініціали)

РЕФЕРАТ

Пояснювальна записка до курсової роботи містить 155 сторінок, 58 рисунків, 0 таблиць, 4 додатки, 20 джерел.

Пояснювальна записка складається з шести розділів.

Розділ «Аналіз предметної області» містить в собі аналіз сучасних методів стеганографії цифрових зображень, аналіз сучасних інструментів та детальний аналіз двох найбільш популярних методів.

Розділ «Вибір та обґрунтування структури системи, яка проєктується» містить спроектовану структуру застосунку та аналіз інструментарію для розробки програмного забезпечення.

Розділ «Основні рішення щодо реалізації компонентів системи» містить опис всіх рішень, які були прийняті під час реалізації системи, щоб зробити її максимально надійною.

Розділ «Керівництво програміста» містить інструкцію про запуск застосунку для розробки, його налагодження та майбутню підтримку.

Розділ «Керівництво оператора» містить інструкцію як використовувати застосунок, а також детальний опис всіх його функцій.

Розділ «Тестування і результати» містить опис методики та проведення тестування застосунку, аналіз його недоліків та функціоналу, який можна покращити.

STEGANOGRAPHY, GOLANG, JAVASCRIPT, TYPESCRIPT,
ПРИХОВУВАННЯ ІНФОРМАЦІЇ, АНАЛІЗ, ТЕСТУВАННЯ

ЗМІСТ

Перелік умовних термінів	7
Вступ.....	8
1 Аналіз предметної області	9
2 Вибір та обґрунтування структури системи, яка проектується	22
2.1 Вибір інструментарію розробки програмного забезпечення	22
2.2 Структурна схема програми та опис модулів розроблюваної системи ..	28
3 Основні рішення щодо реалізації компонентів системи	32
4 Керівництво програміста	39
4.1 Призначення й умови застосування програми	39
4.2 Характеристики програми	39
4.3 Звертання до програми.....	41
4.4 Вхідні і вихідні дані.....	43
4.5 Повідомлення	44
5 Керівництво оператора.....	46
5.1 Призначення програми.....	46
5.2 Умови виконання програми.....	47
5.3 Виконання програми	47
5.4 Повідомлення оператору.....	58
6 Тестування і результати	61
6.1 Опис методики проведення тестування	61
6.2 Результати використання розробленого програмного забезпечення	61
Висновки	78
Перелік джерел посилання	79
ДОДАТОК А Технічне завдання	81
А.1 Вступ	83
А.2 Призначення розробки	83
А.3 Вимоги до програми чи програмному виробу	84
А.3.1 Вимоги до функціональних характеристик.....	84

A.3.2	Вимоги до надійності.....	86
A.3.3	Умови експлуатації.....	87
A.3.4	Вимоги до складі і параметрів технічних засобів.....	87
A.3.5	Вимоги до інформаційної і програмної сумісності	88
A.3.6	Вимоги до транспортування й збереження	88
A.4	Вимоги до програмної документації	89
A.5	Стадії й етапи розробки.....	89
A.6	Порядок контролю й приймання.....	90
ДОДАТОК Б Специфікація		92
ДОДАТОК В Опис програми.....		96
V.1	Загальні відомості.....	98
V.2	Функціональне призначення.....	98
V.3	Опис логічної структури	99
V.4	Використані технічні засоби.....	100
V.5	Виклик і завантаження	101
V.6	Початкові дані, вихідні дані	102
ДОДАТОК Д Текст програми		104

ПЕРЕЛІК УМОВНИХ ТЕРМІНІВ

BPCS – Bit Plane Complexity Segmentation (сегментація бітових площин за складністю);

CPU – Central Processing Unit (центральний процесор);

DOM – Document Object Model (об'єктна модель документа);

HTML – HyperText Markup Language (мова розмітки гіпертексту);

JS – JavaScript (мова програмування);

LSB – Least Significant Bit (найменш значущий біт);

OS – Operating System (операційна система);

RAM – Random Access Memory (оперативна пам'ять);

UI/UX – User Interface / User Experience (користувачський інтерфейс та досвід);

WASM – WebAssembly (бінарний формат інструкцій для браузерів);

W3C – World Wide Web Consortium (Консорціум Всесвітньої павутини);

БЗПД – безпека та захист програм і даних;

ЕОМ – електронна обчислювальна машина;

ОП – опис програми;

ОЗП – оперативний запам'ятовувальний пристрій;

ПЗ – програмне забезпечення;

ПЗП – пояснювальна записка.

ВСТУП

Сучасний стан розвитку інформаційних технологій характеризується постійним зростанням обсягів передачі даних через відкриті мережі, що зумовлює гостру потребу в удосконаленні методів захисту інформації.

На сьогодні практично розв'язаними є задачі криптографічного шифрування, проте вони мають суттєву прогалину, оскільки сам факт наявності зашифрованого трафіку може стати сигналом для цілеспрямованої атаки. У цій галузі активно працюють провідні фахівці та вчені, серед яких варто відзначити Г. Сіммонса, який заклав основи сучасної стеганографії, а також наукові колективи таких компаній, як Microsoft та Adobe, що досліджують цифрові водяні знаки.

Актуальність даної роботи зумовлена необхідністю створення інструментів, що забезпечують конфіденційність не лише змісту, а й самого факту передачі даних у браузерному середовищі.

Підставою для виконання роботи є потреба у впровадженні високоефективних технологій для реалізації складних алгоритмів приховування даних прямо в браузері користувача для забезпечення максимальної зручності використання та швидкості роботи.

Метою роботи є розробка вебзастосунку для вбудовування інформації в цифрові зображення.

Галузь застосування розробленого рішення охоплює сфери конфіденційного корпоративного листування, захисту авторських прав на графічний контент та передачі ключів у багатофакторних системах аутентифікації.

Дана робота має безпосередній взаємозв'язок із дослідженнями у сфері кібербезпеки та оптимізації вебзастосунків, доповнюючи існуючі рішення інструментарієм для локальної обробки стего-об'єктів без залучення серверних ресурсів.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ

Стеганографія, тайнопис, приховування інформації - це техніка приховування інформації всередині іншого повідомлення або об'єкта, з метою уникнення її виявлення [1].

Приховування даних було розроблене з метою ускладнити для користувачів виявлення інформації шляхом маскуванню її у різних носіях, таких як зображення, текст, аудіо та відео. Конфіденційне повідомлення документа, що передається, маскується в носії таким чином, щоб його виявлення ставало складним. Інформація, пов'язана з відправником і одержувачем повідомлення, також може бути прихована таким способом [2]. Вбудовані дані не повинні бути сприйнятними для органів чуття людини, оскільки вони мають бути невидимими та нечутними.

Стеганографія тісно пов'язана з криптографією, але друга має великий недолік - її видно відразу. Зашифровані символи вирізняються своєю вираженою випадковістю. Саме ця помітність становить головну загрозу безпеці криптографії, оскільки щойно зломисник розуміє, що перед ним шифр, він може відразу розпочати розшифрування зашифрованих даних [3].

Метою приховування даних є уникнення привертання підозри до передавання прихованого повідомлення. У разі виникнення підозри ця мета втрачає сенс. Виявлення таких прихованих повідомлень і позбавлення їх корисності є новою галуззю, відомою як атака [4].

Стеганографія передувє сучасному цифровому світові приблизно на 2 500 років. Від самого її зародження мета цієї галузі залишалася незмінною [3].

Гарною демонстрацією віку стеганографії, але абсолютно не єдиною, є картина Вінсента ван Гога "Patch of Grass". В цій картині, за допомогою техніки рентгенівського випромінювання, було знайдено портрет жінки [5]. Оригінальна картина показана на рисунку 1.1, а знайдений портрет показаний на рисунку 1.2 [6].



Рисунок 1.1 - Оригінальна картинка Вінсента ван Гога "Patch of Grass"



Рисунок 1.2 - Знайдений портрет в картині

Методи приховування даних являють собою сукупність різноманітних засобів, зокрема вбудовування інформації про авторські права (водяні знаки) та приховування критично важливої інформації, у різні форми комунікаційних каналів, таких як текст, зображення, аудіо та відео [7].

Всі створені застосунки для стеганографії поділять за трьома основними категоріями, а саме за типом файлу-носія [8], за доменною областю та за принципом використання програми [9]. Схема показана на рисунку 1.3.

Tool name	Concealed secret information	Carrier file type				Technique for concealing secret information	Additional features
		Image file	Audio file	Video file	Text file		
Steghide	Plain text	JPEG,BMP	Wav, AU	–	–	Graphic theoretical technique	Encryption, Hashing
SteganPEG	Any file type	JPEG	–	–	–	Partial decoding technique	Encryption, Password protection
OurSecret 2.5	Any file type	BMP, JPG, PNG	Wav	Mp4	–	–	Encryption
Snow	Plain text	–	–	–	Text	Whitespace	–
S-tool	Plain text	GIF, BMP	Wav			LSB, Large capacity of concealing	Encryption, Hashing, Compression
OpenStego	Plain text	JPEG, JPG, BMP, GIF, PNG	–	–	–	LSB Watermarking	Encryption
DeepSound 2.0	Plain text	JPEG	Wav, Flac	–	–	–	Encryption, Password protection
StegoStick	Plain Text	BMP	Wav	Mp4	PDF, EXE, CHM	–	Encryption
DeEgger Embedder	Any file type	AVI, JPG, PNG	Mp3	Mp4	–	–	Password protection
OpenPuff	Any file type	BMP, JPG, PCX, PNG, TGA	AIFF, MP3, WAV	3GP, MP4, MPG, VOB	PDF, SWF, FLV	DCT	Encryption, Scrambling, Whitening, Encoding
wbStego4	Any file type	JPEG, BMP	Wav	Mp4	ASCII, HTML, PDF	–	Encryption
and Seek	Any file type	BMP,GIF	Mp3	–	–	LSB	Cryptography, Password protection
VSL 1.1	Plain text	GIF	–	–	–	LSB, DCT	Classifiers SVM and BSM
Outguess-Rebirth	Plain text	JPEG	–	–	–	DCT	Cryptography
JSteg	Plain text	JPEG				LSB, DCT	Encryption, Compression
F5 Steganography	Plain text	JPEG	–	–	–	F5 DCT	Password protection

Рисунок 1.3 - Типи застосунків для стеганографії

За типом файлу-носія інструменти стеганографії поділяють на роботу з текстом, зображеннями, аудіо та відео.

Інструменти текстової стеганографії є найпростішими, мають низьку місткість прихованих даних та низький рівень непомітності. Ці засоби використовують текст як файл-носіє для приховування текстової інформації. Застосування таких інструментів створює складнощі при переписуванні, зміні міжрядкового інтервалу, додаванні або видаленні слів.

Інструменти стеганографії зображень значно переважають текстову стеганографію за рівнем безпеки та місткістю приховування інформації. Ці засоби використовують зображення як файл-носіє і дозволяють приховувати як текст, так і інші зображення. У стеганографії зображень прихована інформація залишається непомітною завдяки незначним змінам значень кольорів. Для підвищення безпеки прихованих даних зазвичай застосовується додаткове заповнення, padding, навколо секретної інформації.

Інструменти аудіостеганографії забезпечують високий рівень безпеки та великий обсяг прихованої інформації. Ці засоби використовують нечутні частоти аудіофайлу як носії та дозволяють приховувати текстову, графічну або аудіоінформацію.

Інструменти відеостеганографії дозволяють приховувати секретну інформацію як у нечутних частотах, так і у відеокадрах. Ці засоби використовують декілька випадкових відеокадрів як файл-носіє і можуть приховувати будь-який цифровий медіафайл. Випадкове приховування секретної інформації підвищує як рівень безпеки, так і місткість цих інструментів [9].

Інструменти стеганографії також поділяють за їх основою, а саме на основі просторової області та на основі області перетворення.

У просторовій області певні біти файлів-носіїв замінюються секретною інформацією. До поширених методів просторової області належать: PVD (диференціація значень пікселів), RPE (випадкове вбудовування пікселів), EBE (вбудовування інформації на основі країв), MPD (багатопіксельна диференціація), LSB (метод найменш значущого біта) тощо.

У частотній області (області перетворення) цифровий медіаносій перетворюється з однієї області в іншу для приховування секретної інформації. До таких методів належать DCT (дискретне косинусне перетворення), DWT (дискретне вейвлет-перетворення), IWT (цілочисельне вейвлет-перетворення) та DFT (дискретне перетворення Фур'є) [9].

Також інструменти стеганографії поділяють на основі способу їх використання, а саме через графічний інтерфейс, або ж через командний рядок [9].

В стеганографії зображення демонструють дуже гарні показники щодо приховування інформації, тому вони є найбільш широко використовуваним варіантом. Водночас основною науковою проблемою в стеганографії зображень є досягнення надійних критеріїв, зокрема місткості, обчислювальної складності, стійкості, захисту від підроблення та прозорості, як показано на рисунку 1.4 [7].

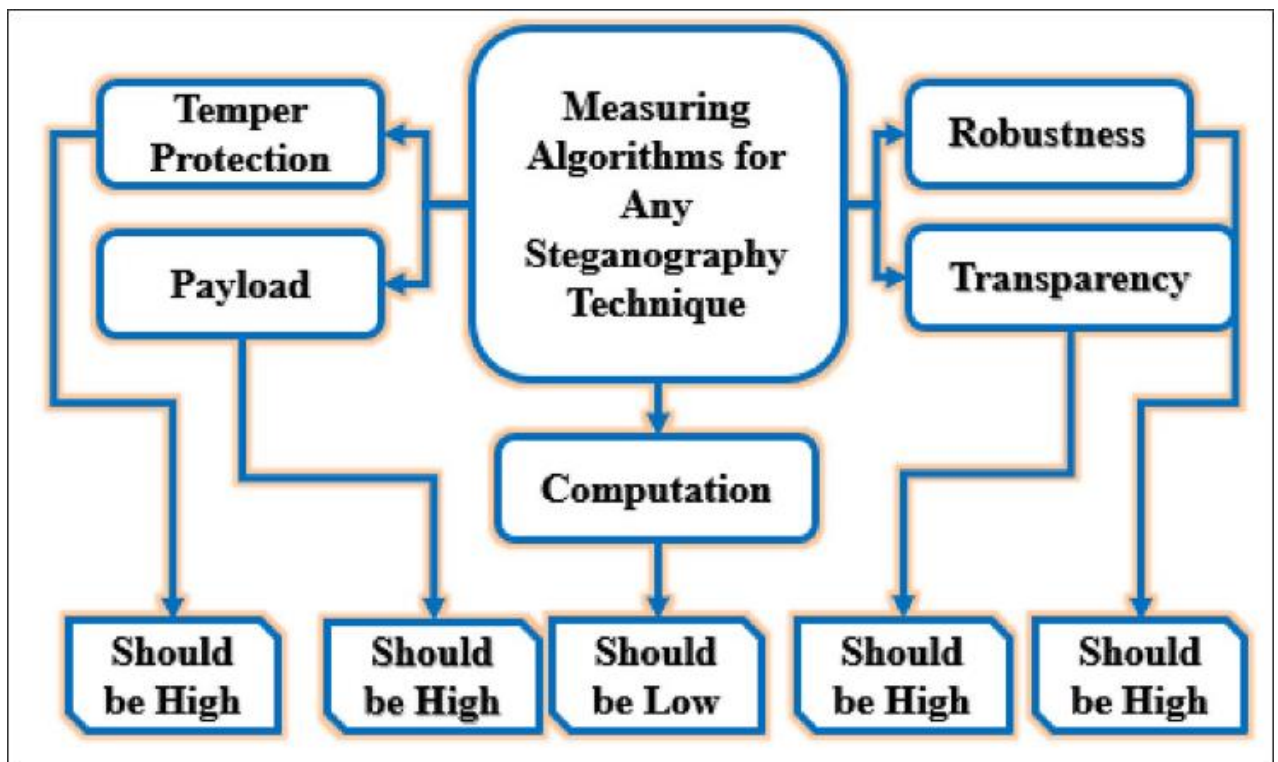


Рисунок 1.4 - Стеганографія рисунків

Доцільно визначити деякі ключові поняття, пов'язані зі стеганографією зображень:

- корисне навантаження (payload) означає зашифрований текст, який вбудовується в контейнерний об'єкт;
- безпека (security) визначає, наскільки оригінальне та стего-зображення є стійкими до статистичних атак, таких як обрізання, модифікація, масштабування тощо;
- сприйняття (perception) описує візуальні властивості контейнерного зображення після вбудовування прихованого повідомлення;
- захист від підроблення (tamper protection) є критично важливим аспектом після забезпечення конфіденційності секретної інформації в контейнерному зображенні, оскільки у разі спроби несанкціонованого вилучення повідомлення цей захист має бути достатньо високим для протидії атакам на стего-зображення;
- обчислювальна складність (computation) характеризує час, необхідний для процесів вбудовування та вилучення інформації, і має бути мінімізованою, щоб ці процеси виконувалися за найменший можливий час [8].

У середині 1980-х років з'явилася техніка, відома як підстановка найменш значущого біта (Least Significant Bit Substitution). Метод LSB приховує дані в тій частині інформації, яка є найменш важливою для якості зображення, тим самим мінімізуючи його спотворення. Ця інформаційно-надлишкова область зображення складається з найменш значущих бітів кожного пікселя.

Найменш значущим бітом кожного пікселя є найправіший біт в двійковому представленні. Означений біт на рисунку 1.5 має найменший вплив на колір пікселя, який задає послідовність. Тобто, якщо змінити цей біт на 0, піксель майже так само виглядатиме червоним, як і до зміни [3].

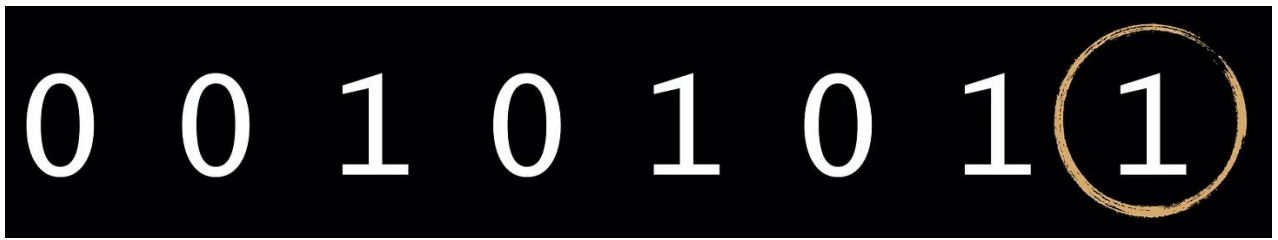


Рисунок 1.5 - Найменш значущий біт

Щоб зрозуміти, наскільки незначною є інформація цього біта праворуч, то при пересуванні від біта до біта ліворуч їхній вплив подвоюється. У 8-бітовій бінарній послідовності зображення найлівіший біт у 128 разів сильніше впливає на те, як піксель сприймається людським оком, ніж найправіший. І це стосується лише одного пікселя. Тобто, зміна останнього біта змінює лише $1/256$ частину значення пікселя — настільки мало, що практично ніхто цього не помічає [3]. Це показано на рисунку 1.6.

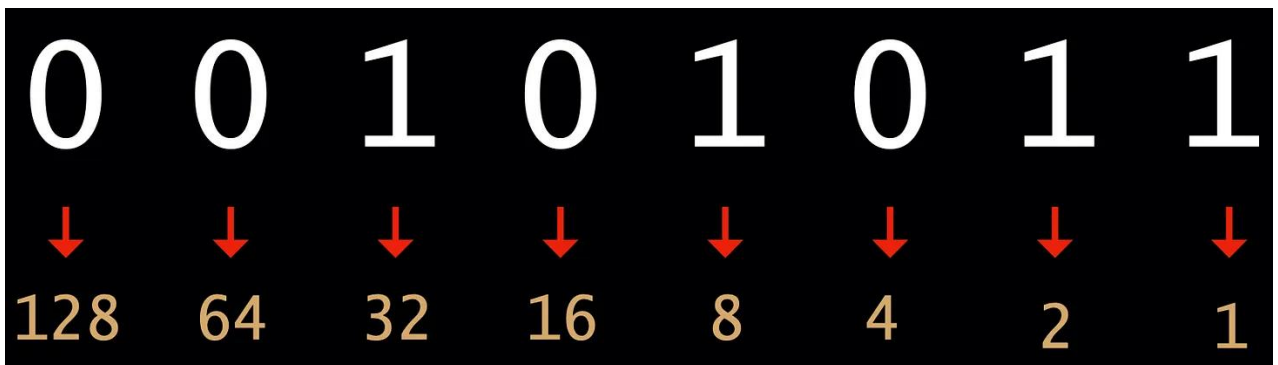


Рисунок 1.6 - Важливість кожного біта

Модуляція найменш значущого біта не призводить до помітної для людського ока різниці, оскільки амплітуда зміни є надто малою. Інші методи обробляють повідомлення за допомогою псевдовипадкової шумової послідовності перед або під час вставки у носійне зображення. Перевагою LSB-вставки є її простота, і багато методів використовують саме цей підхід. LSB-вставка також забезпечує високий рівень перцептивної прозорості.

Проте існує багато слабких місць щодо стійкості, захисту від підробки та інших питань безпеки. Кодування в LSB надзвичайно чутливе до будь-якого

виду фільтрації або маніпуляцій із носійним зображенням. Масштабування, обертання, обрізка, додавання шуму або втрата стиснення носія майже напевно зруйнують приховане повідомлення. Крім того, якщо зломисник підозрює наявність повідомлення, він/вона може легко видалити його шляхом занулення всього LSB-шару з мінімальними змінами перцептивної якості зміненого носія. Конкретна LSB-техніка дозволяє вбудовувати дані у зображення, аудіо та текстові носії. На рисунку 1.7 наведено приклад застосування LSB для RGB-пікселя [4].

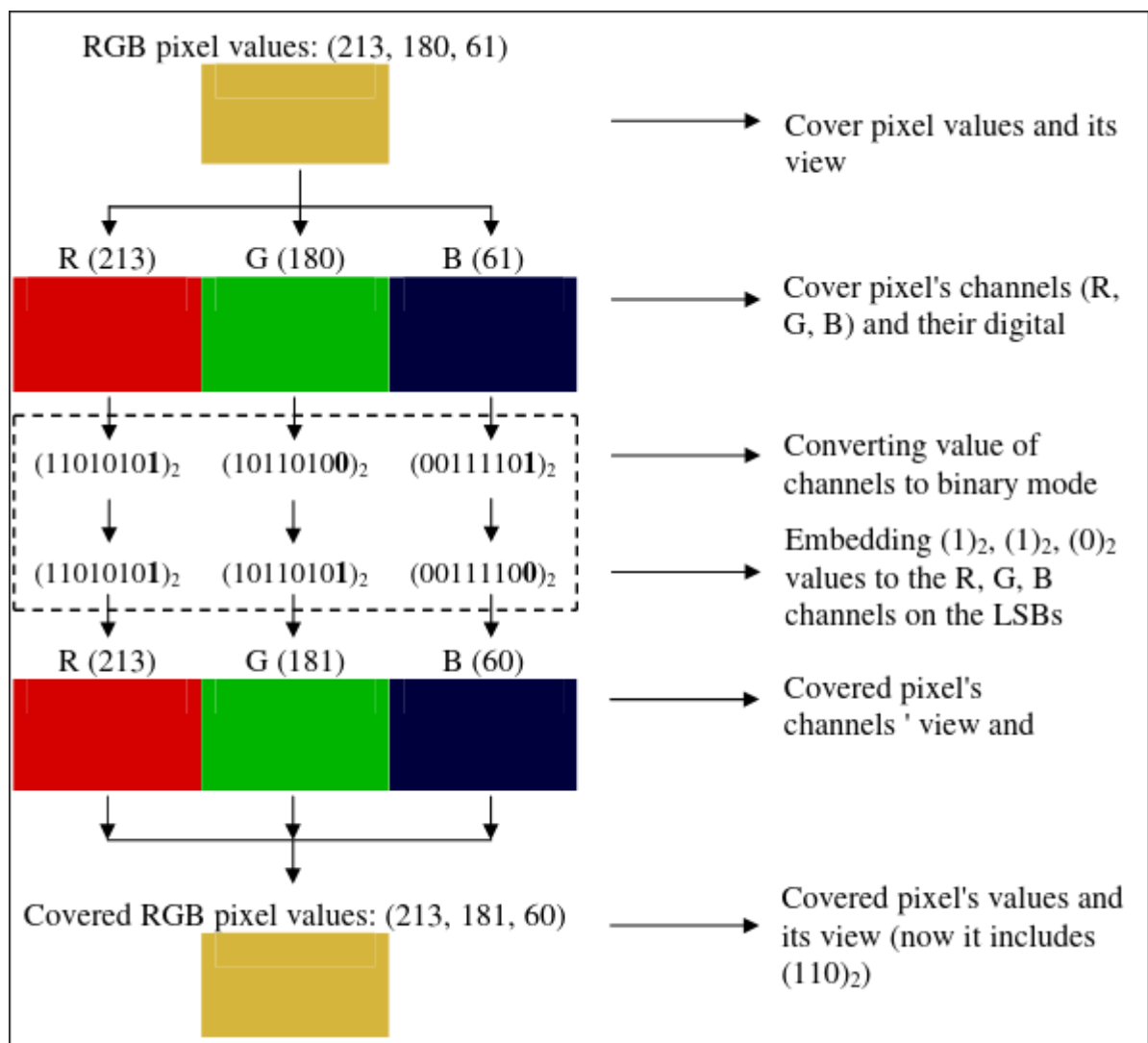


Рисунок 1.7 - Приклад застосування LSB для RGB-пікселя

У типовому зображенні повнокольорового формату, яке використовує 24 біти на піксель, як показано на рисунку 1.7, кожному кольоровому

компоненту виділено 8 бітів. У градаційних зображеннях використовується 8 біт на піксель. Цифрове зображення складається з матриці значень кольору та інтенсивності.

Файли кольорових зображень можуть бути досить великими, проте для зменшення обсягів зберігання та потреб у передачі даних розроблено методи стиснення. Файли формату Bitmap (.bmp) та GIF (.gif) використовують безвтратний алгоритм стиснення. При безвтратному стисненні розпаковане зображення ідентичне оригінальному (тобто такому, яке було до стиснення). Файли JPEG (.jpg) застосовують втратний алгоритм стиснення, який лише апроксимує зображення. При втратному стисненні розпаковане зображення майже таке саме, але не повністю ідентичне оригіналу.

Хоча приховування даних можна застосовувати й до стиснених зображень, воно є складнішим у порівнянні з приховуванням у необроблених зображеннях (наприклад, bmp, raw тощо) [4].

Щоб підвищити планку методу найменш значущого біта, Ісон і Кавагучі використали недоліки людського зору та розробили метод Сегментації складності бітових площин (BPCS). Для максимальної непомітності вони розробили метод поділу зображення на дві окремі зони - «інформативну» та «шумоподібну». Інформативна область складається з простого паттерна, який око розпізнає миттєво; шумоподібна область містить настільки складний паттерн, що наш мозок його просто не реєструє [3].

Кожне зображення містить інформативні та шумові зони, які стають математично доступними для мапування на мікроскопічному рівні. На рисунку 1.8 показано приклад [3].

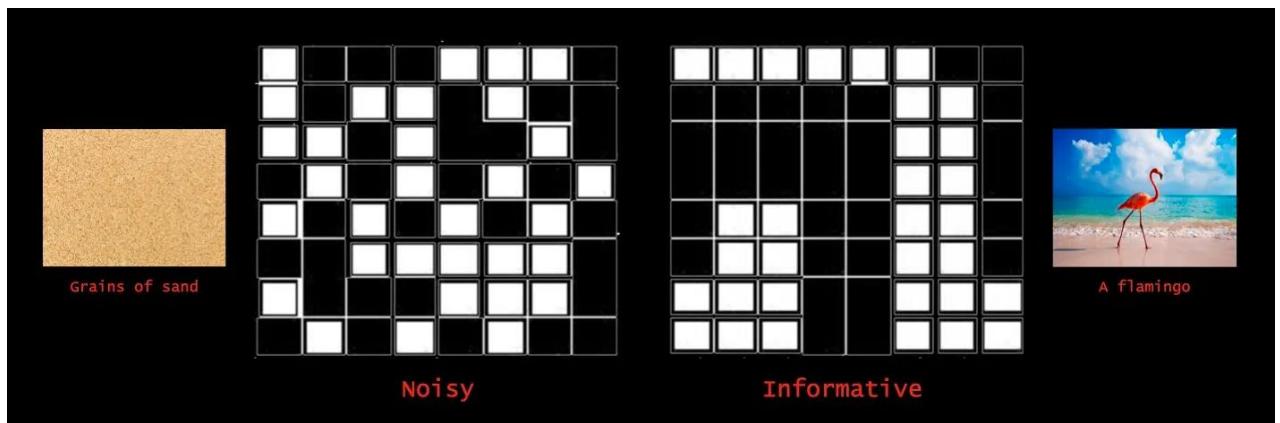


Рисунок 1.8 - Приклад інформативної та шумової зон

Для мапування цих зон стеганографи розрізають зображення на 8 шарів. Якщо розкласти їх поруч, то вони будуть виглядати як на рисунку 1.9.

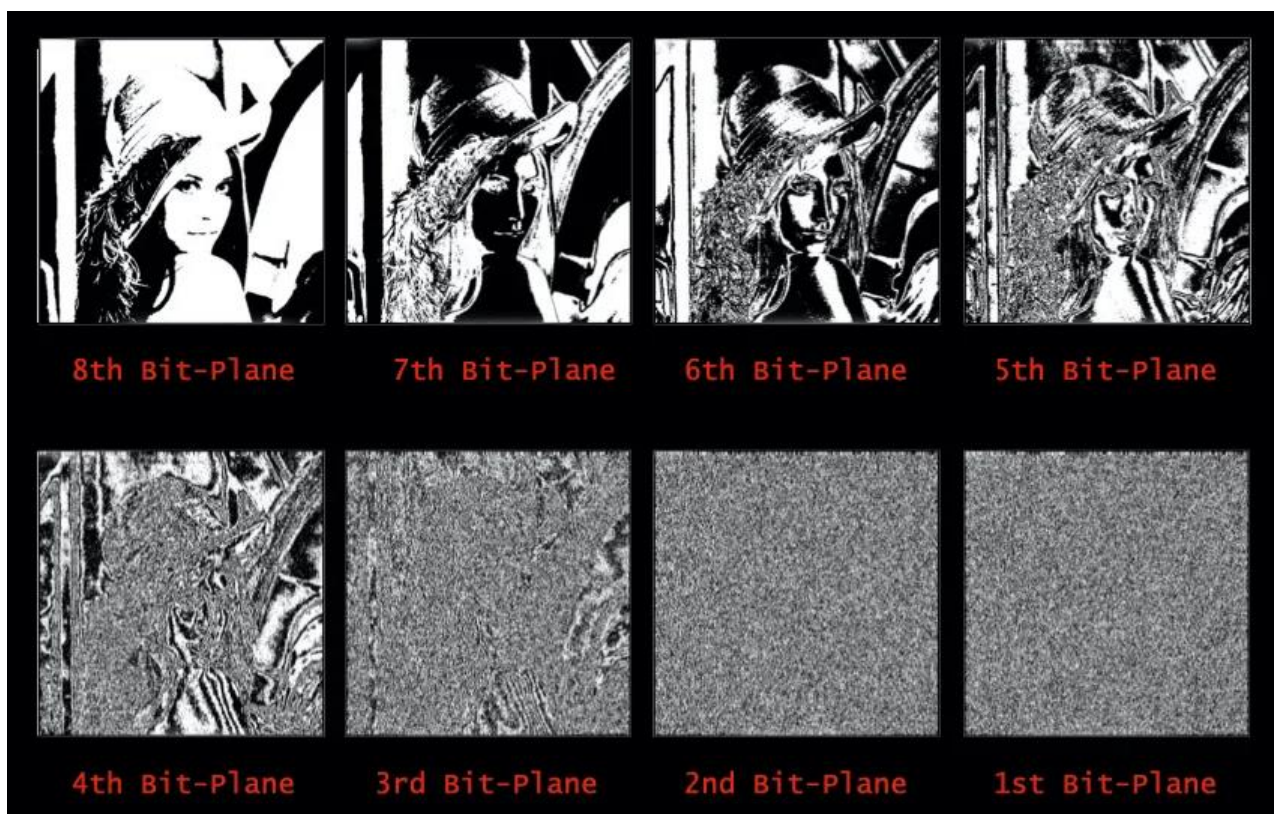


Рисунок 1.9 - Приклад шарів зображення

Тепер, після розбиття на біти, легко ідентифікувати інформативні та шумові зони. Щоб зробити це ще простішим, на зображення накладається сітка, і кожній комірці присвоюється значення шуму. Це досягається шляхом

вимірювання рівня контрастності пікселів всередині кожного сегмента. Це також відомо як граничне значення зображення. Сітка показана на рисунку 1.10 [3].

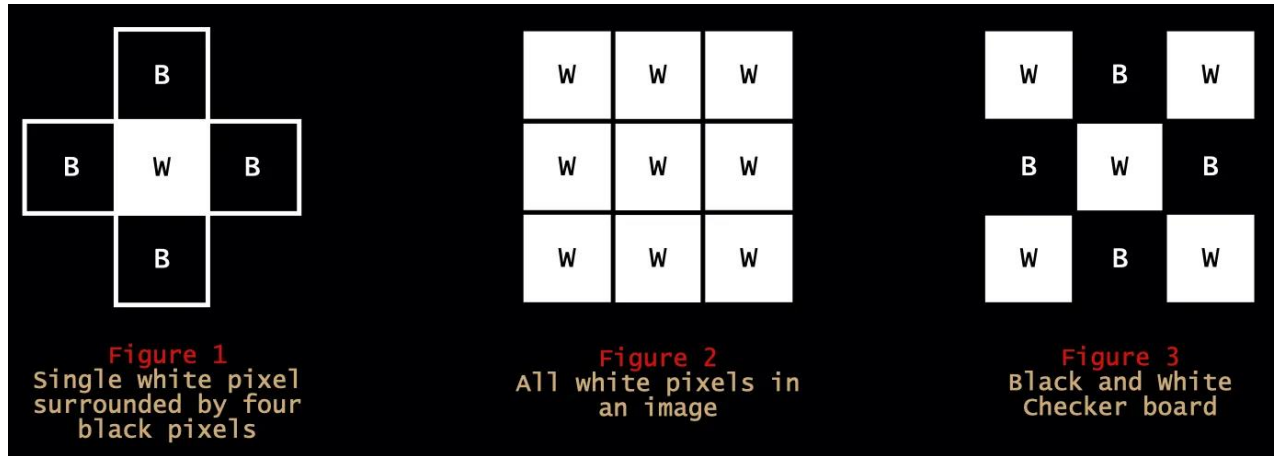


Рисунок 1.10 - Сітка розбиття

На рисунку 1.10, фігурі 1 білий піксель оточений 4 чорними. Це дає йому граничне значення 4, оскільки є чотири контрастні межі. На фігурі 2 немає чорних пікселів, що межують з білими. Це дає граничне значення 0. Фігура 3 має граничне значення 12. Тобто фігура 2 є інформативним, тоді як фігура 3 — це суцільний шум.

Ісон та Ейджі запропонували вбудовувати секретні дані в зображення шляхом їхнього зіставлення з шумовими зонами, які зазвичай становлять 45% зображення.

Щоб замаскувати інформативну область секретного зображення в шумовій зоні, її потрібно перетворити на шум. На піксельному рівні цей процес кон'югації показаний на рисунку 1.11 [3].

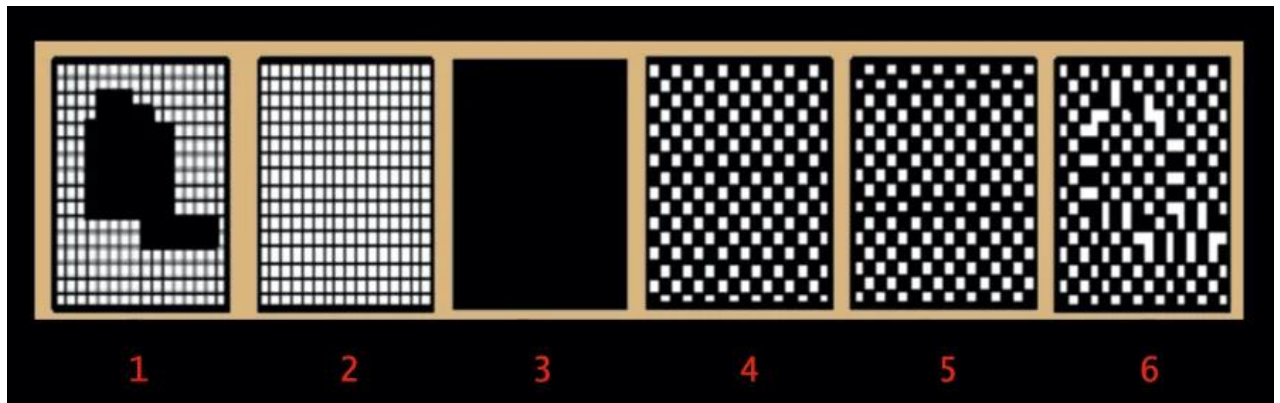


Рисунок 1.11 - Перетворення секретного зображення на шум

На рисунку 1.11, 1 - це фрагмент початкового зображення. Він є візуально помітним. Зіставивши його з фоновим паттерном (4) та переднім планом (5) контейнера, він може розміститися непоміченим.

Носій перетворюється у відтінки сірого. Потім обидва зображення розділяються на площини. Для блоків обох зображень обчислюється показник складності. Виконується кон'югація «простих» або «інформативних» блоків секретного зображення, що перетворює їх на шум. Нарешті, секретне зображення вбудовується в носій для створення фінального стего-зображення. Це показано на рисунку 1.12 [3].

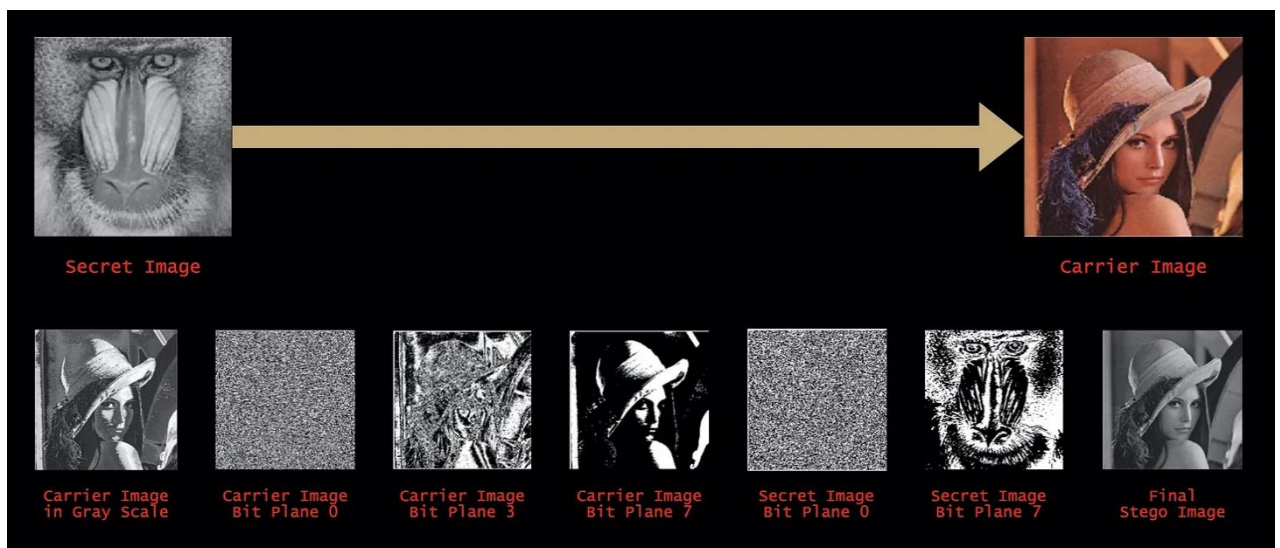


Рисунок 1.12 – Приклад вбудовування через BPCS

Аналіз існуючих методів та програмних засобів показав, що попри велику кількість рішень, вони мають суттєві обмеження. Простий метод LSB, хоч і легкий у реалізації, демонструє низьку стійкість до найменших маніпуляцій із файлом, тоді як складніші алгоритми, як-от BPCS, потребують специфічної підготовки даних і часто відсутні у готових безкоштовних утилітах.

Більшість проаналізованих аналогів мають закритий початковий код або обмежений функціонал, що не дозволяє гнучко налаштовувати параметри вбудовування чи комбінувати різні типи приховування в межах одного застосунку. Це зумовлює необхідність розробки власного програмного забезпечення, яке дозволить усунути ці недоліки, забезпечивши баланс між простотою використання, високою місткістю даних та надійністю захисту від виявлення.

2 ВИБІР ТА ОБҐРУНТУВАННЯ СТРУКТУРИ СИСТЕМИ, ЯКА ПРОЕКТУЄТЬСЯ

2.1 Вибір інструментарію розробки програмного забезпечення

Оскільки, згідно з вимогами з технічного завдання, проєкт повинен працювати в браузері та не мати логіки на сервері для розробки було обрано:

- мову розмітки HTML та мову стилів CSS для реалізації інтерфейсу користувача;
- мову програмування JavaScript для взаємодії з браузером;
- технологію WebAssembly;
- мову програмування Golang для основної логіки.

Для написання будь-якого вебсайту, чи то сервісу або ж додатку в будь-якому випадку використовується мова гіпертекстової розмітки.

Мова гіпертекстової розмітки (далі HTML) – найбільш фундаментальний будівельний блок для всього вебу. HTML визначає значення та структуру вебконтенту.

"Гіпертекст" відноситься до посилань, які з'єднують вебсторінки між собою, як в межах одного вебсайту, так і між вебсайтами. Посилання є фундаментальним аспектом Інтернету. Завантажуючи вміст в Інтернет та посилаючись на сторінки, створені іншими людьми, ви стаєте активним учасником всесвітньої павутини.

HTML використовує "розмітку" для анотування тексту, зображень та іншого контенту для відображення веббраузером [10].

Основною та єдиною мовою програмування доступною в браузері є JavaScript, тому її використання для розробки застосунку є необхідним.

JavaScript – легка, JIT-компільована, об'єктноорієнтована мова з функціями першого класу, а також найвідоміша скриптова мова для вебсторінок, але також використовується в багатьох не браузерних оточеннях.

JIT-компільована означає, що сирцевий код програми не перетворюється повністю у машинний код для виконання, як у звичайних компільованих, а спочатку переводиться у байт-код, в тому числі на цьому етапі виконується синтаксичний аналіз коду, а після програма виконується рядок за рядком з допомогою JavaScript Virtual Machine [11].

JavaScript є об'єктноорієнтованою мовою через те, що в основі мови лежить поняття об'єкта – певної сутності, яка об'єднує в собі поля (дані) і методи (виконувани об'єктом дії).

JavaScript має функції першого класу, бо розглядає функції як об'єкти першого класу, тобто сутність, яка може бути побудована під час виконання програми, передаватись як параметр, повертатись з підпрограми, або присвоюватись змінній.

Це означає, що JavaScript підтримує передачу функції як аргументів інших функцій, повернення їх як результат інших функцій, присвоювання їх змінним або збереження в структурах даних JavaScript може виконуватись як за допомогою браузерного оточення або, наприклад, NodeJS.

NodeJS та більшість браузерів на поточний момент використовують як рушій V8 який компілює JavaScript код безпосередньо у власний машинний код, минаючи стадію проміжного байт-коду, а також ефективно керує пам'яттю.

JavaScript - прототипно-орієнтована мультипарадигменна мова сценаріїв. Прототипно-орієнтованою, бо в цій мові відсутнє поняття класу, а повторне використання (успадкування) проводиться шляхом клонування наявного примірника об'єкта – прототипу.

Мультипарадигменна – бо підтримує декілька підходів або стилів програмування: динамічний, об'єктноорієнтований, імперативний та функціональний стилі.

JavaScript може функціонувати і як процедурна, і як об'єктноорієнтована мова. Об'єкти можна створювати програмно під час виконання, шляхом приєднання методів і властивостей або порожніх об'єктів під час виконання, на відміну від синтаксичних визначень класів у мовах, що компілюються, таких як

C++ або Java. Після того як об'єкт був створений, він може бути використаний як план (або прототип) для створення схожих об'єктів.

Динамічні можливості JavaScript включають створення об'єктів під час виконання, змінну кількість параметрів, динамічне створення скриптів (за допомогою eval), перебір об'єктів (за допомогою for ... in), відновлення вихідного коду (програми на JavaScript можуть декомпілювати тіла функцій назад у вихідний код).

JavaScript є динамічно типізованою мовою програмування, тобто основна частина перевірок типів виконується під час виконання програми, а не під час компіляції. У динамічній типізації значення мають типи, а змінні – ні, тому змінна може містити значення будь-якого типу.

Динамічність дозволяє простіше писати програму, але може заплутати розробника, а в деяких ситуаціях помилка в коді через типізацію може випадково попасти до клієнта та викликати масу помилок [12].

Для реалізації основної логіки застосунку було вирішено не використовувати JavaScript через ряд причин.

Ця мова програмування не має гарних та зручних інструментів для роботи з окремими пікселями картинок, їх зміни. Не має інструментів для роботи з окремими байтами. Не має вбудованих інструментів для роботи з файлами, можливостей їх читати з файлової системи тощо.

Ще одна причина полягає в тому, що для запуску JavaScript коду на сервері потрібно встановлювати додаткове велике програмне забезпечення, наприклад, NodeJS. А при цьому застосунок повинен мати можливість в майбутньому розширюватись, в тому числі до консольної та можливо навіть нативної версії під різні платформи.

Ці причини є важливими, але не є основними. Основна причина полягає у швидкості роботи JavaScript в порівнянні з іншими мовами програмування. JavaScript, коли він виконується з використанням двигуна V8 є дуже швидким, але в той самий інші більш низькорівневі мови програмування все одно будуть працювати швидше.

На рисунку 2.1 можна побачити порівняння швидкості виконання JavaScript з іншими мовами програмування.

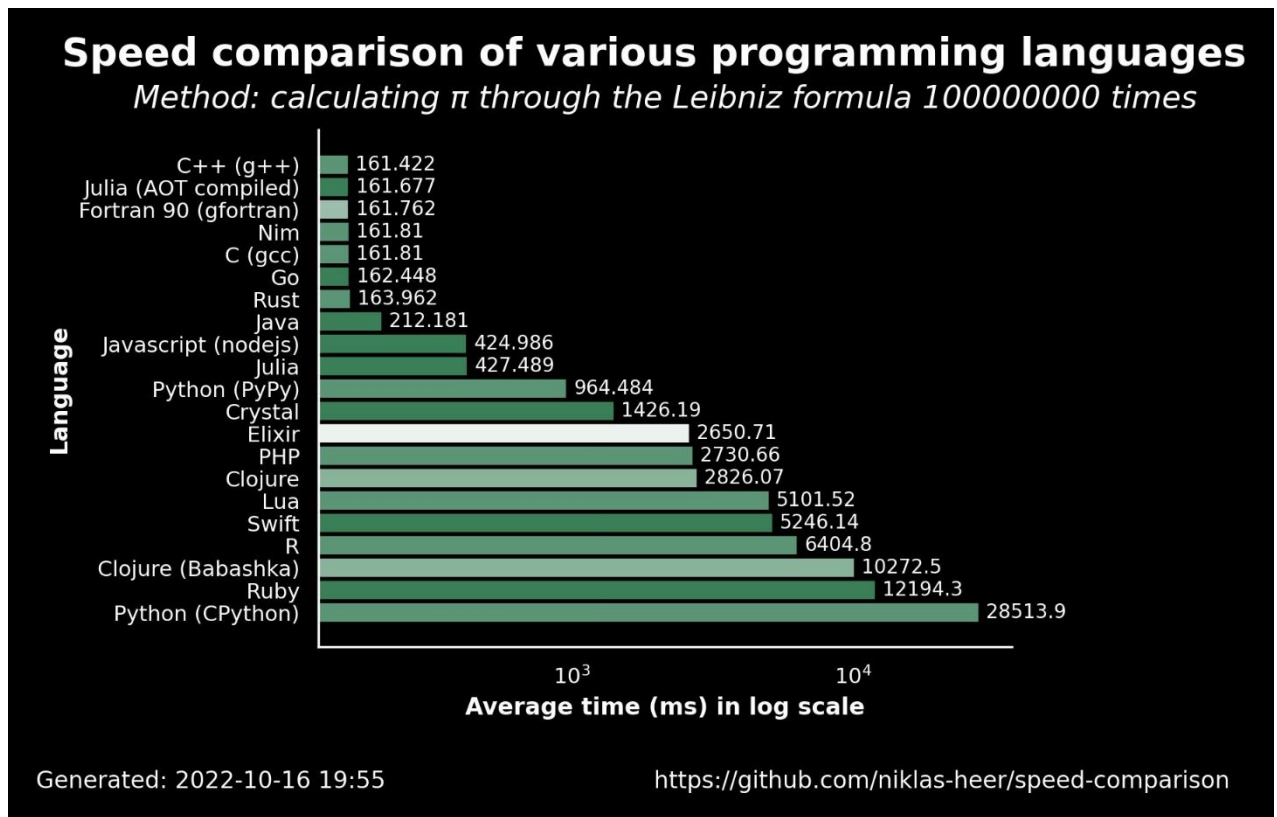


Рисунок 2.1 - порівняння швидкості виконання JavaScript з іншими мовами програмування

Враховуючи всі згадані проблеми з JavaScript для виконання цього завдання було обрано технологію WebAssembly.

WebAssembly дозволяє запускати в браузері спеціально створені компіювані версії програм з будь-яких інших мов програмування, якщо для них існує компілятор. Це працює дуже схожим чином зі звичайними програмами, які спочатку компілюються в бінарний вигляд, а після виконуються процесором.

Наразі JavaScript є єдиною мовою програмування яка підтримується в браузерах нативно. Навіть при використанні WebAssembly нам все ще потрібно використовувати JavaScript для підключення та взаємодії з WebAssembly. Наприклад, нам все ще потрібно використовувати JavaScript для

прослуховування подій та попередньої обробки інформації перед тим, як надіслати її в WebAssembly.

З цього всього ми можемо побудувати гарну ментальну модель: користувач взаємодіє з браузером, JavaScript слухає події та запускає WebAssembly коли це необхідно, а після відображає результат виконання користувачу. Така схема є простою для розуміння та реалізації.

Складність застосунку це одна з великих проблем з якою щоденно борються розробники. Чим складніша логіка, тим складніше та дорожче її підтримувати, додавати новий функціонал або змінювати попередній [13].

Як мову програмування для основної логіки було обрано Golang. Ця мова показує чудові показники у швидкості її виконання, є простою в її синтаксисі та надає велику кількість можливостей. Golang є однією з основних мов програмування, які гарно працюють з WebAssembly.

Мова була створена в Google у 2007 році та наразі її використовує велика кількість компаній по всьому світі.

Golang має велику бібліотеку стандартних функцій, яких достатньо, щоб розробити застосунок з повного нуля без необхідності встановлення додаткового функціоналу. Також вона має вбудований garbage collector та підтримку покажчиків на пам'ять

Наприклад, Golang має чудові вбудовані інструменти для роботи з картинками та файлами. А саме функціонал читання та запису файлів, функціонал розпізнавання типу зображення, зчитування інформації про зображення з масиву байтів, можливість редагування окремих пікселів зображення та переведення зображення в інші формати

Наразі основна ніша використання Golang полягає в невеликих та середніх сервісах, які гарно виконують свою задачу. Причиною цього є інша велика перевага Golang - можливість зручної, швидкої та зрозумілої роботи в паралелі.

Але через її простоту та зручність на ній можливо розробляти і величезні сервіси, наприклад, Google "Chrome Content Optimization Service" повністю написаний на Golang [14].

Ця мова програмування є майже ідеальною для використання в цій курсовій роботі враховуючи описані переваги, а також потрібно враховувати, що Golang можна запускати не тільки в WASM. Тобто, в майбутньому досить просто можна буде створити CLI або Desktop версії застосунку.

Другим варіантом було обрати мову програмування Rust, але вона не має такої підтримки роботи з зображень та є набагато складнішою у використанні та розробці.

Важливо розуміти, що хоч JavaScript і є повільнішим в порівнянні з Golang, але нас все одно потрібно використовувати WebAssembly в ситуаціях, коли ми працюємо з великою кількістю інформації. JavaScript є дуже оптимізованою мовою, а тому на малій кількості значень може працювати навіть швидше, а ніж WebAssembly, це показано на рисунку 2.2. Саме тому вибір Golang та WebAssembly проводився не тільки на основі швидкості, а також на основі зручності, підтримки та спільноти.

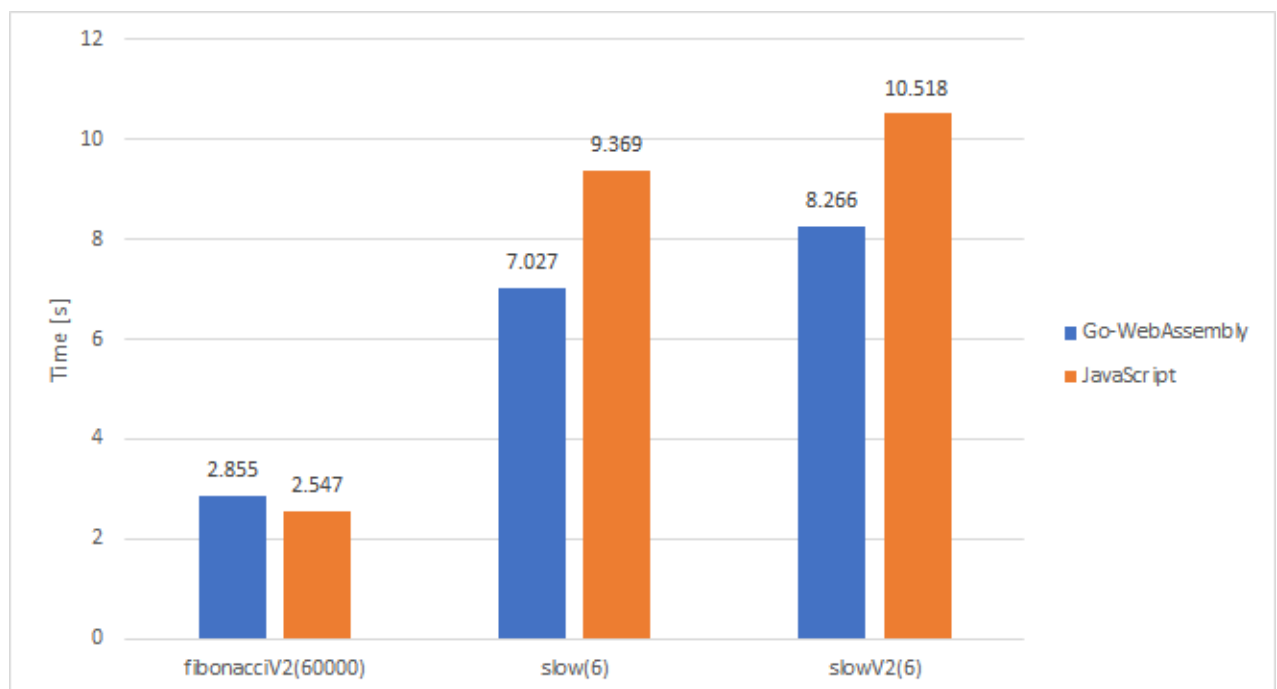


Рисунок 2.2 – Порівняння швидкості виконання JavaScript та Go-WASM

Як основне середовище розробки програму було обрано Neovim.

NeoVim – це розширюваний текстовий редактор, похідний від Vim, який відомий своєю швидкістю, ефективністю та потужними можливостями налаштування.

Хоча NeoVim не є традиційною IDE у повному розумінні (наприклад, як Visual Studio Code або WebStorm), він, завдяки широкому набору плагінів та інтеграцій, дозволяє створити повноцінне робоче середовище для розробки, включаючи автодоповнення, налагодження, інтеграцію з Git та лінери. Його легковажність та висока продуктивність є перевагами для оптимізації робочого процесу розробника [15].

В якості систему контролю версіями був обраний Git. Система контролю версій - це система, що записує зміни у файл або набір файлів протягом деякого часу, так щоб розробник зміг повернутися до певної версії пізніше.

У 2005 році відносини між спільнотою розробників ядра Linux і комерційною компанією, що розробила BitKeeper почали псуватись, і безкоштовне використання продуктом було скасовано. Це підштовхнуло розробників Linux (і зокрема Лінуса Торвальдса, автора Linux) розробити власну систему, ґрунтуючись на деяких з уроків, які вони дізналися під час використання BitKeeper [16].

2.2 Структурна схема програми та опис модулів розроблюваної системи

Структурна схема програми складається з двох основних частин, а саме з клієнтської частини на JavaScript, з якою напряду взаємодіє користувач з браузера та внутрішньої, яка написана з використанням Golang

Схема внутрішньої логіки показана на рисунку 2.3.

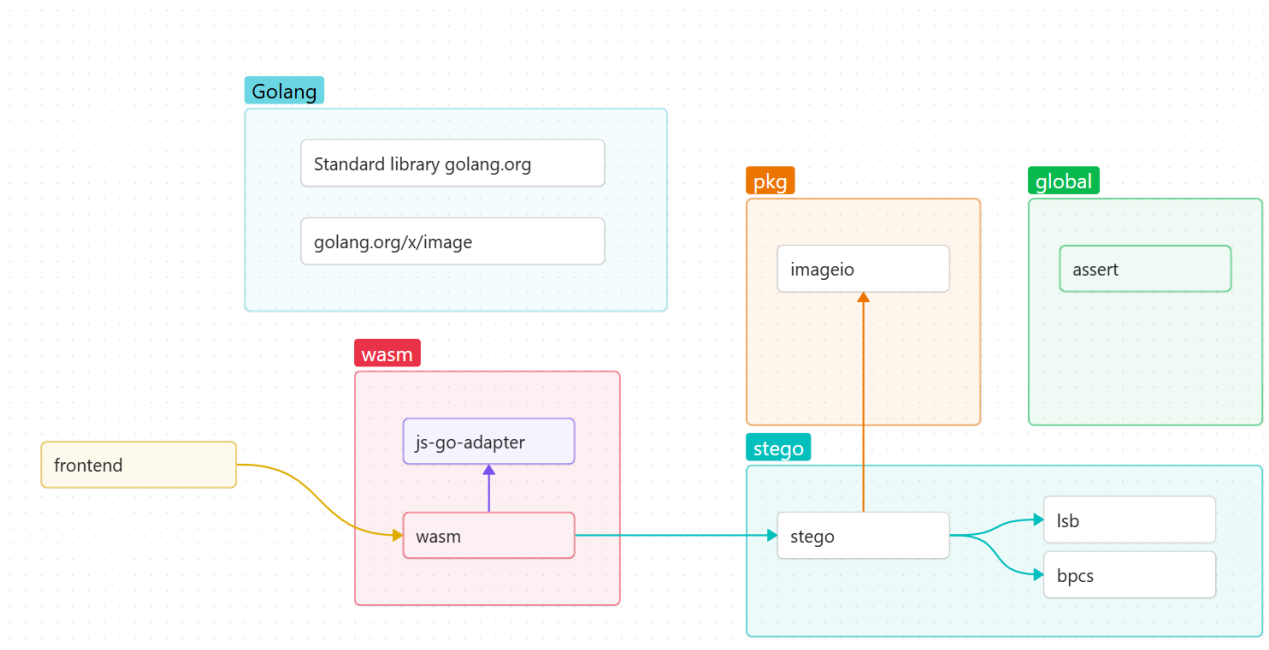


Рисунок 2.3 – Схема внутрішньої структури логіки застосунку

Як можна побачити зі схеми весь застосунок поділений на модулі, деякі модулі залежать від інших модулів, а також всі модулі належать якійсь групі. Групування в тому числі буде відображати внутрішню структуру коду, структуру директорій та файлів.

Важливо зауважити, що кожен модуль на рисунку X1 це окремий файл, але в мові програмування Golang модулем є вся директорія, тому буде більш правильно сказати, що модулями є саме групи, які показані на схемі.

Першою групою є глобальні пакети "global" які можуть використовуватись в усіх інших модулях. Вони є незалежними та можуть вільно використовуватись в тому числі в інших проєктах.

Наразі ця група містить модуль "assert" який представляє набір функцій для перевірки стану програми на наявність критичних помилок зі структурами даних, які ніколи не повинні бути статись під час її виконання.

Наприклад, якщо функція очікує, що перший аргумент буде містити масив з двох елементів, але з якоїсь причини приходить масив з одним або трьома елементами, то це як раз ситуація, яка не повинна ніколи статись та застосунок видасть помилку. Подібні помилки повністю зупиняють роботу застосунку, оскільки вони є критичними.

Згідно з правилами написання коду в NASA ці перевірки повинні застосовуватись лише там, де звичайні механізми мови програмування не зможуть автоматично перевірити дані [17].

Також ця група пакетів може містити обгортку для зручного логування інформації, додаткові системи налаштування системи тощо.

Наступна група "pkg" містить в собі звичайні пакети, які не використовуються по всьому проєкту, а необхідні для одного або декількох модулів.

Ця група містить в собі пакет для зручної роботи з перетворенням картинок "imageio".

Він має дві основні функції:

- функцію перетворення масиву байт в спеціальний загальний тип Golang для картинок "image.Image";
- функцію перетворення загального типу "image.Image" в масив байт.

При завантаженні картинки в програму вона початково передається в вигляді масиву байт та строки, яка визначає її тип. Далі, ця інформація переводиться в один загальний тип, який використовується в Golang для роботи з картинками. Це дуже важливий етап, оскільки саме тут ми перевіряємо чи є картинка правильною, чи є в ній проблеми та чи відповідає вона формату.

Далі над цією загальною картинкою виконуються перетворення, а саме методи приховування інформації.

Після приховування загальний тип картинки переводиться назад в масив байт, який і буде показуватись користувачу. Це також важливий етап, оскільки саме на ньому правильно встановлюються всі заголовки вихідного зображення.

Наступною групою є основний функціонал приховування, який називається "stego". Ця група містить три модулі.

Основний модуль, який також називається "stego" становить собою API для використання застосунку. Саме тут виконується виклик пакетів з групи "pkg", виклик потрібних методів стеганографії та встановлення додаткових

параметрів виконання системи. Інші два модулі містять реалізацію методів LSB та BPCS.

Модуль "stego" є основним модулем всієї системи. Ми можемо використати цей модуль не тільки через WASM, а в тому числі багатьма іншими способами. Наприклад, ми можемо створити CLI яка буде зчитувати інформацію з файлів та викликати цей модуль, ми можемо використати цей модуль в інших локальних проєктах, а також, використовуючи механізми Golang, встановити його як незалежну бібліотеку в будь-який проєкт.

Наступна група модулів це "WASM". Це основна точка входу в застосунок яка напряду взаємодіє з браузером та JavaScript. Ця група містить два модулі, а саме "wasm" як основний, а також додатковий "js-go-adapter" для переведення типів даних з JavaScript в Golang та навпаки.

Ця архітектура є достатньо простою, а тому в неї просто вносити зміни та додавати нову логіку. Наприклад, якщо ми захочемо додати новий метод для приховування інформації нам буде достатньо виконати його реалізацію, або встановити вже готову, та створити для нього API. Цього буде достатньо для використання встановленого методу.

Також в застосунку використовуються стандартні пакети мови програмування Golang, наприклад "image", "image/png", "fmt" тощо. Додатково було встановлено пакет "golang.org/x/image" для роботи з bmp та зображеннями та додатковими можливостями в їх обробці. Пакети Golang X є частиною Go Project, але знаходяться поза основною гілкою Go.

3 ОСНОВНІ РІШЕННЯ ЩОДО РЕАЛІЗАЦІЇ КОМПОНЕНТІВ СИСТЕМИ

Під час проєктування системи в її основу закладалась її надійність та стійкість до всіх можливих проблем та помилок. Основним механізмом для знаходження подібних помилок є assertions – твердження, які повинні бути завжди правдиві. Такі твердження пишуться прямо в коді та виконуються при кожному виклику функції. Згідно зі стандартами NASA assertions не повинні перевіряти логіку, яку може перевірити компілятор або інша система [17].

Вони спеціально використовуються для перевірки саме помилок, які компілятор не може перевірити. Наприклад, у нас є функція яка приймає масив чисел та цей масив, згідно внутрішньої логіки системи повинен мати перші п'ять елементів які дорівнюють нулю. Таку логіку достатньо складно, або навіть не можливо, перевірити з використанням тільки можливостей компілятора.

В якості іншого прикладу можна навести авторизацію в деяких застосунках. Зазвичай в функції деавторизації пишеться умова, що якщо користувач не авторизований, то ми не робимо нічого, але при використанні assertions ми можемо явно сказати, що функція деавторизації не повинна викликатись взагалі, якщо користувач не був авторизований.

Це інший підхід до написання програм. Якщо одне з тверджень не спрацює весь застосунок перестане працювати, оскільки assertion в більшості випадків викликає критичну помилку. Це підхід використовуючи який розробники можуть бути впевнені в тому, що їх код буде працювати коректно, а якщо щось пішло не так, то швидко знайти проблему.

Другим моментом є підхід до написання коду вебсторінки на JavaScript. Сама ця мова програмування є динамічною та не надає зручні можливості для перевірки всіх типів, оголошення інтерфейсів, тощо. Також при використанні JavaScript ми не можемо бути впевнені в типах, які ми отримуємо, оскільки,

наприклад, ми можемо очікувати, що елемент це `div`, але в `html` це насправді `button`.

Для того, щоб гарантувати максимальну якість створеного продукту разом з JavaScript було використано TypeScript.

TypeScript — це строго типізована мова програмування, яка ґрунтується на JavaScript і надає кращі інструменти будь-якого масштабу.

Ця мова додає до JavaScript додатковий синтаксис для підтримки більш тісної інтеграції з редактором. Дозволяє ловити помилки на ранніх стадіях розробки [18].

Впровадження мови TypeScript замість JavaScript додало б додатковий етап транскompіляції застосунку з TypeScript в JavaScript, а тому оригінальний код був-би змінений з використанням все можливих поліфілів тощо. При орієнтуванні на максимальну безпеку застосунку та його стабільність такий підхід не є ідеальним, хоч і додає свої плюси

Тому було вирішено, що вихідний код застосунку, який справді буде запускатись, буде написаний на JavaScript, а TypeScript було використано для створення файлу декларацій, тобто оголошень функцій та основних змінних застосунку. Також було створено файл конфігурацій TypeScript який включає в себе параметри для перевірки файлів JavaScript, а також найбільш строгі правила наскільки це можливо досягти з використанням TypeScript.

Без використання файлу декларацій в застосунку було б достатньо складно знайти помилки, оскільки, наприклад, навіть щоб сказати, що функція `assert`, яка відповідає за `assertion`, справді перевіряє інформацію та їй можна вірити потрібно було б писати додаткові обгортки. Типи з файлу декларацій використовуються напряму в файлі JavaScript через спеціальний формат коментарів `JSDoc`, який в тому числі використовується для документування коду.

Код вебсторінки представляє собою чотири основних файли, які виконуються в браузері, не включаючи файл `wasn`, який генерується з `Golang`. Ці файли це `HTML`, `CSS`, один файл для всього вихідного коду JavaScript

застосунку, а також один додатковий файл `wasm_exes`, який відповідає за взаємодію з WebAssembly з JavaScript.

Весь вихідний код застосунку було написано в одному файлі, оскільки таким чином простіше його передавати через мережу, а також простіше з ним працювати. При спробі розділити його на декілька файлів ми отримаємо достатньо великий та переускладнений проєкт. Файл `wasm_exes` є файлом, який залежить від версії компілятора та поставляється разом з ним.

Код логічної частини, Golang, поділений на декілька модулів для максимальної простоти системи, а також для можливостей розширювати систему в майбутньому. Основні два модулі це модуль `lsb` та модуль `bpccs`, які реалізують основні методи системи. Ці модулі є під-модулями модуля `stego`, який є головним API для всього застосунку. На основі модуля `stego` можна в майбутньому створити інші представлення застосунку, наприклад, CLI.

Далі в застосунку реалізовані пакети – модулі, які можуть використовуватись незалежно, а саме `assert` – модуль для роботи з `assertion`, та модуль `imageio`, для роботи по перетворенню картинок.

Основним модулем системи є `wasm`, який взаємодіє з JavaScript, перетворює дані в типи Golang через спеціальні функції, обробляє результати та відправляє дані назад до JavaScript в спеціальному форматі. Цей формат має поле `ok`, якщо воно відповідає значенню `правда`, то відповідь містить перетворені дані, а якщо не `правда`, то повідомлення містить рядок `“message”` з текстом помилки.

Для зручної та швидкої роботи з Golang в браузері було використано `TinyGo` – компілятор для Go який створює WebAssembly файли в дуже малому розмірі, в порівнянні з стандартним компілятором Golang [19].

Також для зручної розробки вебсторінки було використано форматувальник коду `Prettier`. Він дозволяє забезпечити однаковість коду та притримування однакових стилістичних рішень для вебсторінки. В Golang було використано вбудований форматувальник.

Внутрішня будова коду вебсторінки має декілька основних логічних блоків.

Перший це конфігурація, в якій зберігаються всі унікальні ідентифікатори елементів та їх типи для подальших перевірок.

Другий це стан програми, в якому зберігаються завантажені файли, параметри ключів, параметри відповіді та інше.

Третій це змінні з елементами HTML. Кожен елемент має свій унікальний ідентифікатор, а при його отриманні додатково виконується перевірка, що елемент має правильний тип.

Далі йдуть слухачі подій, які виконують функції при завантаженні файлів, зміні даних в полях вводу або кліках на елементи. Один з таких слухачів це кнопка `run`, яка запускає логіку. При кліці на цю кнопку виконуються перевірки, які запускають потрібно функцію `Golang` та обробляють відповідь.

Наступною важливою функцією є `render`. Ця функція відповідає за синхронізацію стану програму та візуальної частини за допомогою приховування елементів, встановлення їм атрибутів тощо.

В кінці програми йдуть допоміжні функції, які потрібно для виконання деяких специфічних операцій. Наприклад, `assert` або `isRecord`.

Структура коду програми показана на рисунку 3.1.

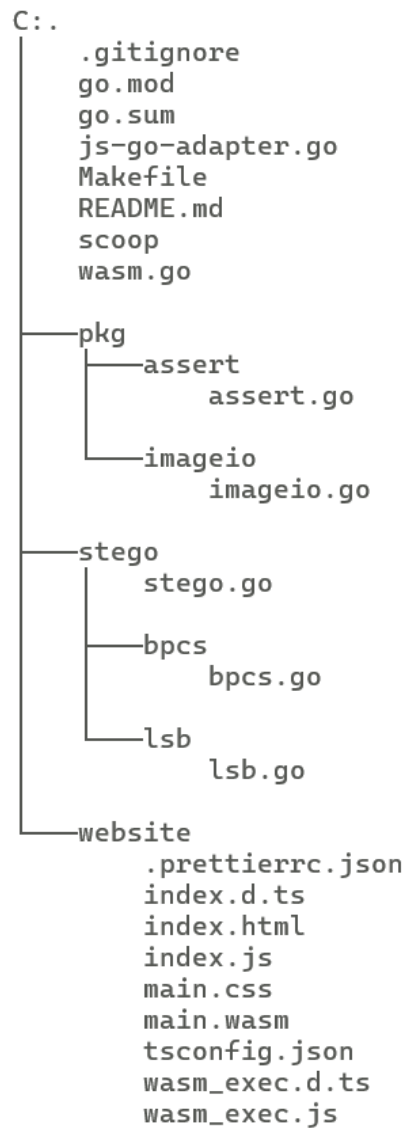


Рисунок 3.1 – Структура коду програми

Як можна побачити, структура файлів Golang відповідає модулям, а також повністю відповідає спроектованому варіанту системи.

Основні методи взаємодії між JavaScript та Golang показані на рисунку 3.2.

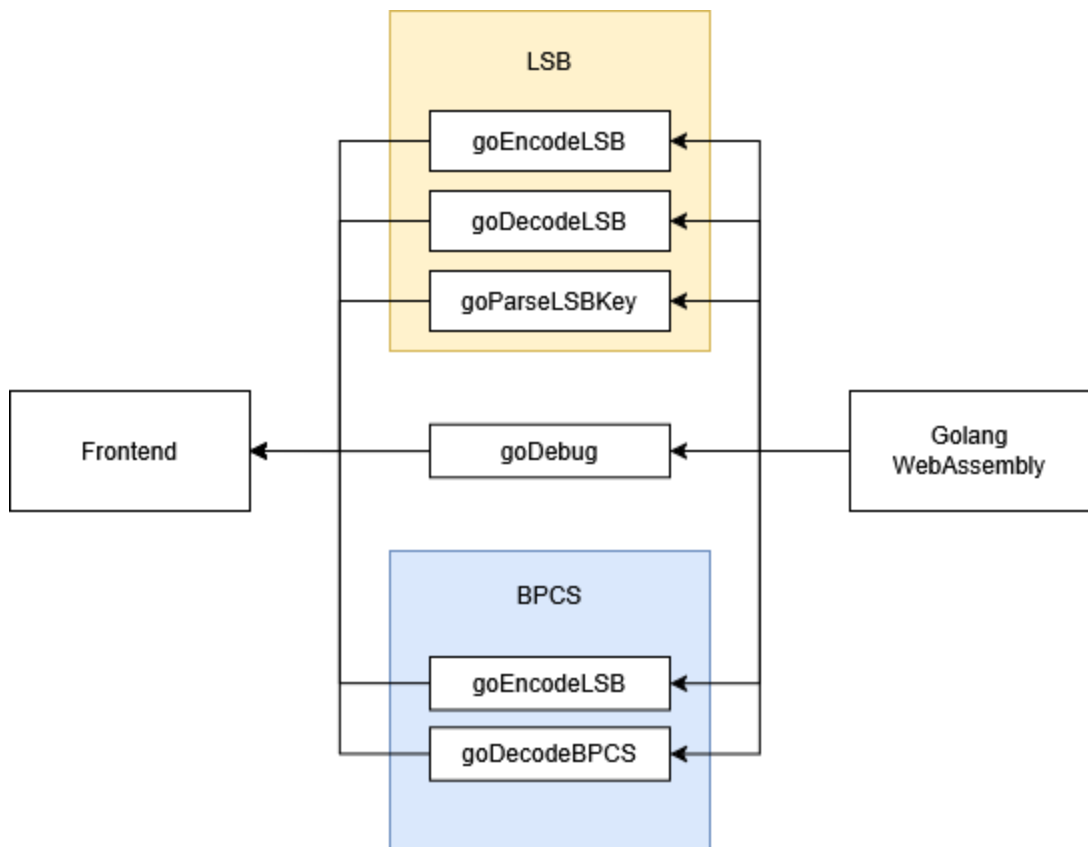


Рисунок 3.2 – Методи взаємодії JavaScript та Golang

Як можна побачити з рисунку 3.2, всього є 6 функцій через які відбувається взаємодія:

- функція “goEncodeLSB” відповідає за приховування інформації в зображенні. Функція приймає LSB ключ як рядок;
- функція “goDecodeLSB” відповідає за витягування прихованих даних з зображення. Функція приймає LSB ключ як рядок;
- функція “goParseLSBKey” виконує перетворення LSB ключа з рядка на окремі параметри. Ця функція потрібна в ситуації, коли користувач вирішить встановити вже існуючий ключ в поле, або змінити його там. Ця функція існує для забезпечення принципу Do not Repeat Yourself та мінімізації важливої логіки на частині JavaScript;
- функція “goEncodeBPCS” виконує приховування інформації в зображенні;

- функція “goDecodeBPCS” виконує витягування прихованої інформації з зображення;
- функція “goDebug” вмикає чи вимикає режим налагодження в Golang.

Застосунок, як і проєктувалось, залежить тільки від одного модулю, який є офіційний пакетом, але не входить в набір стандартних, а саме пакет для роботи з bmp файлами. На рисунку 3.3 показані всі залежності проєкту, з яких видно, що проєкт залежить від Golang, пакету для bmp “golang.org/x/image” та ще додаткового пакету для роботи toolchain. Всі ці пакети є стандартними, а тому застосунок буде мати максимальний рівень безпеки.

```
C:\home\projects\steganography>go mod graph
github.com/ltlaitoff/steganography go@1.24.0
github.com/ltlaitoff/steganography golang.org/x/image@v0.35.0
github.com/ltlaitoff/steganography toolchain@go1.24.11
go@1.24.0 toolchain@go1.24.0
golang.org/x/image@v0.35.0 golang.org/x/text@v0.33.0
golang.org/x/image@v0.35.0 go@1.24.0
```

Рисунок 3.3 – Залежності проєкту

4 КЕРІВНИЦТВО ПРОГРАМІСТА

4.1 Призначення й умови застосування програми

Розроблене програмне забезпечення призначене для приховування та вилучення конфіденційних даних у цифрових зображеннях форматів PNG, BMP та JPEG за допомогою стеганографічних методів LSB та BPCS.

Застосунок підтримує гнучкі налаштування для роботи з LSB через спеціальний секретний ключ, який потрібно передавати окремо від зображення, його використання не є обов'язковим. Наприклад, застосунок підтримує можливість приховувати інформацію тільки в один з каналів зображення, або ж задати, що кожен окремих піксель буде приховуватись в окремий канал.

Для експлуатації застосунку потрібен браузер з підтримкою стандарту WebAssembly (Chrome 57+, Firefox 52+, Safari 11+) [20].

Для першого завантаження застосунку, якщо він запускається не через локальну копію, потрібно мати доступ до мережі інтернет.

Апаратні вимоги включають наявність принаймні 4 ГБ оперативної пам'яті для стабільної роботи та процесор з частотою від 2 ГГц. Ці вимоги впливають в більшій степені від браузера та середовища розробки, яке буде використовуватись.

Програмне забезпечення не потребує специфічних периферійних пристроїв, окрім стандартних засобів введення та монітора з підтримкою глибини кольору 24 біти для коректного відображення результатів приховування даних.

4.2 Характеристики програми

Програмний продукт характеризується високою швидкістю обробки даних завдяки використанню технології WebAssembly, що дозволяє виконувати стеганографічні перетворення майже в реальному часі. Часові показники

роботи застосунку залежать від роздільної здатності вхідного зображення та обсягу секретного повідомлення, проте оптимізовані алгоритми LSB та BPCS забезпечують завершення операцій приховування або вилучення інформації протягом кількох секунд. Для надзвичайно великих зображень цей час може досягати до 5 секунд. Програма функціонує в діалоговому режимі, надаючи користувачу можливість попереднього перегляду оригінальних та модифікованих зображень. Також у користувача є можливість замінити оригінальне зображення на результат, а результат на оригінальне зображення, що є зручно для швидкої перевірки роботи програми без необхідності завантажувати файли.

Засоби контролю правильності виконання операцій інтегровані на рівні архітектурної взаємодії між WASM-модулем та JavaScript-середовищем.

У системі реалізовано диференційований підхід до обробки виключних ситуацій та помилок. Базові функціональні помилки, як-от недостатня місткість контейнера для певного обсягу даних, коректно перехоплюються логікою Golang та передаються на клієнтську частину для відображення відповідного повідомлення в інтерфейсі користувача.

Водночас критичні внутрішні помилки та стан паніки призводять до повної зупинки виконання модуля та показ помилки користувачу, що щось пішло не так. Це гарантує цілісність даних та запобігає некоректним маніпуляціям із пам'яттю. На жаль з використанням WebAssembly наразі не можливо показати повну інформацію про помилку на вебсторінці, але можна показати, що вона існує. Для додаткових деталей про помилки використовується панель розробника.

Самовідновлення програми здійснюється шляхом перезавантаження сторінки браузера, що дозволяє скинути стан середовища виконання до початкового. Оскільки застосунок виконується лише в браузері та не має прямого доступу до файлів системи, скидання стану немає негативного впливу на операційну систему чи файли користувача.

4.3 Звертання до програми

Для подальшої розробки застосунку програмісту потрібно встановити Golang версії 1.24.0 або вище, поточну версію можна знайти всередині файлу який оголошує модуль для всього застосунку. Так важливо встановити необхідні пакети. Наразі в застосунку використовується тільки один пакет, який не йде за замовчуванням разом з Golang, це “golang.org/x/image”. Цей пакет є офіційним пакетом який розширяє функціонал для роботи з картинками.

Для зручної розробки рекомендується використовувати налаштоване середовище, яке буде підтримувати роботу як в Golang, так і в JavaScript та TypeScript файлах одночасно. Гарними прикладами є Neovim з налаштованою конфігурацією та встановленими серверами lsp, або ж Goland чи Webstorm.

Частина проєкту, яка взаємодіє напряму написана з використанням JavaScript, але в проєкті існує файл конфігурації TypeScript, а також файл з деклараціями типів, які потім, за допомогою JSDoc, використовуються в коді JavaScript. Дуже важливо, щоб середовище розробки в тому числі мало підтримку роботи з цими типами, оскільки вони автоматично перевіряють код на помилки та вказують на проблеми. Наразі в проєкті не було встановлено лінтер, оскільки перевірок з використанням TypeScript було повністю достатньо, але в майбутньому можна встановити ESLint або його аналоги для ще більшої безпеки під час розробки.

Також для розробки застосунку потрібно встановити компілятор TinyGo. Він використовується через його швидкість та можливість створювати файли WebAssembly, які менші в розмірі, а ніж при використанні звичайного компілятора Golang.

Для локального запуску застосунку проєкта потрібно запустити локальний сервер. Для цього можна використати, наприклад, live server. Цей крок є необхідним, оскільки браузері мають проблеми з роботою з WebAssembly при простому відкритті html файлу.

Після зміни коду вебсторінки, тобто HTML, CSS або JavaScript сервер, в залежності від того який використовується, автоматично перезавантажить сторінку веббраузера та покаже зміни. При зміні коду в Golang для оновлення інформації на сторінці потрібно запустити процес компіляції. На жаль наразі це є єдиною опцією, оскільки браузер не працює напряму з Golang, а перетворювати код Golang в WASM при кожній зміні файлу буде достатньо проблематично, оскільки час компіляції займає приблизно в середньому 15 секунд.

Взаємодія з програмним засобом здійснюється через веббраузер шляхом відкриття необхідної сторінки.

Процедура передачі керування реалізована через графічний інтерфейс, де користувач взаємодіє з елементами керування для вибору необхідної операції для роботи, а саме приховування або вилучення секретної інформації, а також вибору методу який буде використовуватись.

Параметри даних передаються до виконуваного модуля шляхом завантаження файлу-контейнера через поле введення, секретна інформація передається через текстове або поле для завантаження файлу, всі інші налаштування встановлюються через відповідні елементи, а саме поля вводу для текстових даних та чисел, а також прапорці.

Програма підтримує роботу з вхідними цифровими зображеннями у форматах PNG, BMP та JPEG. У процесі обробки та вбудовування даних усі вхідні файли трансформуються у внутрішнє представлення, а результатом роботи є сформоване стего-зображення у форматах PNG або BMP, що забезпечує збереження цілісності прихованих бітів. Зображення-контейнер в форматі JPEG перетворюється за стего-зображення в форматі PNG, оскільки зберігати інформацію в JPEG напряму з використанням поточних методів, які були використані в цій курсовій роботі, неможливо або занадто складно, оскільки JPEG є форматом з втратами інформації.

Після завершення обробки керування повертається до клієнтської частини для візуалізації результату при приховуванні інформації, або ж

надання можливості завантаження модифікованого файлу на локальний диск користувача при витягуванні інформації, якщо вона була прихована з використанням файлу.

4.4 Вхідні і вихідні дані

Вхідною інформацією для роботи програмного засобу є цифрові зображення-контейнери, дані для приховування та секретний ключ при використанні LSB. Система підтримує формати зображень PNG, BMP та JPEG, які після завантаження в пам'ять розкодовуються у масив пікселів, де кожен піксель представлений комбінацією кольірних каналів RGBA.

Секретні дані, що підлягають вбудовуванню, сприймаються програмою як послідовність байтів незалежно від їхнього початкового формату.

Секретний ключ може бути введений користувачем у вигляді текстового рядка або завантажений як файл довільного розширення, що згодом перетворюється на байтовий масив для ініціалізації параметрів стеганографічного алгоритму.

Вихідними даними програми є модифіковане стего-зображення та вилучена секретна інформація у разі виконання операції декодування.

Організація вихідного зображення відповідає форматам PNG або BMP, оскільки ці формати використовують стиснення без втрат, що є критично важливим для збереження цілісності вбудованих бітів.

При використанні методу LSB вихідні дані кодуються шляхом заміни найменш значущих бітів у вибраних каналах (Red, Green, Blue) згідно з заданим кроком та координатами. Застосунок не змінює інформацію в альфа каналі, який відповідає за прозорість.

У результаті роботи алгоритму BPCS вихідна інформація структурується шляхом заміни складних блоків бітових площин зображення на блоки секретних даних, попередньо підданих процедурі кон'югації для підвищення їхньої візуальної складності.

Вилучена з контейнера інформація видається користувачеві в її первинному бінарному вигляді, що дозволяє відновити оригінальні файли або текст без жодних змін.

4.5 Повідомлення

Під час функціонування програмного засобу система генерує інформаційні та діагностичні повідомлення, які дозволяють контролювати стан виконання операцій та оперативно реагувати на виключні ситуації. Усі повідомлення поділяються на візуальні сповіщення в інтерфейсі користувача та технічні виводи в консолі розробника, що зумовлено специфікою роботи середовища WebAssembly.

Для оператора передбачено наступний перелік основних повідомлень у графічному інтерфейсі:

- показ результату виконання операцій вбудовування або вилучення даних;
- повідомлення про перевищення місткості контейнера, яке виникає, якщо обсяг секретних даних разом із метаданими перевищує доступну кількість бітів у вибраних каналах зображення, при цьому оператору необхідно обрати менший обсяг даних або змінити налаштування інтервалів та кількості каналів в випадку використання LSB;
- попередження про невідповідність формату вхідного файлу, що вимагає перевірки розширення зображення на відповідність стандартам PNG, BMP або JPEG;
- повідомлення про помилку зчитування секретних даних, що потребує перевірки цілісності завантаженого файлу-ключа або коректності введеного тексту.

Для програміста або спеціаліста з технічної підтримки передбачені системні повідомлення, що транслюються в консоль браузера:

- повідомлення про стан ініціалізації WASM-модуля, що підтверджує успішне завантаження виконуваного коду Golang та реєстрацію глобальних функцій у контексті JavaScript;
- вивід результатів роботи механізму відслідковування внутрішніх помилок системи assertion, який у разі логічних помилок у коді вказує на конкретну ділянку програми, де виник збій, що потребує перезапуску застосунку та аналізу вхідного набору даних;
- повідомлення про стан паніки у середовищі Golang, яке супроводжується трасуванням стека викликів та вказує на критичну помилку обробки пам'яті або структури файлу, після чого необхідно оновити сторінку для відновлення роботи системи.

Дії користувача при отриманні будь-якого з перелічених повідомлень про помилку повинні бути спрямовані на перевірку вхідних параметрів та, у разі критичного завершення роботи модуля, на повторну ініціалізацію робочого середовища.

5 КЕРІВНИЦТВО ОПЕРАТОРА

5.1 Призначення програми

Програмне забезпечення розроблене для виконання операцій стеганографічного приховування та вилучення цифрових даних у графічних файлах форматів PNG, BMP та JPEG.

Основним призначенням застосунку є забезпечення конфіденційності передачі інформації шляхом її вбудовування у візуальний контент, що дозволяє приховати сам факт існування секретного повідомлення.

Програма орієнтована на використання в умовах, де необхідно забезпечити захищений обмін даними без використання явних методів криптографічного шифрування, або як додатковий рівень захисту інформації.

Експлуатація програми передбачає виконання двох основних операцій, а саме приховування секретних даних у зображення-контейнер та витягування прихованої інформації з отриманого стего-об'єкта.

Процес приховування базується на алгоритмах LSB (заміна найменш значущих бітів) та BPCS (заміна складних бітових площин), що дозволяє оператору обирати оптимальний баланс між обсягом вбудованих даних та візуальною непомітністю внесених змін.

Програма повністю запускається в браузері користувача не виконуючи операцій на серверах, а тому є абсолютно безпечною для використання, а також може використовувати без доступу до мережі інтернет, якщо встановлена локально.

Застосунок надає інструменти для гнучкого налаштування параметрів безпеки, що робить експлуатацію системи інтуїтивно зрозумілою навіть для користувачів без глибоких знань у галузі стеганографії.

5.2 Умови виконання програми

Мінімальний склад апаратних засобів включає персональний комп'ютер або портативний пристрій, оснащений центральним процесором з тактовою частотою не менше 2 ГГц та обсягом оперативної пам'яті від 4 ГБ.

Такий об'єм пам'яті є необхідним для стабільного розміщення в пам'яті браузера масивів пікселів при роботі з зображеннями високої роздільної здатності та запобігання аварійному завершенню модуля WASM.

Відеосистема повинна підтримувати роздільну здатність екрана не менше 1280x720 пікселів для адекватного відображення елементів графічного інтерфейсу та палітри кольорів True Color (24 біти) для візуального контролю результатів стеганографічного вбудовування.

Для експлуатації застосунку потрібен браузер з підтримкою стандарту WebAssembly (Chrome 57+, Firefox 52+, Safari 11+) [20].

Операційна система може бути довільною (Windows, Linux, macOS), оскільки середовище виконання ізольоване всередині браузера. Додатковою умовою для коректної експлуатації є дозвіл на виконання JavaScript у налаштуваннях безпеки браузера та наявність вільного місця на накопичувачі для збереження вихідних стего-об'єктів у форматах PNG або BMP.

Програмний продукт не потребує інсталяції додаткових бібліотек або сторонніх компонентів, оскільки всі необхідні залежності скомпільовані у виконуваний файл формату .wasm.

5.3 Виконання програми

Для запуску застосунку потрібно відкрити його віддалену або локально встановлену копію.

При першому запуску застосунку користувач бачить основний вид застосунку, який показаний на рисунку 5.1.

The screenshot displays the main interface of the application. At the top, there are four buttons: 'LSB' (pink), 'BPCS' (light blue), 'Encode' (purple), and 'Decode' (light blue). Below these is a horizontal line. Under the line, the 'Options' section includes a checkbox for 'Secret message as file instead of text'. The 'Encode' section has input fields for 'Secret message' and 'Secret file'. The 'LSB Key' section contains a text input for 'Key' (with the value 'S0T0E0N0H0V0P3CRGCB') and several dropdown menus for 'Start position of X', 'Start position of Y', 'End position of X (0 is image end)', 'End position of Y (0 is image end)', 'Gap between X', 'Gap between Y', 'How much channels used per one pixel?' (set to 3), and 'Used channels? Example: "RGB", "RB", "G"' (set to 'RGB'). To the right, there are two large boxes labeled 'Original image' and 'Result image', both containing the text 'Drop images here, or click to upload.' and a double-headed arrow between them. Below the 'Original image' box is a 'Run options' section with a 'Debug' checkbox and a green 'Submit' button.

Рисунок 5.1 – Основний вид застосунку

Вебсторінка складається з двох основних частин. Зліва знаходиться меню, в якому можна обрати метод для приховування або витягування інформації та операцію, яка буде використовуватись в поточний момент часу. Вигляд меню показаний на рисунку 5.2.

The screenshot shows a menu with four buttons arranged in a 2x2 grid: 'LSB' (pink), 'BPCS' (light blue), 'Encode' (purple), and 'Decode' (light blue). A horizontal line is positioned below the buttons.

Рисунок 5.2 – Вигляд меню

Під меню знаходяться всі потрібні опції для обраного методу та обраної операції. При виборі операції приховування інформації, encode, завжди буде показано меню для вводу секретного повідомлення. Воно показано на рисунку 5.3.

Options

Secret message as file instead of text

☐

Encode

Secret message

Рисунок 5.3 – Ввід секретного повідомлення в текстовому форматі

Також в цьому блоці є прапорець, якщо ввімкнути який замість вводу секретного повідомлення в текстовому форматі тепер повідомлення можна завантажити в вигляді файлу. Важливо, що файл може бути в будь-якому форматі, а також при приховуванні даних його розширення не зберігається. На рисунку 5.4 показано поле вводу для секретного файлу, а на рисунку 5.5 показано поле вводу з обраним секретним файлом.

Options

Secret message as file instead of text

☒

Encode

Secret file

Browse...

No file selected.

Рисунок 5.4 – Поле вводу для секретного файлу

Options

Secret message as file instead of text

☒

Encode

Secret file

Browse...

secret.txt

Рисунок 5.5 – Поле вводу для секретного файлу з обраним файлом

В операції витягування даних з стего-зображення існують відповідні функції для показу інформації після її витягування. Для текстової інформації існує поле для виводу, а для файлової існує прапорець при активації якого з'являється кнопка для завантаження секретного файлу. Якщо було приховано текст, а запис відбувається в файл, то файл буде містити цей текст, а якщо було приховано файл, а витягування відбувається в текстовий формат, то в полі виводу буде показана внутрішня будова файлу. Поле для результатів витягування в текстовому форматі показано на рисунку 5.6, а поле для результатів в форматі файлу показано на рисунку 5.7.

Options

Secret message as file instead of text

☐

Decode

Decoded message

Рисунок 5.6 – Вивід текстової інформації після приховування

Options

Secret message as file instead of text

☒

Decode

Download secret file

Рисунок 5.7 – Вивід інформації в вигляді файлу після приховування

Для методів в застосунку передбачена можливість мати налаштування.

Метод LSB має свої опції для приховування та витягування інформації, а саме параметри ключа. Ключ представляє собою рядок в спеціальному форматі, спочатку йде літера, яка позначає поле, а після йде інформація про це поле. Наприклад, “S0” в ключі означає, що стартова позиція по x буде 0.

В ключі можна вказати точку початку, з якої буде відбуватись приховування або витягування, точку кінця, відстань між пікселями по X та Y, кількість каналів, які будуть використовуватись в одному пікселі, а також список каналів, які будуть використовуватись загалом. Важливо, що кількість каналів на один піксель не повинні перевищувати кількість каналів загалом, оскільки інакше застосунок буде працювати не правильно.

Для зручної взаємодії з ключем було створено спеціальну форму, яка показана на рисунку 5.8.

LSB Key

Key	S0T0E0N0H0V0P3CRCGCB
Start position of X	0
Start position of Y	0
End position of X (0 is image end):	0
End position of Y (0 is image end):	0
Gap beetween X:	0
Gap beetween Y:	0
How much channels used per one pixel?	3
Used channels? Example: "RBG", "RB", "G"	RGB

Рисунок 5.8 – Форма для встановлення параметрів ключа

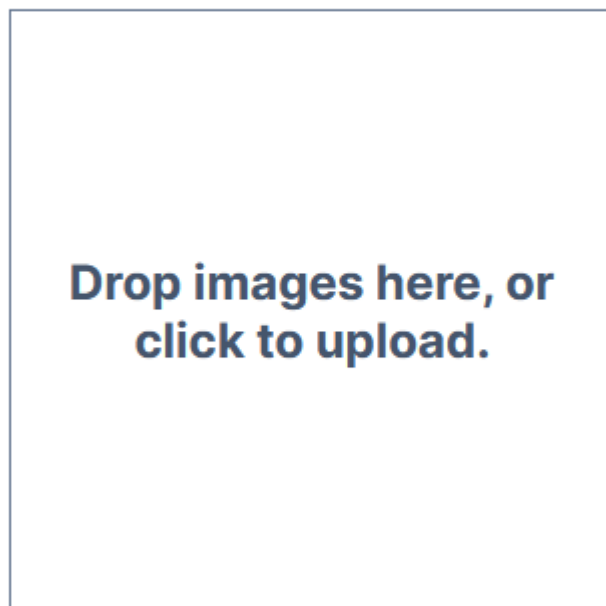
В той самий час ВРРС не має унікальних опцій.

Справа на вебсторінці знаходяться опції які відносяться до безпосереднього запуску програми.

В цьому блоці є блок завантаження оригінального зображення. Після завантаження, яке можна виконати через клік або перетягування, блок вводу

зображення заміниться на блок, в якому можна переглянути як воно виглядає.
Це показано на рисунках 5.9 та 5.10.

Original image



Run options

Рисунок 5.9 – Поле для завантаження оригінального зображення

Original image



Run options

Рисунок 5.10 – Поле попереднього перегляду оригінального зображення

Під оригінальним зображенням знаходиться прапорець який вмикає режим налагодження програми, він працює тільки LSB. Під час приховування інформації, якщо ввімкнений режим налагодження, пікселі які будуть змінені замінюються на контрастні, щоб можна було візуально зрозуміти які пікселі будуть змінені з поточними налаштуваннями. Цей режим є дуже корисним при використанні саме з параметрами ключа.

Під прапорцем з режимом налагодження знаходиться кнопка для запуску алгоритму. Це показано на рисунку 5.11.

Run options

Debug

☐

Submit

Рисунок 5.11 – Блок опцій запуску застосунку

Після запуску результат виконання буде записаний в блок, який попередньо показує результати виконання. Це показано на рисунку 5.12

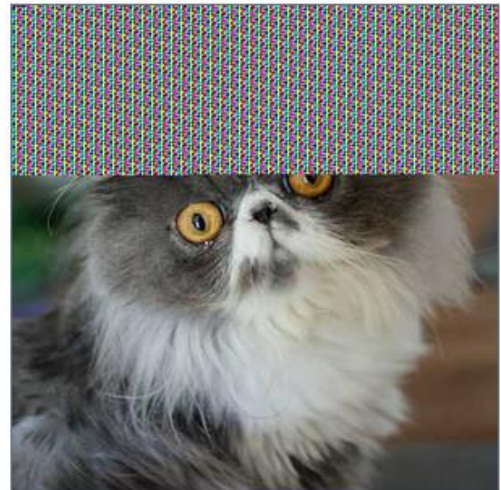
Рисунок 5.12 – Вигляд застосунку з отриманим результатом

Якщо ввімкнути режим налагоджування та написати достатньо велике повідомлення, то в результаті велика частина зображення буде змінена. Це показано на рисунку 5.13.

Original image



Result image



Run options

Debug



Submit

Рисунок 5.13 – Приклад використання режиму налагодження

Режим налагодження краще працює при його використанні зі зміненими параметрами для ключа LSB. Наприклад, ми можемо змінити відстань між пікселями по Y та по X, це показано на рисунку 5.14.

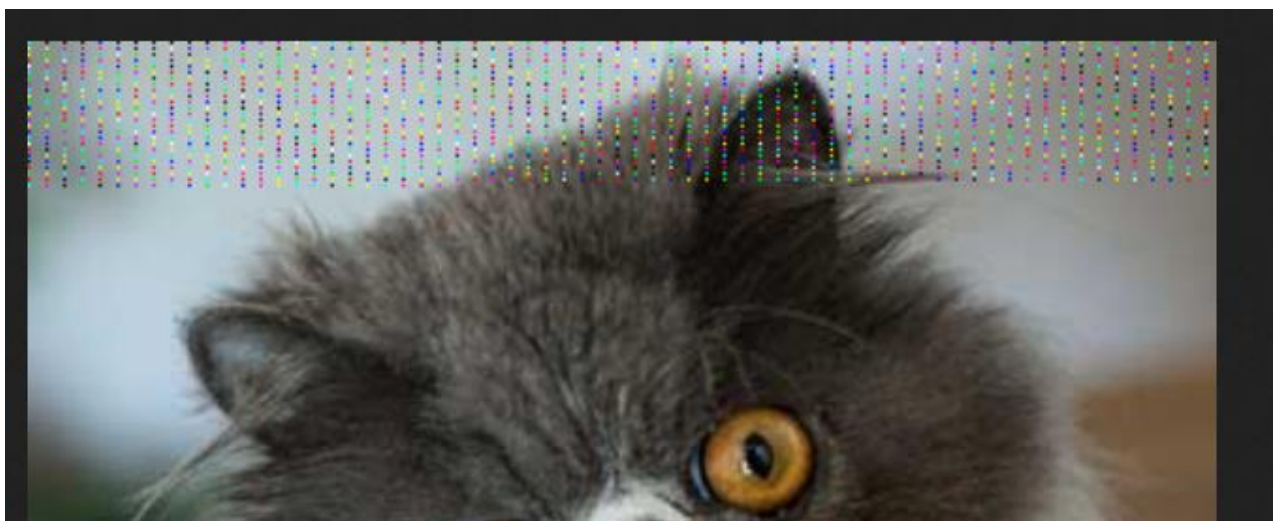


Рисунок 5.14 – Режим налагодження з пробілами в 5 пікселів по X та 1 по Y

Або ж ми можемо змінити кількість каналів, яка використовується для кодування одного пікселя. Це показано на рисунках 5.15 та 5.16.

Рисунок 5.15 – Приклад використання режиму налагодження на два канали

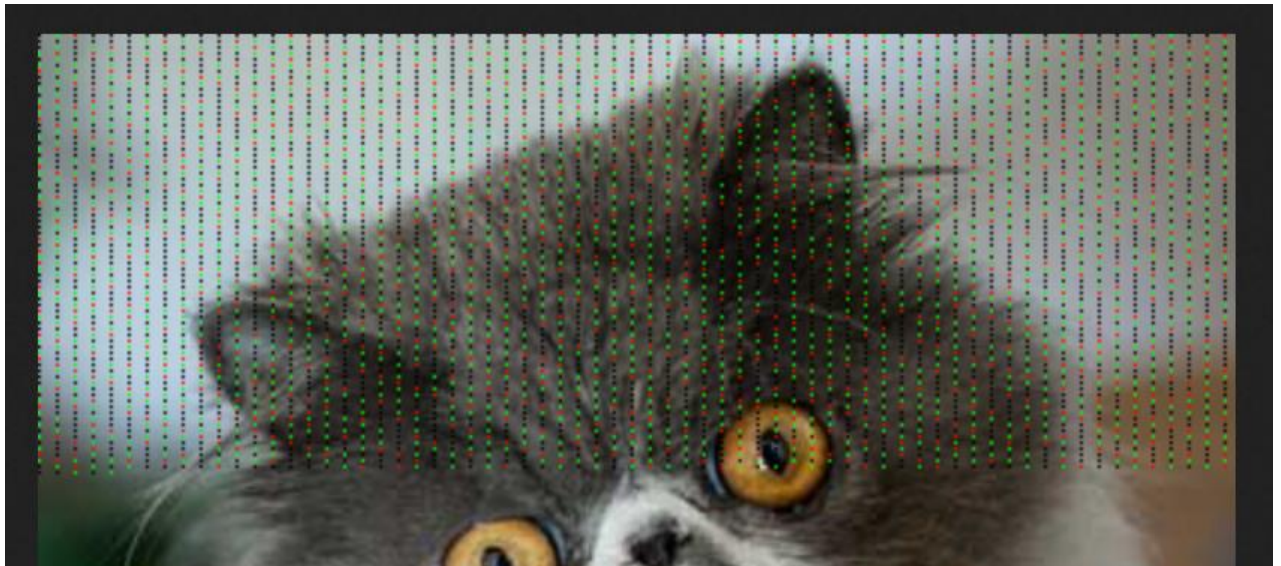


Рисунок 5.16 – Приклад зображення при приховуванні даних в два канали та один канал на піксель

Також в застосунку присутня кнопка для заміни оригінального зображення на результат та результат на оригінальне зображення. Основною ціллю цієї кнопки є швидка перевірка чи працює дешифрування інформації в

застосунку. Зображення до зміни показані на рисунку 5.17, після зміни на рисунку 5.18, а логіка дешифрування з результатом показана на рисунку 5.19.

LSB
BPCS

Encode
Decode

Options

Secret message as file instead of text ☐

Encode

Secret message

LSB Key

Key

Start position of X

Start position of Y

End position of X (0 is image end):

End position of Y (0 is image end):

Gap between X:

Gap between Y:

How much channels used per one pixel?

Used channels? Example: "RBG", "RB", "G"

Original image



Result image



↔

Run options

Debug ☒

Submit

Рисунок 5.17 – Зображення до зміни порядку

LSB
BPCS

Encode
Decode

Options

Secret message as file instead of text ☐

Decode

Decoded message

LSB Key

Key

Start position of X

Start position of Y

End position of X (0 is image end):

End position of Y (0 is image end):

Gap between X:

Gap between Y:

How much channels used per one pixel?

Used channels? Example: "RBG", "RB", "G"

Original image



Result image



↔

Run options

Debug ☒

Submit

Рисунок 5.18 – Зображення після зміни порядку

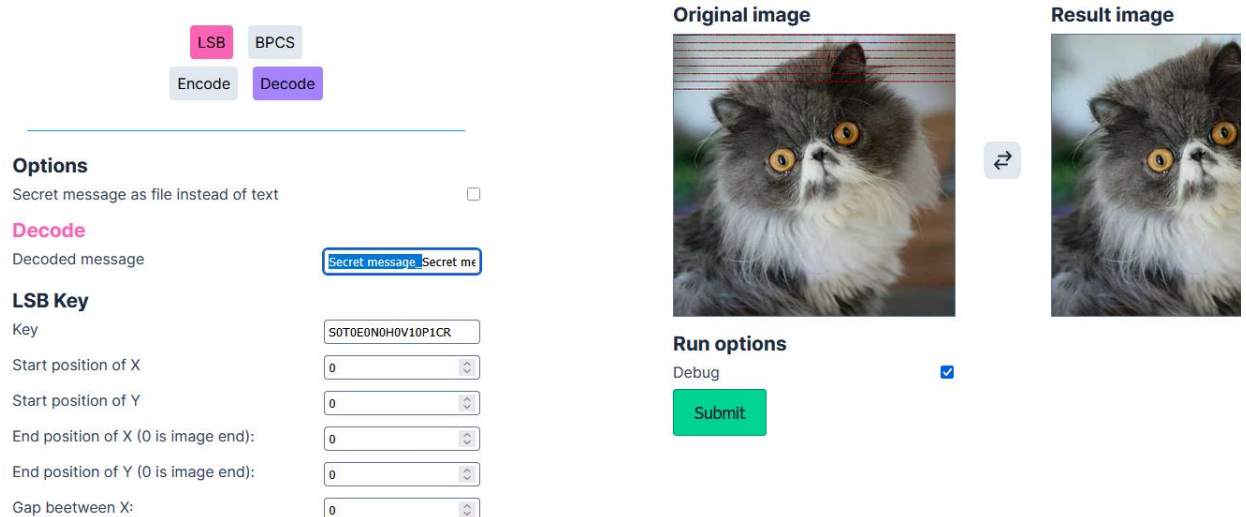


Рисунок 5.19 – Приклад роботи витягування інформації

Режим налагодження працює тільки під час приховування інформації в LSB, але не під час її витягування. Але витягування все ж працює правильно, оскільки режим налагодження візуально змінює пікселі враховуючи логіку алгоритму LSB.

5.4 Повідомлення оператору

Основне повідомлення, яке отримує оператор це повідомлення про помилки. Помилки існують двох основних видів: ті, які програма очікує отримати та є помилками користувача та ті, які програма не очікувала отримати та є помилками системи. Помилки користувача показані візуально вверху вебсторінки, а помилки системи, якщо це помилка на стороні JavaScript, також показуються візуально, але внутрішні помилки Golang не будуть показуватись візуально через обмеження стандарту WebAssembly.

При виникненні внутрішньої помилки застосунку найкраще це перезавантажити сторінку веббраузера, щоб скинути стан програми.

Найчастіше користувачу показується повідомлення про занадто великий розмір секретних даних в порівнянні з зображенням. В тому числі це можливо через зміненні налаштування ключа. Його приклад показаний на рисунку 5.20

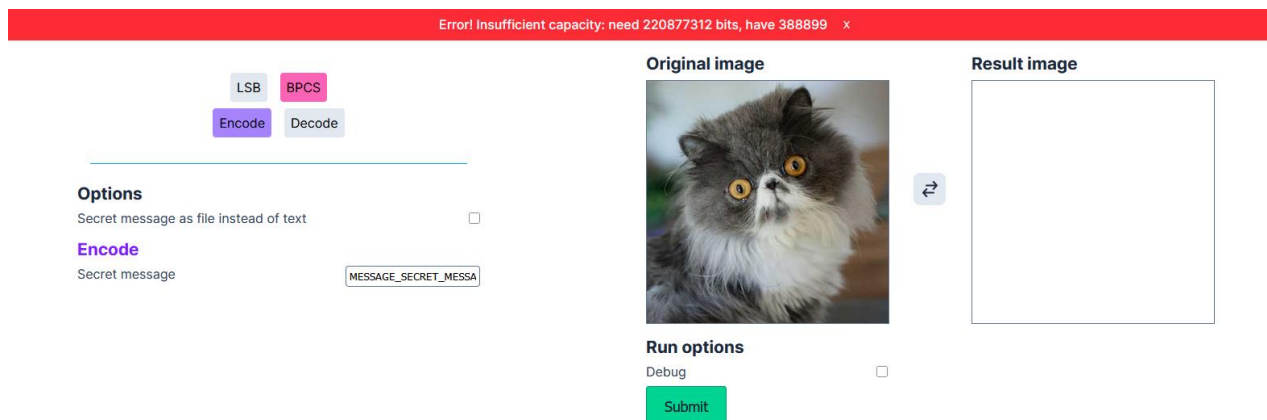


Рисунок 5.20 – Повідомлення про занадто великий розмір секретних даних

Також основним повідомленням є помилка про не правильний формат оригінального зображення. Вебсторінка зроблена таким чином, щоб їх не можна було завантажити, але все ще застосунок буде намагатись працювати з, наприклад, не правильним файлом зображення. Ця помилка показана на рисунку 5.21.

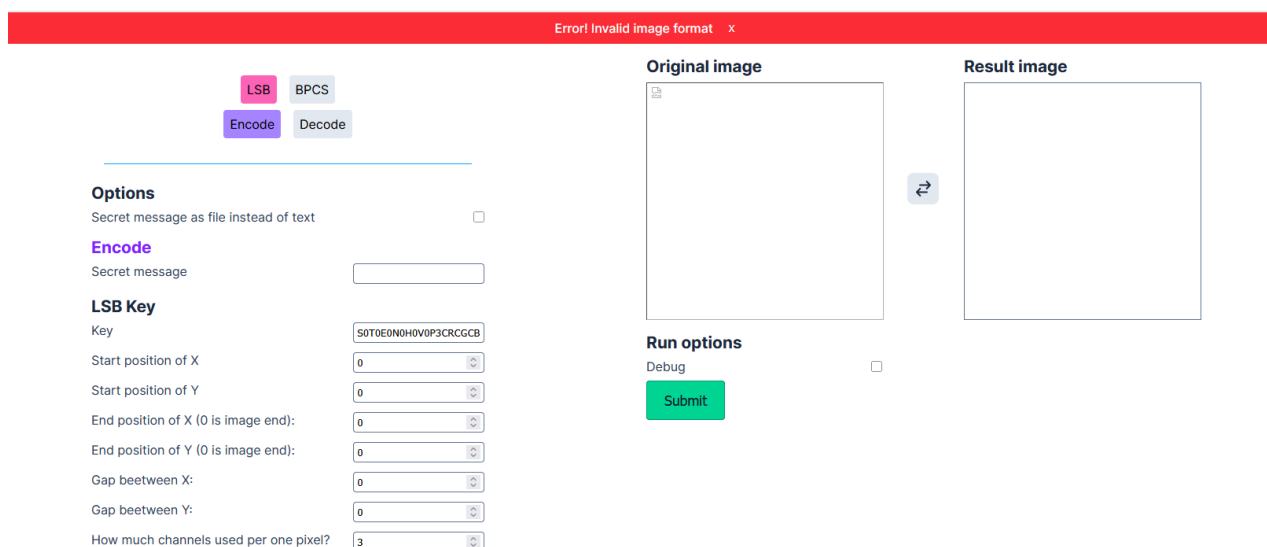


Рисунок 5.21 – Повідомлення про не правильний формат файлу

Під час виникнення подібних проблем користувачу достатньо закрити помилку та змінити дані. Іноді потрібно буде скидати стан застосунку через його перезавантаження. Важливо, що в деяких браузерах потрібно

використовувати не звичайне перезавантаження, а перезавантаження зі скиданням значень полів вводу.

Іншим основним повідомленням, яке отримує користувач є результат виконання програми. Картинка, яка показується в блоці з результатом може бути відкрита в новій вкладці, або збережена через контекстне меню браузера. Застосунок не має кнопок для завантаження цього зображення, або автоматично завантаження, оскільки цей функціонал гарно працює в браузері та це могло б заважати користувачам

6 ТЕСТУВАННЯ І РЕЗУЛЬТАТИ

6.1 Опис методики проведення тестування

Метою тестування є перевірка функціональної придатності розробленого програмного забезпечення для приховування та вилучення даних, а також оцінка стійкості стеганографічних алгоритмів до візуального аналізу.

Постановка задачі передбачає підтвердження цілісності інформації після циклу вбудовування-вилучення та перевірку коректності роботи механізмів захисту від переповнення контейнера.

Методика досліджень базується на проведенні серії експериментів із використанням тестових зображень форматів PNG, BMP та JPEG різної роздільної здатності.

Експерименти проводяться на персональному комп'ютері з процесором Intel Core i5 (2.4 ГГц) та 8 ГБ оперативної пам'яті під управлінням операційної системи Windows 11.

Для тестування використовується браузер Firefox останньої версії з активованою консоллю розробника для моніторингу станів WebAssembly-модуля. Процес тестування включає перевірку роботи методу LSB із різними параметрами налаштувань, такими як координати початку вбудовування та крок між пікселями, а також методу BPCS.

На кожному етапі оцінюється відсутність візуальних артефактів на вихідному зображенні та повна ідентичність вилучених даних початковому секретному повідомленню.

6.2 Результати використання розробленого програмного забезпечення

Першим тестом було проведено перевірку приховування текстових даних всередину зображення з використанням алгоритмів LSB та BPCS. Для

цього було взято початкове зображення, яке показано на рисунку 6.1. Воно було взято з Wiggles n wags.



Рисунок 6.1 – Початкове зображення

В якості секретного повідомлення було взято текст з вебсайту мови програмування Golang: «Go is expressive, concise, clean, and efficient. Its concurrency mechanisms make it easy to write programs that get the most out of multicore and networked machines, while its novel type system enables flexible and modular program construction. Go compiles quickly to machine code yet has the convenience of garbage collection and the power of run-time reflection. It's a fast, statically typed, compiled language that feels like a dynamically typed, interpreted language.». Для того, щоб краще було видно різницю текст буде повторюватись 5 разів.

Спочатку секретне повідомлення було приховано в зображенні використовуючи метод LSB та стандартні параметри ключа. Результат приховування показаний на рисунку 6.2. На рисунку 6.3 показаний результат з

ввімкненим режимом налагодження, а на рисунку 6.4 показана різниця між початковим та новим зображенням через використання сервісу metadata2go.



Рисунок 6.2 – Зображення з прихованими даними через LSB

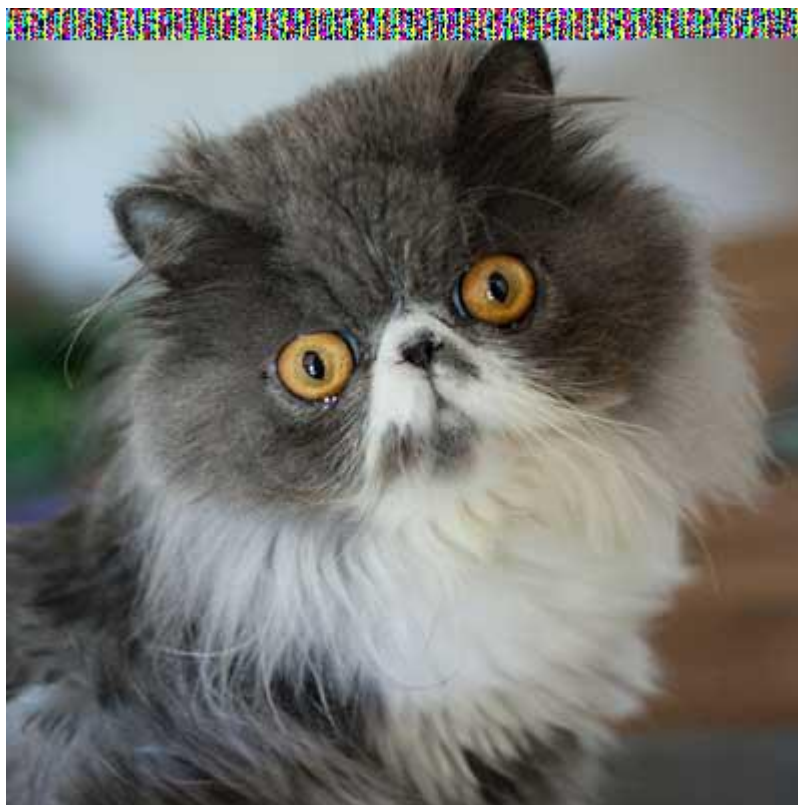


Рисунок 6.3 – Зображення з прихованими даними через LSB з ввімкненим режимом налагодження

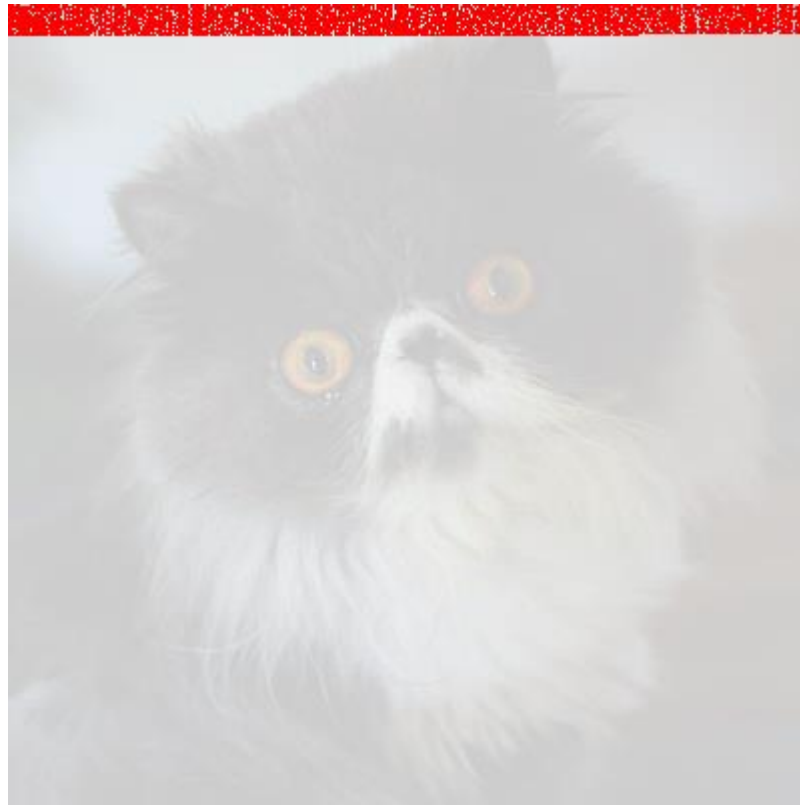


Рисунок 6.4 – Порівняння оригінального зображення та зображення, з прихованими даними

Наступним кроком є витягування прихованих даних з зображення. Для цього стего-зображення було завантажено в програму, а значення ключа LSB не змінювались. Результат показаний на рисунку 6.5.

LSB

BPCS

Encode

Decode

Original image



Options

Secret message as file instead of text ☐

Decode

Decoded message Go is expressive, concise, i

LSB Key

Key S0T0E0N0H0V0P3CRCGCB

Start position of X 0

Start position of Y 0

End position of X (0 is image end): 0

End position of Y (0 is image end): 0

Run options

Debug ☐

Submit

Рисунок 6.5 – Результат витягування даних

Таким чином ми перевірили, що LSB приховування працює правильно при стандартних налаштуваннях ключа. Тепер змінимо налаштування, використаємо лише червоний та синій канали, з 1 каналом на піксель, стартовою позицією на 10 по у та 50 по X, а також відстанню між пікселями по X в 1 та по Y в 3. Форма з ключем показана на рисунку 6.6, стего-зображення показано на рисунку 6.7, стего-зображення в режимі налаштування показано на рисунку 6.8, а також різниця між стего-зображенням та оригіналом показана на рисунку 6.9.

Options

Secret message as file instead of text

☐

Encode

Secret message

ped, interpreted language.

LSB Key

Key

S50T10E0N0H1V3P1CRCB

Start position of X

50

Start position of Y

10

End position of X (0 is image end):

0

End position of Y (0 is image end):

0

Gap between X:

1

Gap between Y:

3

How much channels used per one pixel?

1

Used channels? Example: "RBG", "RB", "G"

RB

Рисунок 6.6 – Форма опцій ключа LSB



Рисунок 6.7 – Стего-зображення зі зміненими параметрами ключа

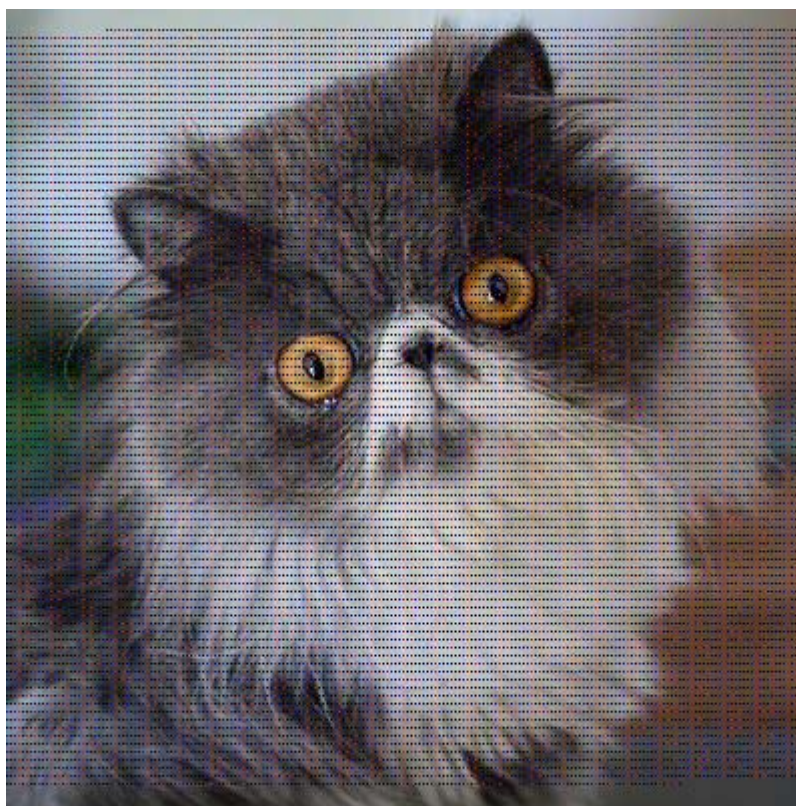


Рисунок 6.8 – Стего-зображення зі зміненими параметрами ключа LSB та ввімкненим режимом налагодження



Рисунок 6.9 – Різниця між оригінальним та стего зображеннями

Далі було виконано витягування інформації та секретне повідомлення було однаковим з тим, яке передавалось при приховуванні. Таким чином було перевірено функціонал роботи LSB з текстовими даними. Всі приховування займали менше ніж пів секунди часу.

Далі була виконана перевірка приховування файлової інформації, для цього було взято файл з вихідним кодом застосунку. Для цього було використано стандартні налаштування ключа, але з відстанню між пікселями по X в 1. Стего-зображення з прихованим файлом в форматі LSB показано на рисунку 6.10, його різниця з оригіналом показана на рисунку 6.11.



Рисунок 6.10 – Стего-зображення з прихованим файлом через LSB



Рисунок 6.11 – Порівняння оригінального зображення та стего-зображення

Після приховування було виконано також витягування даних. Файл, який було отримано в результаті точно відповідав файлу, який був прихований.

Таким чином було перевірено весь функціонал приховування інформації через LSB.

Далі було виконано приховування того самого файлу, але вже з використанням ВРСС. Стего-зображення показано на рисунку 6.12, а порівняння з оригіналом на рисунку 6.13.



Рисунок 6.12 – Стего-зображення з файлом через ВРСС

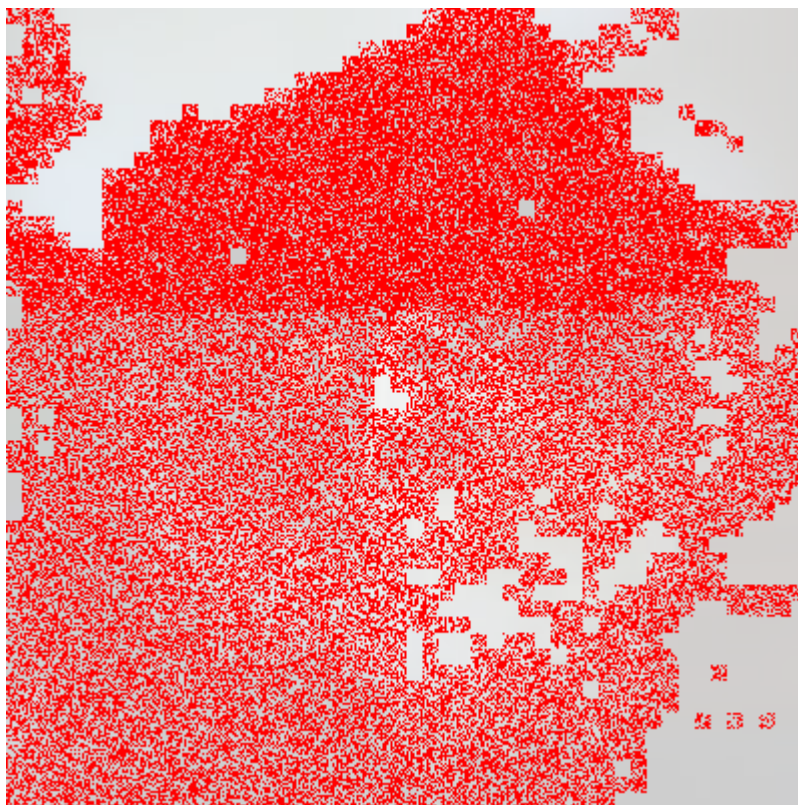


Рисунок 6.13 – Порівняння зображення з BPCS з оригіналом

Як можна побачити BPCS працює по іншому в порівнянні з LSB та записує інформацію тільки в блоки, які мають шум. Далі виконано витягування інформації та файл, який було отримано точно відповідає файлу, який було приховано.

Наступним етапом є перевірка застосунку на помилки. Було перевірено, що застосунок показує помилку, якщо зображення-контейнер має не правильний або формат який не підтримується. Це показано на рисунку 6.14.

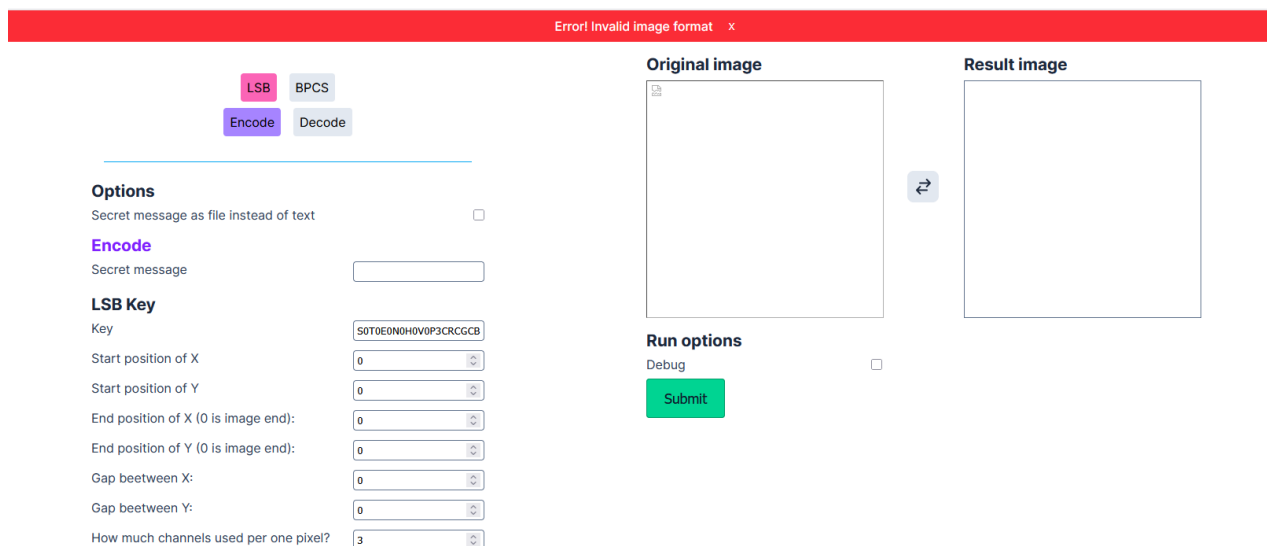


Рисунок 6.14 – Повідомлення про помилку при не правильному форматі зображення-контейнера

Далі було перевірено, що методи будуть показувати помилки в випадках, якщо секретне повідомлення є довшим, а ніж може в себе вмістити зображення. Це показано на рисунку 6.15.

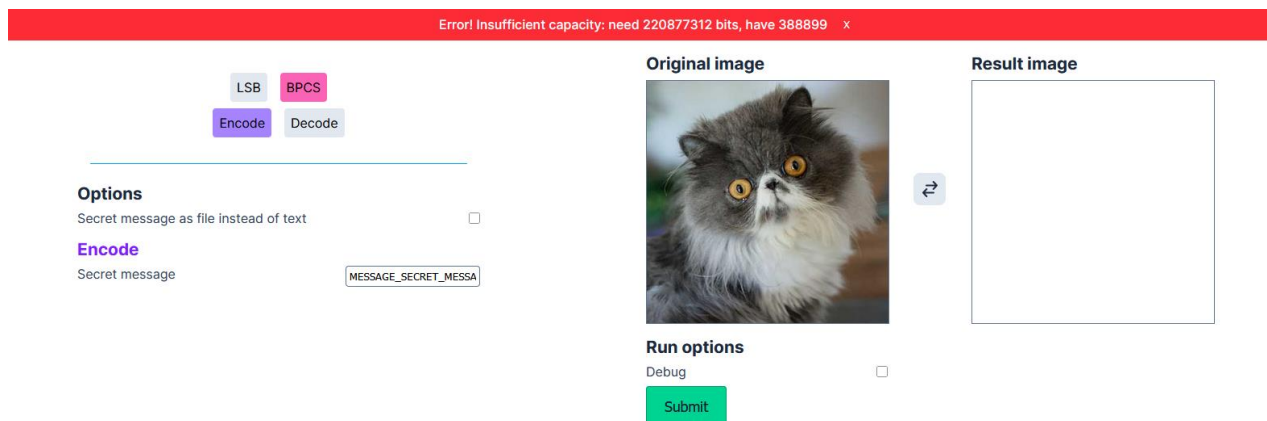


Рисунок 6.15 – Повідомлення про помилку, що зображення не може вмістити поточне повідомлення

Наступним кроком було перевірено чи працює вебсайт на різних розширеннях екрану. Це показано на рисунках 6.16, 6.17 та 6.18.

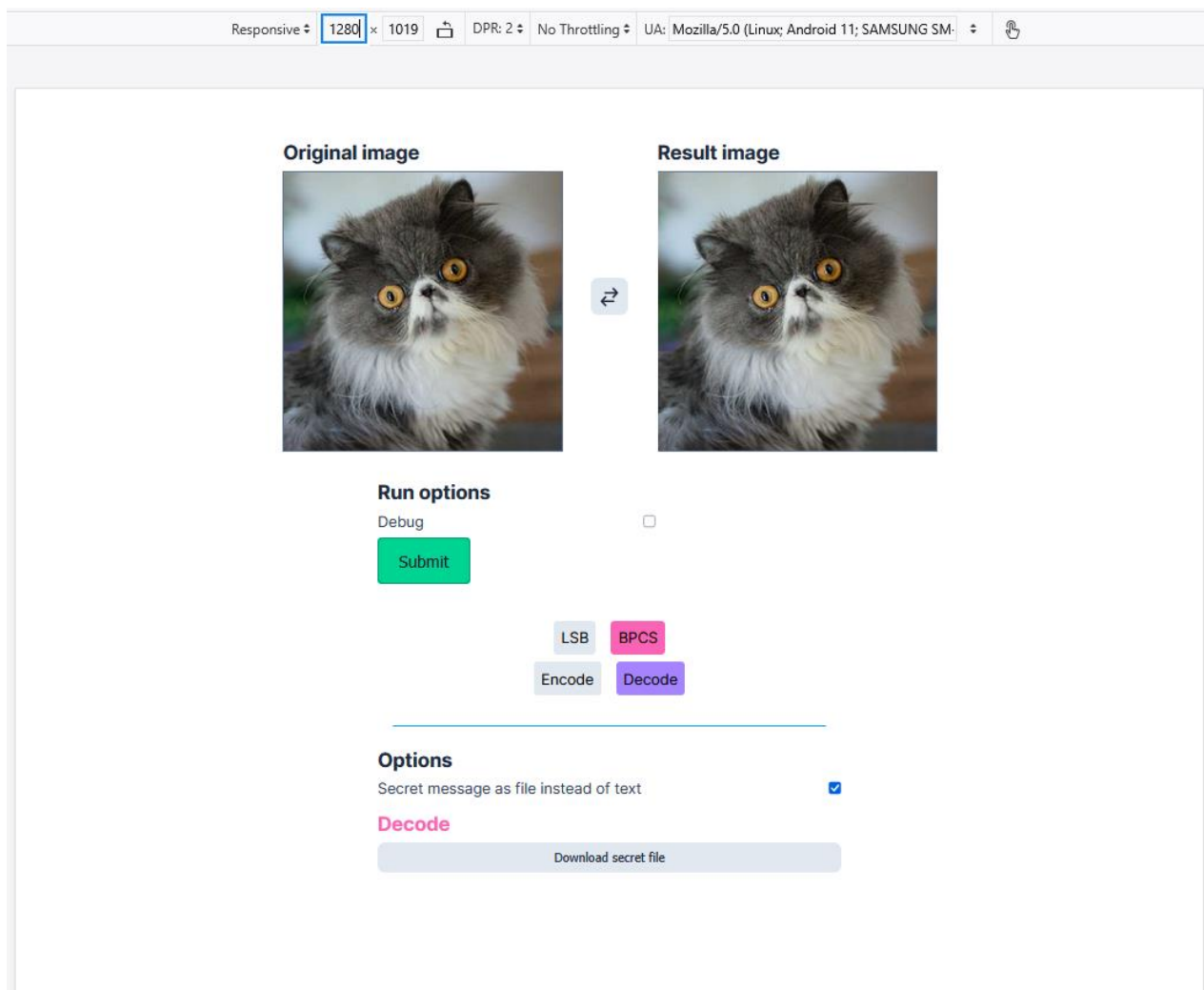


Рисунок 6.16 – Вигляд застосунку на розширенні 1280 пікселів

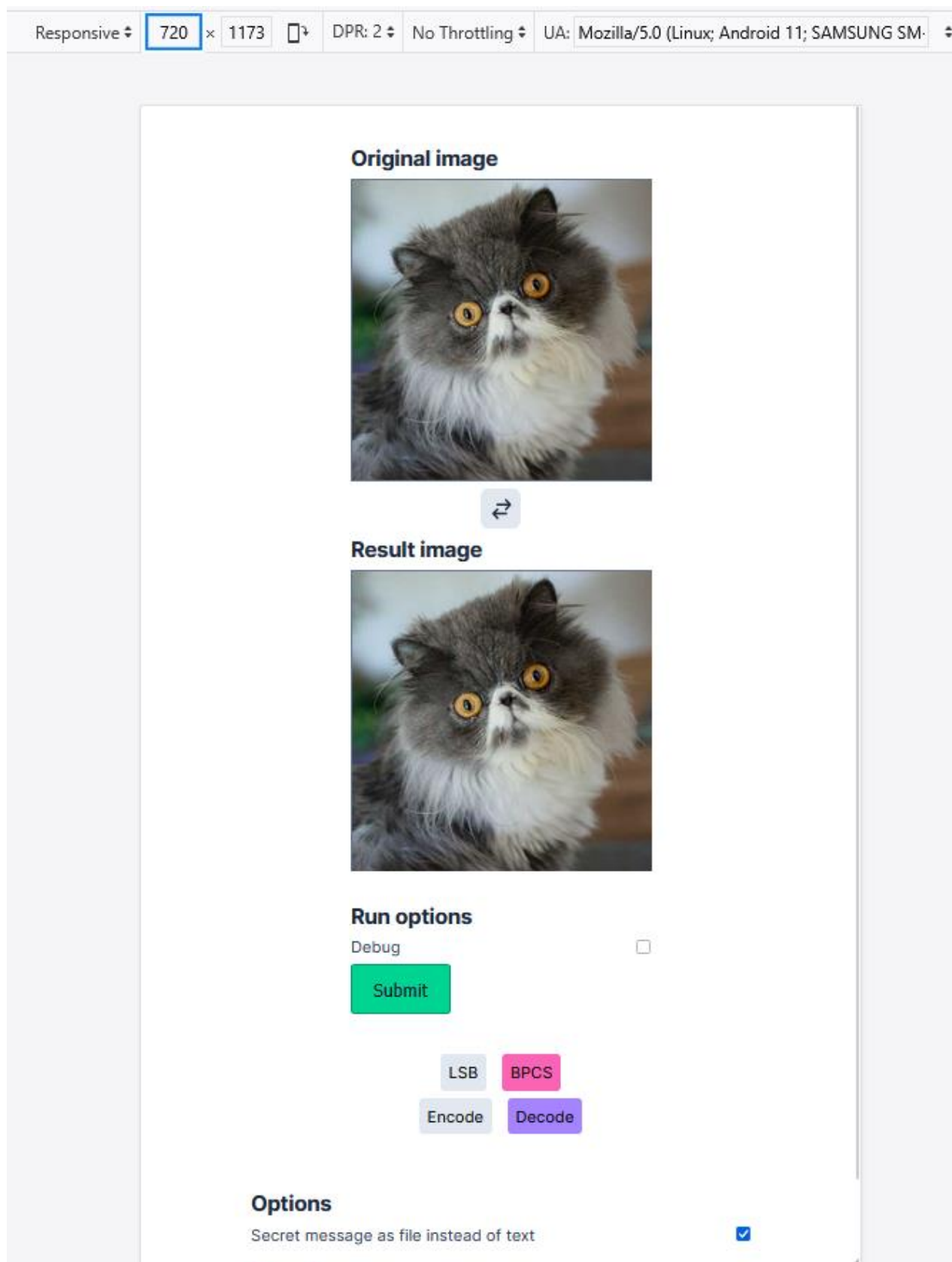


Рисунок 6.17 – Вигляд застосунку на розширенні 720 пікселів

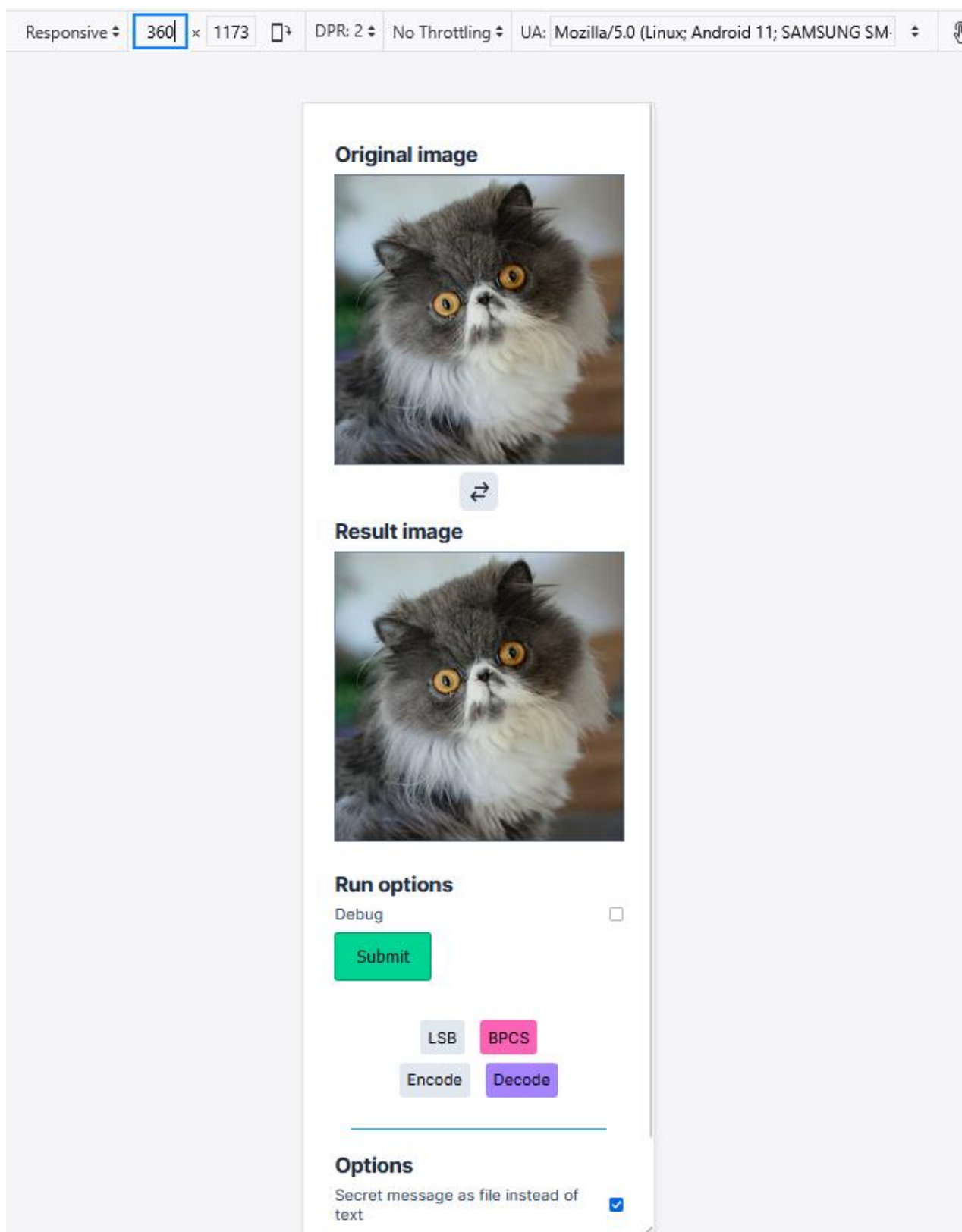


Рисунок 6.18 – Вигляд застосунку на розширенні 360 пікселів

Внутрішні помилки системи під час тестування не були знайдені, система працює стабільно та швидко.

Також в системі повинні бути передбачений функціонал при внутрішніх помилках системи, але тестування цього функціоналу є неможливим, оскільки наразі застосунок не має внутрішніх помилок.

В результаті тестування застосунок виконує всі функції, які було поставлені в технічному завданні. Застосунок правильно приховує інформацію без видимих артефактів, а також правильно її витягує з стего-зображень.

З можливих покращень застосунку можна віднести додати кнопку для завантаження стего-зображення, імплементувати режим налагодження для ВРCS режиму та додати поле для завантаження секретного файлу через його перетасування, а не вибір через поле вводу. Також одним з покращень є пришвидшення роботи алгоритму LSB та можлива майбутня реалізація CLI версії застосунку.

ВИСНОВКИ

У ході виконання курсової роботи було створено застосунок, який забезпечує можливість приховування даних в зображеннях різних видів для майбутньої їх передачі через мережу.

Під час аналізу предметної області було визначено основні методи для приховування зображень, набуто знань про інші методи, а також існуючі програми та логіку їх роботи. На основі аналізу було створено технічне завдання, по якому далі було створено застосунок.

В результаті вибору та обґрунтування структури системи, яка проектується було визначено архітектуру майбутньої системи, які розділена на модулі. Також було визначено середовища для розробки та головні інструменти, які будуть використовуватись.

Під час розробки програми було додано в тому числі інші інструменти, а також прийнято багато рішень щодо реалізації системи. Всі рішення були описані в відповідно пункті пояснювальної записки.

Після реалізації програми було створено керівництво програміста, де описано як запускається та працює програма та як в неї вносити зміни.

Було створено керівництво оператора, в якому детально описаний весь функціонал застосунку та як його використовувати.

На цьому етапі було створено додадок «Опис програми», який містить в собі детальний опис програми та взаємодії з нею, а також специфікацію.

Останнім етапом було проведено тестування застосунку. Тестування показало, що застосунок працює ефективно та відповідає технічному завданню. Також в результаті тестування було знайдено моменти, які можна буде покращити під час підтримки застосунку.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Що таке стеганографія: принципи та застосування в кібербезпеці [Електронний ресурс] // HackYourMom. – 2023. – Режим доступу: <https://hackyourmom.com/en/osvita/shho-take-steganografiya-prynczyipy-ta-zastosuvannya-v-kiberbezpeczi/>.
2. Hong W., Chen T. S. A novel data embedding method using adaptive pixel pair matching // IEEE Transactions on Information Forensics and Security. – 2012. – Vol. 7. – Pp. 176-184.
3. The Exo guide to data cloaking [Електронний ресурс] // Exo Substack. – Режим доступу: <https://exo.substack.com/p/the-exo-guide-to-data-cloaking>.
4. Akar F., Yalman Y., Varol H. Data hiding in digital images using a partial optimization technique based on classical LSB method // Turkish Journal of Electrical Engineering & Computer Sciences. – 2013. – Vol. 21. – Pp. 10.3906/elk-1205-58.
5. Steganography in art and design [Електронний ресурс] // The Guardian. – 2008. – Режим доступу: <https://www.theguardian.com/artanddesign/2008/jul/31/2>.
6. Elbasi E. Robust multimedia watermarking: hidden markov model approach for video sequences // Turkish Journal of Electrical Engineering and Computer Sciences. – 2010. – Vol. 18. – Pp. 159-170.
7. Rahman S., Uddin J., Hussain H. A novel and efficient digital image steganography technique using least significant bit substitution // Scientific Reports. – 2017. – Vol. 7.
8. Menon N. A survey on image steganography // International Conference on Technological Advancements in Power and Energy (TAP Energy). – IEEE, 2017. – Pp. 1-5.
9. Pilia U., Tanwar R., Gupta P. A roadmap of steganography tools: conventional to modern // Spatial Information Research. – 2021. – Vol. 29. – Pp. 10.1007/s41324-021-00393-7.

10. HTML: HyperText Markup Language [Электронный ресурс] // MDN Web Docs. – Режим доступа: <https://developer.mozilla.org/en-US/docs/Web/HTML>.
11. Simpson K. You Don't Know JS Yet: Get Started. – 2nd ed. – O'Reilly Media, 2020. – 160 p.
12. JavaScript Reference [Электронный ресурс] // MDN Web Docs. – Режим доступа: <https://developer.mozilla.org/ru/docs/Web/JavaScript>.
13. Ousterhout J. A Philosophy of Software Design. – Yaknyam Press, 2018. – 178 p.
14. Why leading companies use Go [Электронный ресурс] // Go.dev. – Режим доступа: <https://go.dev/solutions/google>.
15. Neovim Documentation [Электронный ресурс]. – Режим доступа: <https://neovim.io/doc>.
16. Git Documentation [Электронный ресурс]. – Режим доступа: <https://git-scm.com/docs>.
17. Holzmann G. J. The Power of Ten – Rules for Developing Safety Critical Code. – NASA/JPL Laboratory for Reliable Software, 2006.
18. TypeScript Documentation [Электронный ресурс]. – Режим доступа: <https://www.typescriptlang.org/docs/>.
19. TinyGo: Go compiler for small places [Электронный ресурс]. – Режим доступа: <https://tinygo.org/>.
20. Can I use... Support tables for HTML5, CSS3, etc [Электронный ресурс]. – Режим доступа: <https://caniuse.com/>.

ДОДАТОК А

Технічне завдання

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Запорізька політехніка»

ТЕХНІЧНЕ ЗАВДАННЯ

до курсового проєкту з дисципліни «Безпека та захист програм і даних»

(назва дисципліни)

на тему: Розробка програмного засобу стеганографічного захисту інформації
в цифрових зображеннях

Студента 4 курсу КНТ-113сп групи
спеціальності 121 Інженерія
програмного забезпечення
освітня програма (спеціалізація) інженерія
програмного забезпечення
Щедровського І. А.

(прізвище та ініціали)

Керівник доцент, Зайко Т. А.

(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Затверджую

(підпис)

Зайко Т. А.

(прізвище та ініціали)

2025 рік

A.1 Вступ

Найменуванням розробки є програмне забезпечення для стеганографічного приховування інформації в цифрових зображеннях.

Програма призначена для вбудовування конфіденційних даних у візуальні носії таким чином, щоб сам факт передачі повідомлення залишався непомітним для стороннього спостерігача.

Областю застосування виробу є сфера захисту інформації та конфіденційного зв'язку через відкриті канали мережі інтернет. На відміну від криптографії, розробка дозволяє уникнути привернення уваги до зашифрованих даних, забезпечуючи високу місткість приховування та візуальну цілісність носія.

Об'єктом, у якому використовується програма, є системи захищеного обміну файлами та персональні робочі станції користувачів. Програмний виріб застосовується як інструмент для автоматизації процесів маскування секретних повідомлень у графічних файлах перед їх відправленням або зберіганням

A.2 Призначення розробки

Призначенням розробки програмного вибору є виконання курсового проекту з дисципліни «Безпека та захист програм та даних». Програма реалізується як практична частина роботи для підтвердження теоретичних знань у галузі стеганографічного захисту інформації.

Функціональне призначення виробу полягає у забезпеченні прихованого передавання даних шляхом їх вбудовування у цифрові зображення-носії. Програма буде автоматизувати процеси вбудовування та подальшого вилучення секретної інформації без візуального спотворення носія.

Експлуатаційне призначення розробки передбачає її використання як інструменту для захисту конфіденційної кореспонденції у відкритих мережах.

Виріб орієнтований на користувачів, які потребують засобів прихованого зв'язку, де сам факт передачі інформації має залишатися таємним.

А.3 Вимоги до програми чи програмному виробу

А.3.1 Вимоги до функціональних характеристик

Застосунок повинен працювати на основі приховування даних всередині зображення-обкладинки, тобто те, в яке будуть вбудовуватись дані. Користувач повинен мати змогу завантажувати його як файл або вставляти з буфера обміну.

Зображення-обкладинка повинно бути в одному з форматів: BMP, JPEG та PNG. Такий перелік забезпечить достатнє покриття та не переускладнить функціонал застосунку

На зображенні А.1 показані найбільш популярні програми для стеганографії та формати зображень, які вони підтримують [9].

Tool name	Concealed secret information	Carrier file type				Technique for concealing secret information	Additional features
		Image file	Audio file	Video file	Text file		
Steghide	Plain text	JPEG,BMP	Wav, AU	–	–	Graphic theoretical technique	Encryption, Hashing
SteganPEG	Any file type	JPEG	–	–	–	Partial decoding technique	Encryption, Password protection
OurSecret 2.5	Any file type	BMP, JPG, PNG	Wav	Mp4	–	–	Encryption
Snow	Plain text	–	–	–	Text	Whitespace	–
S-tool	Plain text	GIF, BMP	Wav	–	–	LSB, Large capacity of concealing	Encryption, Hashing, Compression
OpenStego	Plain text	JPEG, JPG, BMP, GIF, PNG	–	–	–	LSB Watermarking	Encryption
DeepSound 2.0	Plain text	JPEG	Wav, Flac	–	–	–	Encryption, Password protection
StegoStick	Plain Text	BMP	Wav	Mp4	PDF, EXE, CHM	–	Encryption
DeEgger Embedder	Any file type	AVI, JPG, PNG	Mp3	Mp4	–	–	Password protection
OpenPuff	Any file type	BMP, JPG, PCX, PNG, TGA	AIFF, MP3, WAV	3GP, MP4, MPG, VOB	PDF, SWF, FLV	DCT	Encryption, Scrambling, Whitening, Encoding
wbStego4	Any file type	JPEG, BMP	Wav	Mp4	ASCII, HTML, PDF	–	Encryption
and Seek	Any file type	BMP,GIF	Mp3	–	–	LSB	Cryptography, Password protection
VSL 1.1	Plain text	GIF	–	–	–	LSB, DCT	Classifiers SVM and BSM
Outguess-Rebirth	Plain text	JPEG	–	–	–	DCT	Cryptography
JSteg	Plain text	JPEG	–	–	–	LSB, DCT	Encryption, Compression
F5 Steganography	Plain text	JPEG	–	–	–	F5 DCT	Password protection

Рисунок А.1 – Порівняння існуючих інструментів стеганографії

Дані, які будуть приховуватись повинні завантажуватись в вигляді файлу. Застосунок повинен працювати з будь-якими типами файлів приховуючи їх біт за бітом.

Важливо зауважити, що програма не буде підтримувати методи шифрування даних, які приховуються. Методів шифрування даних досить багато та їх реалізація буде надлишковою.

Застосунок повинен мати можливість приховувати дані з використанням двох популярних методів стеганографії, а саме приховування інформації з використанням найменш значущого біта, LSB, а також з використанням сегментації складності бітових площин, BPCS.

Але, в той самий час програма буде підтримувати простий секретний ключ для стеганографії даних. Цей ключ буде працювати тільки з режимом LSB та буде включати версію ключа, місце початку шифрування, які біти шифруються та які ні, які канали зображення шифруються і так далі. Цей функціонал є дещо надлишковим, але він дасть більш гнучкий контроль над приховуванням інформації для забезпечення гарного рівня безпеки.

Застосунок повинен мати можливість не тільки приховувати інформацію, а також виконувати її відновлення для обох методів.

Час виконання операцій приховування та відновлення даних залежить від розміру зображення-обкладинки та обсягу прихованої інформації і не повинен перевищувати розумні межі для інтерактивної роботи користувача.

A.3.2 Вимоги до надійності

Оскільки застосунок буде працювати як вебсайт, то він буде надаватись користувачу через мережу Інтернет. Сам застосунок не повинен напряду взаємодіяти з мережею Інтернет. Тобто, в ньому не буде реклами, статистики й так далі.

Всі обчислення всередині програми повинні виконуватись строго на клієнтській частині. Сервер тільки надає файли для виконання.

Застосунок не повинен взаємодіяти зі сховищами інформації в браузері, камерою, мікрофоном тощо. Це означає, що при перезавантаженні вкладки браузера або його закритті вся інформація повинна пропадати, для забезпечення найбільшої надійності даних.

В програмі повинна бути передбачена система помилок. Наприклад, коли користувач буде намагатись приховати більше інформації, а ніж фізично можливо всередині картинки, то в цьому випадку застосунок повинен показувати, що тільки деякий процент інформації може бути прихований. Або ж якщо користувач завантажує формат зображення-обкладинки, яке застосунок не підтримує.

Також в застосунку повинна бути система, яка повністю припиняє його роботу при виникненні внутрішньої помилки з даними, що називається Assertion Error, коли в якусь функцію приходять дані, які не відповідають заданим умовам. Подібні помилки мають свої правила та, наприклад, використовуються в NASA [17].

А.3.3 Умови експлуатації

Для експлуатації застосунку потрібен браузер з підтримкою стандарту WebAssembly (Chrome 57+, Firefox 52+, Safari 11+) [20].

Для використання застосунку користувачами він повинен знаходитись в мережі Інтернет та бути доступним для відкриття. Це забезпечується через доменне ім'я, через яке можна буде знайти вебсайт, а також сервер-хост, з якого можна буде завантажувати файли, щоб використовувати вебсайт.

Сам застосунок не потребуватиме іншої підтримки, окрім його розміщення в мережі, окрім можливого виправлення можливих помилок в його роботі та додавання нового функціоналу

Користувач застосунку повинен мати базові знання роботи з Інтернетом браузером, процес завантаження файлів в браузер тощо.

А.3.4 Вимоги до складі і параметрів технічних засобів

Для роботи програмного застосунку необхідний персональний комп'ютер користувача з доступом до мережі Інтернет.

Мінімальні технічні характеристики:

- процесор з архітектурою x86-64 або ARM64;
- не менше 4 ГБ оперативної пам'яті;
- не менше 100 МБ вільного місця на накопичувачі для тимчасових файлів браузера;
- будь-який графічний адаптер, що підтримує сучасні браузери;

– базові пристрої введення, а саме клавіатура, миша або сенсорна панель тощо.

Додаткове серверне обладнання не потребується.

A.3.5 Вимоги до інформаційної і програмної сумісності

Зображення-обкладинка, тобто зображення, в якому буде приховуватись інформація, повинне бути в форматі BMP, JPEG або PNG.

Прихована інформація повинна надаватись в вигляді файлу в будь-якому форматі. Застосунок не підтримує завантаження декількох файлів з прихованою інформацією

На виході програми буде одне зображення, яке буде містити приховану інформацію, та, для методу LSB, простий секретний ключ для її дешифрування в вигляді бінарного файлу. Якщо оригінальне зображення було в форматі JPEG, то в результаті воно повинно бути в форматі PNG для збереження прихованих даних в ньому.

Застосунок буде написаний з використанням мови програмування Golang як основної, з використанням мови розмітки HTML та мови стилів CSS для візуальної частини, а також з використанням мови програмування JavaScript та технології WebAssembly для виконання коду Golang в браузері.

Вихідний код застосунку не буде залежати від сторонніх бібліотек в режимі виконання. В режимі розробки можливо використання спеціальних програм, які будуть автоматично перевіряти код на помилки, виконувати форматування тощо, для забезпечення його максимальної якості

A.3.6 Вимоги до транспортування й збереження

Програмний застосунок розповсюджується в електронному вигляді через мережу Інтернет.

Транспортування програмного виробу здійснюється шляхом передавання файлів з вебсервера до браузера користувача.

Збереження програмного виробу забезпечується на сервері-хості у вигляді файлів вебзастосунку.

Обмежень щодо терміну збереження програмного виробу не передбачається за умови збереження працездатності серверного обладнання.

Спеціальні умови зберігання та складування не потребуються.

A.4 Вимоги до програмної документації

Розроблена документація повинна містити п'ять основних документів:

- пояснювальна записка;
- технічне завдання;
- специфікація програмного забезпечення;
- опис програми;
- текст програми.

Пояснювальна записка повинна охоплювати:

- аналіз предметної області на основі якого виконується постановка вимог;
- опис проєктування системи;
- опис основних рішень щодо реалізації компонентів системи;
- керівництва для оператора та користувача;
- методика проведення тестування та його результати, включаючи конкретні приклади.

A.5 Стадії й етапи розробки

Курсовий проєкт передбачає виконання декількох етапів.

Огляд існуючого стану проблеми, який передбачає докладне вивчення предметної області та програмних засобів для автоматизації її основних

процесів. Цей етап завершується підготовкою першого розділу пояснювальної записки та погодженням технічного завдання.

Розроблення програмного забезпечення, що відповідає вимогам, визначеним у завданні. На цій стадії розробляється структурна схема системи, алгоритм функціонування, програмні модулі та інтерфейс користувача з можливістю роботи через мережу інтернет.

Оформлення документації до розробленого програмного забезпечення, що включає підготовку пояснювальної записки, специфікації, опису програми та її тексту.

Виконання тестування розробленого застосування на основі обраних наборів даних для перевірки відповідності функціональним вимогам.

Фінальним етапом є повне оформлення звітності та захист курсового проекту.

Виконавцем усіх робіт є студент-розробник. Терміни розробки визначаються навчальним планом протягом відповідного навчального семестру.

А.6 Порядок контролю й приймання

Порядок контролю та приймання розробки передбачає проведення функціонального тестування готового програмного продукту.

Основним видом іспитів є перевірка коректності приховування та вилучення даних з використанням контрольних прикладів зображень різних форматів та даних різних форматів в тому числі.

Загальною вимогою до приймання роботи є повна цілісність відновленої інформації, що підтверджується порівнянням початкових даних із тими, що були отримані після вилучення зі стего-зображення.

Також перевіряється коректність обробки помилок при спробах перевищення місткості носія, загальна стабільність роботи інтерфейсу.

Однією з основних вимог успішного приймання роботи є наявність повного пакета програмної документації, оформленої згідно з чинними стандартами та вимогами методичних вказівок.

Остаточне рішення про прийняття роботи приймається під час захисту проєкт на основі демонстрації всіх заявлених функцій.

ДОДАТОК Б

Специфікація

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Запорізька політехніка»

СПЕЦИФІКАЦІЯ

до курсового проєкту з дисципліни «Безпека та захист програм і даних»

(назва дисципліни)

на тему: Розробка програмного засобу стеганографічного захисту інформації
в цифрових зображеннях

Студента 4 курсу КНТ-113сп групи
спеціальності 121 Інженерія
програмного забезпечення
освітня програма (спеціалізація) інженерія
програмного забезпечення
Щедровського І. А.

(прізвище та ініціали)

Керівник доцент, Зайко Т. А.

(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Затверджую

(підпис)

Зайко Т. А.

(прізвище та ініціали)

2025 рік

Позначення	Найменування	Примітка
	Документація	
КП.121.001.ОП	Опис програми	
	Комплекси	
	Компоненти	
INDEX-JS	Головний файл логіки клієнтської частини. Програмний модуль	
INDEX-HTML	Основний контент вебсторінки. Програмний модуль	
INDEX-CSS	Стили вебсторінки. Програмний модуль	
WASM	Головний модуль ініціалізації WebAssembly та роботи з клієнтською частиною. Програмний модуль	
JS-GO	Модуль функцій для перетворення інформації з JavaScript в Golang та навпаки. Програмний модуль	
STEGO-LSB	Модуль реалізації логіки LSB. Програмний модуль	
STEGO-BPCS	Модуль реалізації логіки BPCS. Програмний модуль	

STEGO	Модуль для взаємодії з методами LSB та BPCS. Програмний модуль	
IMAGEIO	Пакет для перетворень зображень в внутрішній тип Golang та навпаки. Програмний пакет	
ASSERT	Пакет для використання тверджень в коді. Програмний пакет	

ДОДАТОК В

Опис програми

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Запорізька політехніка»

ОПИС ПРОГРАМИ

до курсового проєкту з дисципліни «Безпека та захист програм і даних»

(назва дисципліни)

на тему: Розробка програмного засобу стеганографічного захисту інформації
в цифрових зображеннях

Студента 4 курсу КНТ-113сп групи
спеціальності 121 Інженерія
програмного забезпечення
освітня програма (спеціалізація) інженерія
програмного забезпечення
Щедровського І. А.

(прізвище та ініціали)

Керівник доцент, Зайко Т. А.

(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Затверджую

(підпис)

Зайко Т. А.

(прізвище та ініціали)

В.1 Загальні відомості

Програмний засіб має повне найменування «Застосунок для стеганографічного приховування даних у цифрових зображеннях за методами LSB та BPCS».

Для функціонування програми необхідне наступне програмне забезпечення:

- сучасний браузер з підтримкою стандарту WebAssembly. Наприклад Chrome 57+, Firefox 52+ або Safari 11+ [20];
- операційна система з підтримкою одного із зазначених браузерів . Найбільш популярними є Windows, Linux, macOS, Android або iOS;
- для збірки та модифікації проєкту (програмісту) необхідно встановити компілятор Go, версія зазначена в вихідних файлах проєкту Golang, компілятор TinyGo, який використовується для оптимізації WASM-модуля.

Програма написана на Golang, JavaScript разом з HTML та CSS.

В.2 Функціональне призначення

Програма призначена для розв'язання широкого класу задач у сфері стеганографічного захисту інформації, що включає приховування конфіденційних даних усередині цифрових зображень без візуальної зміни їхнього вигляду.

Основним завданням застосунку є створення прихованого каналу зв'язку, де секретна інформація інтегрується в структуру файлу-контейнера. Це дозволяє користувачеві не лише захистити зміст повідомлення, але й повністю приховати сам факт його існування, що є критично важливим для забезпечення високого рівня приватності.

Функціональні можливості системи охоплюють процеси інкапсуляції довільних байтових послідовностей у графічні об'єкти форматів PNG, BMP та JPEG. Завдяки реалізації двох різних підходів, а саме методу заміни найменш

значущих бітів LSB та сегментації складних бітових площин BPCS, програма дозволяє адаптуватися до різних типів зображень.

Застосування програми має певні функціональні обмеження, зумовлені математичними особливостями обраних методів та специфікою вебсередовища. Головне обмеження стосується обсягу секретних даних, який не може перевищувати фізичну місткість контейнера; при цьому збільшення параметрів безпеки, як-от інтервал між пікселями в ключі LSB, пропорційно зменшує корисний об'єм.

Також важливою умовою є збереження стего-об'єкта виключно у форматах без втрат, оскільки повторне стиснення у формат JPEG після вбудовування призведе до незворотного руйнування прихованих бітів.

Продуктивність та стабільність обробки безпосередньо залежать від ресурсів оперативної пам'яті клієнтського пристрою, що може створювати затримки при роботі з графічними файлами надвеликої роздільної здатності.

V.3 Опис логічної структури

Логічна побудова програми базується на розподілі обчислювальних потужностей між високорівневим графічним інтерфейсом та низькорівневим алгоритмічним ядром.

Основний алгоритм функціонування реалізовано за допомогою технології WebAssembly, що дозволяє виконувати скомпільований Go-код безпосередньо у середовищі браузера. Процес роботи починається з ініціалізації віртуальної машини WASM, після чого програма переходить у стан очікування вхідних даних від користувача через подієву модель JavaScript.

Метод LSB працює шляхом послідовної модифікації молодших бітів кольорних каналів пікселів зображення-контейнера. Більш складний метод BPCS аналізує кожен бітову площину окремо, розділяючи її на блоки розміром 8x8. Програма обчислює складність кожного блоку і, якщо вона перевищує встановлений поріг, замінює цей "шумовий" фрагмент на частину секретного

повідомлення, що забезпечує значно вищу місткість при збереженні візуальної цілісності.

Структура програмного забезпечення складається з декількох взаємопов'язаних компонентів. Пакет “main” забезпечує реєстрацію функцій у глобальному просторі імен браузеру та управління життєвим циклом модуля. Також цей модуль називається “wasm”, але для Golang важливо, щоб він називався “main”.

Пакет imageio відповідає за перетворення масивів байтів, отриманих з клієнтської частини, у внутрішні структури даних для подальшої маніпуляції пікселями.

Алгоритмічні пакети stego/lsb та stego/bpcs містять безпосередню логіку стеганографічних перетворень, будучи повністю незалежними від графічного інтерфейсу. А пакет stego виступає в ролі публічного API для взаємодії з методами.

Зв'язок між компонентами реалізовано через механізм Golang syscall/js, який виступає мостом для передачі вхідних зображень та параметрів від JavaScript до Go.

Після завершення обробки ядро повертає результат у вигляді модифікованого масиву байтів, який JavaScript конвертує у файл для завантаження, або показує користувачу напряду.

Програма не має зовнішніх зв'язків з іншими сторонніми сервісами чи серверами, оскільки весь цикл обробки інформації відбувається локально в межах однієї вкладки веббраузера, що гарантує високий рівень безпеки даних.

В.4 Використані технічні засоби

Основним пристроєм для роботи є персональний комп'ютер або робоча станція, обладнана центральним процесором з архітектурою x86-64 або ARM та тактовою частотою від 2.0 ГГц.

Оскільки основне обчислювальне навантаження припадає на виконання бінарного коду WebAssembly в межах браузера, пристрій повинен мати достатній запас потужності для маніпулювання великими масивами даних у пам'яті.

Критичним параметром технічних засобів є обсяг оперативної пам'яті, необхідний для стабільної роботи з графічними контейнерами високої роздільної здатності. Для обробки зображень формату Full HD та вище рекомендується використання систем з обсягом ОЗП не менше 4 ГБ, що дозволяє браузеру виділяти достатньо простору для "купи" (heap) WebAssembly-модуля.

Це мінімізує ризик виникнення помилок примусового завершення процесу через нестачу пам'яті під час виконання ітераційних алгоритмів LSB або аналізу бітових площин у BPCS.

Програма також передбачає використання стандартних периферійних пристроїв введення та виведення інформації.

До них належать монітор із підтримкою кольоропередачі True Color (24 або 32 біти на піксель) для коректного візуального контролю стего-об'єктів, а також клавіатура та маніпулятор типу "миша" для взаємодії з елементами інтерфейсу.

Мережеві адаптери використовуються лише для початкового отримання компонентів системи з сервера, після чого технічні засоби забезпечують автономну роботу програми без необхідності постійного підключення до мережі.

В.5 Виклик і завантаження

Програма запускається локально шляхом запуску локального серверу або через мережевий протокол HTTP/HTTPS при розміщенні на сервері. Локально потрібно саме використовувати сервер, щоб уникнути проблем з не правильним завантаженням WebAssembly.

Процес завантаження є двохетапним. Спочатку браузер виконує скрипти JavaScript, після чого виконується асинхронний запит на отримання бінарного файлу `wasm`. Вхідною точкою в програму з боку інтерфейсу є метод ініціалізації у файлі `“index.js”`, який створює екземпляр об'єкта `Go` на основі отриманого бінарного файлу.

Логічне ядро програми має вхідну точку у вигляді функції `“main”` у середовищі `Go`, яка після виконання початкових налаштувань переходить у режим очікування зовнішніх викликів.

Загальний обсяг скомпільованого `WASM`-модуля складає приблизно 1.3 МБ, за умови використання `TinyGo`, що забезпечує швидке розгортання в оперативній пам'яті.

Використання пам'яті динамічно регулюється браузером.

Основна частина адресного простору виділяється під лінійну пам'ять `WebAssembly` для зберігання масивів пікселів зображення, обсяг якої залежить від розміру оброблюваного файлу-контейнера.

Вихідний код застосунку знаходиться в відкритому доступі на платформі `Github` та доступний за посиланням <https://github.com/ltlaitoff/steganography>.

В.6 Початкові дані, вихідні дані

Початкові дані програми складаються з файлу-контейнера, секретного повідомлення та параметрів конфігурації алгоритму.

Основним об'єктом є цифрове зображення у форматах `PNG`, `BMP` або `JPEG`, яке повинно мати достатню роздільну здатність для розміщення запланованого обсягу інформації.

Попередня підготовка початкових даних системою включає декодування графічного файлу з використанням спеціальних алгоритмів, які в тому числі перевіряють чи є дані правильними.

Секретне повідомлення, що підлягає приховуванню, сприймається програмою як послідовність байтів незалежно від його початкового формату, який був переданий через поле вводу на клієнтській частині.

Додатково до початкових даних належать керуючі параметри, а саме ключ налаштувань для алгоритму LSB.

Вихідними даними програми є стего-зображення – модифіковане цифрове зображення, яке візуально не відрізняється від оригіналу, але містить у своїй структурі приховану інформацію.

Організація вихідних даних передбачає формування файлу у форматах без втрат, зокрема PNG або BMP. Вибір цих форматів є критичним, оскільки вони забезпечують збереження кожного біта інформації у незмінному вигляді після запису на носій, що гарантує можливість подальшого успішного вилучення повідомлення.

Формат вихідних даних суворо відповідає специфікаціям обраного графічного стандарту. Кодування здійснюється шляхом перезапису цільових бітів у відповідних байтах масиву зображення з наступною упаковкою в бінарний контейнер засобами внутрішнього пакету `imageio`, який використовує механізми `Golang` для перетворення форматів.

У режимі вилучення вихідними даними є відновлений файл або текстовий рядок, формат і кодування якого повністю збігаються з оригінальними початковими даними.

Результат вилучення надається користувачеві через інтерфейс браузера як готовий до завантаження об'єкт, що забезпечує коректну інтерпретацію отриманої інформації операційною системою.

ДОДАТОК Д
Текст програми

```
FILE: js-go-adapter.go
//go:build js && wasm
```

```
package main
```

```
import (
    "syscall/js"
)
```

```
func JSToGoBytes(jsImageBytes js.Value) []byte {
    goImageBytes := make([]byte, jsImageBytes.Get("byteLength").Int())
    js.CopyBytesToGo(goImageBytes, jsImageBytes)

    return goImageBytes
}
```

```
func GoToJsBytes(goImageBytes []byte) js.Value {
    uint8Array := js.Global().Get("Uint8Array").New(len(goImageBytes))
    js.CopyBytesToJS(uint8Array, goImageBytes)

    return uint8Array
}
```

```
func JsError(message string) any {
    return js.ValueOf(map[string]any{
        "ok":    false,
        "message": message,
    })
}
```

```
=====
```

```
FILE: README.md
# steganography
```

```
=====
```

```
FILE: wasm.go
//go:build js && wasm
```

```
package main
```

```
import (
    "fmt"
    "syscall/js"

    "github.com/ltlaitoff/steganography/stego"
)
```

```
func encodeLsbWrapper(this js.Value, args []js.Value) interface{} {
    imageType := args[1].String()
    message := JSToGoBytes(args[2])
    containerImage := JSToGoBytes(args[0])
    key := args[3].String()

    encodedImage, err := stego.EncodeLSB(containerImage, imageType, message, key)

    if err != nil {
        return JsError(err.Error())
    }
}
```

```

        return js.ValueOf(map[string]any{
            "ok":    true,
            "data": GoToJsBytes(encodedImage),
        })
    }

func decodeLsbWrapper(this js.Value, args []js.Value) interface{} {
    fmt.Println("[GO]: Run Decode LSB")

    image := JSToGoBytes(args[0])
    imageType := args[1].String()
    key := args[2].String()

    result, err := stego.DecodeLSB(image, imageType, key)

    if err != nil {
        return JsError(err.Error())
    }

    return js.ValueOf(map[string]any{
        "ok":    true,
        "data": result,
    })
}

func encodeBpcsWrapper(this js.Value, args []js.Value) interface{} {
    fmt.Println("[GO]: Run BPCS Encode")

    imageType := args[1].String()
    message := JSToGoBytes(args[2])
    containerImage := JSToGoBytes(args[0])

    encodedImage, err := stego.EncodeBPCS(containerImage, imageType, message)

    if err != nil {
        return JsError(err.Error())
    }

    return js.ValueOf(map[string]any{
        "ok":    true,
        "data": GoToJsBytes(encodedImage),
    })
}

func decodeBpcsWrapper(this js.Value, args []js.Value) interface{} {
    fmt.Println("[GO]: Run Decode LSB")

    image := JSToGoBytes(args[0])
    imageType := args[1].String()

    result, err := stego.DecodeBPCS(image, imageType)

    if err != nil {
        return JsError(err.Error())
    }

    return js.ValueOf(map[string]any{
        "ok":    true,
        "data": result,
    })
}

```

```

func debug(this js.Value, args []js.Value) interface{} {
    debugMode := args[0].Bool()
    stego.SetDebugMode(debugMode)

    return nil
}

func parseLSBKey(this js.Value, args []js.Value) interface{} {
    key := args[0].String()

    result, err := stego.ParseLsbKey(key)

    fmt.Println("Parsed!", result)

    if err != nil {
        return JsError(err.Error())
    }

    // Cast for js.ValueOf
    channels := make([]any, len(result.Channels))
    for i := range channels {
        channels[i] = result.Channels[i]
    }

    return js.ValueOf(map[string]any{
        "ok": true,
        "data": map[string]any{
            "StartX":      result.StartX,
            "StartY":      result.StartY,
            "EndX":         result.EndX,
            "EndY":         result.EndY,
            "GapX":         result.GapX,
            "GapY":         result.GapY,
            "ChannelsPerPixel": result.ChannelsPerPixel,
            "Channels":      js.ValueOf(channels),
        },
    })
}

func main() {
    c := make(chan bool)

    js.Global().Set("goEncodeLSB", js.FuncOf(encodeLsbWrapper))
    js.Global().Set("goDecodeLSB", js.FuncOf(decodeLsbWrapper))
    js.Global().Set("goParseLSBKey", js.FuncOf(parseLSBKey))

    js.Global().Set("goEncodeBPCS", js.FuncOf(encodeBpcsWrapper))
    js.Global().Set("goDecodeBPCS", js.FuncOf(decodeBpcsWrapper))

    js.Global().Set("goDebug", js.FuncOf(debug))

    <-c
}

=====

FILE: pkg\assert\assert.go
package assert

func Assert(condition bool, message string) {

```

```

        if !condition {
            panic(message)
        }
    }
}

=====

FILE: pkg\imageio\imageio.go
package imageio

import (
    "bytes"
    "fmt"
    "image"
    "image/jpeg"
    "image/png"

    "github.com/ltlaitoff/steganography/pkg/assert"
    "golang.org/x/image/bmp"
)

func ParseImage(imageBytes []byte, imageType string) (image.Image, error) {
    assert.Assert(imageType != "", "Image type should have value")

    switch imageType {
    case "image/png":
        img, err := png.Decode(bytes.NewReader(imageBytes))

        if err != nil {
            return nil, fmt.Errorf("Unable to decode png")
        }

        return img, nil

    case "image/jpeg":
        img, err := jpeg.Decode(bytes.NewReader(imageBytes))
        if err != nil {
            return nil, fmt.Errorf("Unable to decode jpeg")
        }

        return img, nil

    case "image/bmp":
        img, err := bmp.Decode(bytes.NewReader(imageBytes))

        if err != nil {
            return nil, fmt.Errorf("Unable to decode bmp")
        }

        return img, nil
    }

    return nil, fmt.Errorf("Invalid image format")
}

func EncodeImage(image image.Image, imageType string) ([]byte, error) {
    assert.Assert(imageType != "", "Image type should have value")

    buf := new(bytes.Buffer)

    switch imageType {

```

```

    case "image/png":
        if err := png.Encode(buf, image); err != nil {
            return nil, fmt.Errorf("unable to encode png")
        }
    case "image/jpeg":
        if err := jpeg.Encode(buf, image, nil); err != nil {
            return nil, fmt.Errorf("Unable to encode jpeg")
        }
    case "image/bmp":
        if err := bmp.Encode(buf, image); err != nil {
            return nil, fmt.Errorf("Unable to encode bmp")
        }
    }

    return buf.Bytes(), nil
}

```

=====

FILE: stego\stego.go
package stego

```

import (
    "encoding/binary"
    "fmt"
    "image"
    "image/draw"
    "reflect"
    "strconv"
    "unicode"

    "github.com/ltlaitoff/steganography/pkg/assert"
    "github.com/ltlaitoff/steganography/pkg/imageio"
    "github.com/ltlaitoff/steganography/stego/bpcs"
    "github.com/ltlaitoff/steganography/stego/lsb"
)

type State struct {
    debugMode bool
}

var state State = State{
    debugMode: false,
}

func SetDebugMode(debugMode bool) {
    state.debugMode = debugMode
}

func ParseLsbKey(key string) (*lsb.Key, error) {
    parsingSchema := map[rune]string{
        'S': "StartX",
        'T': "StartY",
        'E': "EndX",
        'N': "EndY",
        'H': "GapX",
        'V': "GapY",
        'P': "ChannelsPerPixel",
        'C': "Channels",
    }
}

```

```

result := &lsb.Key{}
val := reflect.ValueOf(result).Elem()

current := ""
buffer := ""

saveCurrent := func() error {
    assert.Assert(current != "", "TODO 4")
    field := val.FieldByName(current)

    assert.Assert(field.IsValid(), fmt.Errorf("Field is not valid", field,
"current:", current).Error())

    if current == "Channels" {
        assert.Assert(field.Kind() == reflect.Slice, "Channels key field
should be slice!")

        field.Set(reflect.Append(field, reflect.ValueOf(buffer)))
    } else {
        assert.Assert(field.Kind() == reflect.Int, "Other fields without
Channels should be int!")
        num, err := strconv.Atoi(buffer)

        if err != nil {
            return err
        }

        field.SetInt(int64(num))
    }

    return nil
}

for _, char := range key {
    if unicode.IsLetter(char) {
        if property, ok := parsingSchema[char]; ok {
            newField := val.FieldByName(property)

            if newField.IsValid() {
                if current != "" {
                    err := saveCurrent()

                    if err != nil {
                        return nil, err
                    }
                }

                current = property
                buffer = ""
                continue
            } else {
                buffer += string(char)
            }
        } else {
            buffer += string(char)
        }
    } else {
        buffer += string(char)
    }
}

if current != "" {

```

```

        saveCurrent()
    }

    return result, nil
}

// Default is png, for jpeg png too
func getResultImageType(imageType string) string {
    if imageType == "image/png" {
        return "image/png"
    }

    if imageType == "image/bmp" {
        return "image/bmp"
    }

    return "image/png"
}

// TODO:: Remove imageType?

func appendSecretLengthToSecret(message []byte) []byte {
    secretLength := make([]byte, 4)
    binary.LittleEndian.PutUint32(secretLength, uint32(len(message)))

    return append(secretLength, message...)
}

func EncodeLSB(imageBytes []byte, imageType string, message []byte, key string)
([]byte, error) {
    assert.Assert(imageType != "", "Image type should have value")

    inputImage, err := imageio.ParseImage(imageBytes, imageType)

    if err != nil {
        return nil, err
    }

    lsbKey, err := ParseLsbKey(key)

    if err != nil {
        return nil, err
    }

    fmt.Println("Key:", lsbKey)

    options := lsb.Options{
        VisualDebug: state.debugMode,
        Key:         *lsbKey,
    }

    encodedImage, err := lsb.Encode(inputImage,
appendSecretLengthToSecret(message), options)

    if err != nil {
        return nil, err
    }

    encodedBytes, err := imageio.EncodeImage(encodedImage,
getResultImageType(imageType))

    if err != nil {
        return nil, err
    }

```

```

    }

    return encodedBytes, nil
}

func DecodeLSB(imageBytes []byte, imageType string, key string) (string, error) {
    assert.Assert(imageType != "", "Image type should have value")
    inputImage, err := imageio.ParseImage(imageBytes, imageType)

    if err != nil {
        return "", err
    }

    lsbKey, err := ParseLsbKey(key)

    if err != nil {
        return "", err
    }

    fmt.Println("Key:", lsbKey)

    options := lsb.Options{
        VisualDebug: state.debugMode,
        Key:         *lsbKey,
    }

    secretLengthString, err := lsb.Decode(inputImage, options, 4)

    if err != nil {
        return "", err
    }

    secretLength := binary.LittleEndian.Uint32(secretLengthString)

    result, err := lsb.Decode(inputImage, options, int(4+secretLength))

    if err != nil {
        return "", err
    }

    return string(result[4:]), nil
}

func EncodeBPCS(imageBytes []byte, imageType string, message []byte) ([]byte, error) {
    assert.Assert(imageType != "", "Image type should have value")

    inputImage, err := imageio.ParseImage(imageBytes, imageType)

    if err != nil {
        return nil, err
    }

    bounds := inputImage.Bounds()
    rgba := image.NewRGBA(image.Rect(0, 0, bounds.Dx(), bounds.Dy()))
    draw.Draw(rgba, rgba.Bounds(), inputImage, bounds.Min, draw.Src)

    err = bpcs.EncodeBPCS(rgba, appendSecretLengthToSecret(message))

    if err != nil {
        return nil, err
    }
}

```

```

        encodedBytes, err := imageio.EncodeImage(rgba, getResultImageType(imageType))

        if err != nil {
            return nil, err
        }

        return encodedBytes, nil
    }

func DecodeBPCS(imageBytes []byte, imageType string) (string, error) {
    assert.Assert(imageType != "", "Image type should have value")
    inputImage, err := imageio.ParseImage(imageBytes, imageType)

    if err != nil {
        return "", err
    }

    bounds := inputImage.Bounds()
    rgba := image.NewRGBA(image.Rect(0, 0, bounds.Dx(), bounds.Dy()))
    draw.Draw(rgba, rgba.Bounds(), inputImage, bounds.Min, draw.Src)

    secretLengthString := bpcs.DecodeBPCS(rgba, 4)
    secretLength := binary.LittleEndian.Uint32(secretLengthString)

    result := bpcs.DecodeBPCS(rgba, int(4+secretLength))

    return string(result[4:]), nil
}

```

=====

```

FILE: stego\bpcs\bpcs.go
package bpcs

import (
    "fmt"
    "image"
)

const (
    BlockSize  = 8
    Threshold  = 0.3
    MaxChanges = 112
)

func BinaryToGray(n uint8) uint8 {
    return n ^ (n >> 1)
}

func GrayToBinary(g uint8) uint8 {
    var b uint8
    for ; g > 0; g >>= 1 {
        b ^= g
    }
    return b
}

func CalculateCapacity(img *image.RGBA) int {
    capacityBits := 0
    bounds := img.Bounds()

```

```

    for plane := 0; plane < 8; plane++ {
        for y := bounds.Min.Y; y <= bounds.Max.Y-BlockSize; y += BlockSize {
            for x := bounds.Min.X; x <= bounds.Max.X-BlockSize; x += BlockSize
{
                var block [8][8]uint8
                for by := 0; by < BlockSize; by++ {
                    for bx := 0; bx < BlockSize; bx++ {
                        pixel := img.RGBAAt(x+bx, y+by)
                        block[by][bx] = (BinaryToGray(pixel.R) >>
plane) & 1
                    }
                }

                if CalculateComplexity(block) > Threshold {
                    capacityBits += 63
                }
            }
        }
    }
    return capacityBits
}

func CalculateComplexity(block [8][8]uint8) float64 {
    changes := 0
    for y := 0; y < BlockSize; y++ {
        for x := 0; x < BlockSize; x++ {
            if x < BlockSize-1 && block[y][x] != block[y][x+1] {
                changes++
            }
            if y < BlockSize-1 && block[y][x] != block[y+1][x] {
                changes++
            }
        }
    }
    return float64(changes) / float64(MaxChanges)
}

func Conjugate(block *[8][8]uint8) {
    for y := 0; y < BlockSize; y++ {
        for x := 0; x < BlockSize; x++ {
            if (x+y)%2 != 0 {
                block[y][x] ^= 1
            }
        }
    }
}

func EncodeBPCS(img *image.RGBA, secretData []byte) error {
    dataBitIndex := 0
    totalBits := len(secretData) * 8
    bounds := img.Bounds()

    maxBits := CalculateCapacity(img)

    if totalBits > maxBits {
        return fmt.Errorf("Insufficient capacity: need %d bits, have %d",
totalBits, maxBits)
    }

    for plane := 0; plane < 8; plane++ {
        for y := bounds.Min.Y; y <= bounds.Max.Y-BlockSize; y += BlockSize {

```

```

    for x := bounds.Min.X; x <= bounds.Max.X-BlockSize; x += BlockSize
{
    if dataBitIndex >= totalBits {
        return nil
    }

    var block [8][8]uint8
    for by := 0; by < BlockSize; by++ {
        for bx := 0; bx < BlockSize; bx++ {
            pixel := img.RGBAAt(x+bx, y+by)

            block[by][bx] = (BinaryToGray(pixel.R) >>
plane) & 1
        }
    }

    if CalculateComplexity(block) > Threshold {
        var dataBlock [8][8]uint8
        for i := 1; i < 64; i++ {
            if dataBitIndex < totalBits {
                byteIdx := dataBitIndex / 8
                shift := uint(7 - (dataBitIndex % 8))
                dataBlock[i/8][i%8] =
(secretData[byteIdx] >> shift) & 1
                dataBitIndex++
            }
        }

        if CalculateComplexity(dataBlock) <= Threshold {
            Conjugate(&dataBlock)
            dataBlock[0][0] = 1
        } else {
            dataBlock[0][0] = 0
        }

        for by := 0; by < BlockSize; by++ {
            for bx := 0; bx < BlockSize; bx++ {
                p := img.RGBAAt(x+bx, y+by)

                g := BinaryToGray(p.R)

                if dataBlock[by][bx] == 1 {
                    g |= (1 << plane)
                } else {
                    g &= ^(1 << plane)
                }

                p.R = GrayToBinary(g)
                img.SetRGBA(x+bx, y+by, p)
            }
        }
    }
}

return nil
}

func DecodeBPCS(img *image.RGBA, expectedSize int) []byte {
    secretData := make([]byte, expectedSize)
    dataBitIndex := 0

```

```

totalBits := expectedSize * 8
bounds := img.Bounds()

for plane := 0; plane < 8; plane++ {
    for y := bounds.Min.Y; y <= bounds.Max.Y-BlockSize; y += BlockSize {
        for x := bounds.Min.X; x <= bounds.Max.X-BlockSize; x += BlockSize
{
            if dataBitIndex >= totalBits {
                return secretData
            }

            var block [8][8]uint8
            for by := 0; by < BlockSize; by++ {
                for bx := 0; bx < BlockSize; bx++ {
                    pixel := img.RGBAAt(x+bx, y+by)
                    block[by][bx] = (BinaryToGray(pixel.R) >>
plane) & 1
                }
            }

            if CalculateComplexity(block) > Threshold {
                wasConjugated := block[0][0] == 1
                if wasConjugated {
                    Conjugate(&block)
                }

                for i := 1; i < 64; i++ {
                    if dataBitIndex >= totalBits {
                        return secretData
                    }
                    bit := block[i/8][i%8]
                    byteIdx := dataBitIndex / 8
                    shift := uint(7 - (dataBitIndex % 8))
                    if bit == 1 {
                        secretData[byteIdx] |= (1 << shift)
                    }
                    dataBitIndex++
                }
            }
        }
    }
}
return secretData
}

```

=====

```

FILE: stego\lsb\lsb.go
package lsb

```

```

import (
    "fmt"
    "image"
    "image/color"
    "image/draw"
)

```

```

type Key struct {
    StartX      int
    StartY      int
    EndX        int

```

```

    EndY          int
    GapX          int
    GapY          int
    ChannelsPerPixel int
    Channels      []string
}

type Options struct {
    // If enabled encoding will produce visible to eye result
    // Use to check what pixels are affected in result of algorithm
    VisualDebug bool

    // Key with LSB settings
    Key Key
}

func Encode(containerImage image.Image, message []byte, options Options) (*image.RGBA,
error) {
    bounds := containerImage.Bounds()
    rgba := image.NewRGBA(image.Rect(0, 0, bounds.Dx(), bounds.Dy()))
    draw.Draw(rgba, rgba.Bounds(), containerImage, bounds.Min, draw.Src)

    key := options.Key

    res := ""
    for _, r := range message {
        res += fmt.Sprintf("%08b", r)
    }

    x, y := key.StartX, key.StartY

    if key.StartX < bounds.Min.X {
        x = bounds.Min.X
    }

    if key.StartY < bounds.Min.Y {
        y = bounds.Min.Y
    }

    endX, endY := key.EndX, key.EndY

    if key.EndX == 0 || key.EndX > bounds.Max.X {
        endX = bounds.Max.X
    }

    if key.EndY == 0 || key.EndY > bounds.Max.Y {
        endY = bounds.Max.Y
    }

    pixelsCount := 0
    rowCount := (endY - y + key.GapY) / (key.GapY + 1)

    if rowCount <= 1 {
        pixelsCount = (endX - x) / (key.GapX + 1)
    } else {
        pixelsCount = ((bounds.Max.X - x + key.GapX) / (key.GapX + 1)) +
            ((bounds.Max.X - bounds.Min.X + key.GapX) / (key.GapX + 1) *
(rowCount - 2)) +
            ((endX - bounds.Min.X + key.GapX) / (key.GapX + 1))
    }

    capacityBits := pixelsCount * key.ChannelsPerPixel

```

```

    if len(res) > capacityBits {
        return nil, fmt.Errorf("Insufficient capacity: need %d bits, have %d",
len(res), capacityBits)
    }

    counter := 0
    channelCounter := 0

    channelsMap := map[string]bool{}

    for _, value := range key.Channels {
        channelsMap[value] = true
    }

    for _, char := range res {
        one := uint8(0)

        if char == '1' {
            one = 1
        }

        cr, cg, cb, ca := rgba.At(x, y).RGBA()

        r := uint8(cr >> 8)
        g := uint8(cg >> 8)
        b := uint8(cb >> 8)
        a := uint8(ca >> 8)

        currentChannel := key.Channels[channelCounter]

        if currentChannel == "R" && r&1 != one {
            r ^= 1
        }

        if currentChannel == "G" && g&1 != one {
            g ^= 1
        }

        if currentChannel == "B" && b&1 != one {
            b ^= 1
        }

        if options.VisualDebug {
            if key.ChannelsPerPixel == 1 {
                r, g, b = 0, 0, 0
            } else {
                if _, ok := channelsMap["R"]; !ok {
                    r = 0
                }

                if _, ok := channelsMap["G"]; !ok {
                    g = 0
                }

                if _, ok := channelsMap["B"]; !ok {
                    b = 0
                }
            }
        }

        if one == 1 {
            if currentChannel == "R" {

```

```

        r = 255
    }
    if currentChannel == "G" {
        g = 255
    }
    if currentChannel == "B" {
        b = 255
    }
}

if one == 0 {
    if currentChannel == "R" {
        r = 0
    }
    if currentChannel == "G" {
        g = 0
    }
    if currentChannel == "B" {
        b = 0
    }
}

}

rgba.Set(x, y, color.RGBA{r, g, b, a})

counter++
channelCounter++

if channelCounter >= len(key.Channels) {
    channelCounter = 0
}

if counter == key.ChannelsPerPixel {
    counter = 0
    x += 1 + key.GapX

    isXEnd := x >= bounds.Max.X

    if y+key.GapY >= endY {
        isXEnd = x >= endX
    }

    if isXEnd {
        y += 1 + key.GapY

        if y >= endY {
            break
        }

        x = bounds.Min.X
    }
}

}

return rgba, nil
}

func Decode(encodedImage image.Image, options Options, length int) ([]byte, error) {
    bounds := encodedImage.Bounds()
    rgba := image.NewRGBA(image.Rect(0, 0, bounds.Dx(), bounds.Dy()))
    draw.Draw(rgba, rgba.Bounds(), encodedImage, bounds.Min, draw.Src)
}

```

```

data := make([]byte, 0)
var newByte byte = 0

key := options.Key
i := 7

addByteToData := func() {
    i--
    if i < 0 {
        data = append(data, newByte)
        i = 7
        newByte = 0
    }
}

startX, startY := key.StartX, key.StartY

if key.StartX < bounds.Min.X {
    startX = bounds.Min.X
}

if key.StartY < bounds.Min.Y {
    startY = bounds.Min.Y
}

endX, endY := key.EndX, key.EndY

if key.EndX == 0 || key.EndX > bounds.Max.X {
    endX = bounds.Max.X
}

if key.EndY == 0 || key.EndY > bounds.Max.Y {
    endY = bounds.Max.Y
}

channelCounter := 0

for y := startY; y < endY; y += 1 + key.GapY {
    x := startX

    if y != startY {
        x = bounds.Min.X
    }

    for ; x < bounds.Max.X; x += 1 + key.GapX {
        cr, cg, cb, _ := rgba.At(x, y).RGBA()
        r := uint8(cr >> 8)
        g := uint8(cg >> 8)
        b := uint8(cb >> 8)

        for range key.ChannelsPerPixel {
            currentChannel := key.Channels[channelCounter]

            if currentChannel == "R" {
                if r&1 == 1 {
                    newByte = newByte | (1 << i)
                }

                addByteToData()
            }

            if currentChannel == "G" {

```

```

        if g&1 == 1 {
            newByte = newByte | (1 << i)
        }

        addByteToData()
    }

    if currentChannel == "B" {
        if b&1 == 1 {
            newByte = newByte | (1 << i)
        }

        addByteToData()
    }

    channelCounter++
    if channelCounter >= len(key.Channels) {
        channelCounter = 0
    }
}

if y+key.GapY >= endY && x >= endX {
    break
}

if len(data) >= length {
    return data, nil
}
}

return data, nil
}

```

=====

```

FILE: website\index.d.ts
type Methods = 'LSB' | 'BPCS'
type Operation = 'ENCODE' | 'DECODE'

type UIVariants = `${Methods}_${Operation}`

/*
 * LSB Encode:
 * Checkox type
 * Message
 * Input secret file
 * Key
 *
 * LSB Decode:
 * Key
 * Checkbox type
 * Message
 */

interface ElementInfo<T extends HTMLElement = HTMLElement> {
    id: string
    type: new () => T
}

interface Config {

```

```

wasmUrl: string

globalIds: {
  originalImageInput: ElementInfo<HTMLInputElement>
  originalImageInputDropZone: ElementInfo<HTMLLabelElement>
  originalImagePreview: ElementInfo<HTMLImageElement>
  resultImagePreview: ElementInfo<HTMLImageElement>
  submitButton: ElementInfo<HTMLButtonElement>
}

menu: {
  name: {
    methods: string
    operation: string
  }
  value: {
    methods: string[]
    operation: string[]
  }
}

UIIds: {
  menu: {
    base: ElementInfo<HTMLDivElement>
    methods: ElementInfo<HTMLDivElement>
  }
  lsbBlock: ElementInfo<HTMLDivElement>
  encodeBlock: ElementInfo<HTMLDivElement>
  decodeBlock: ElementInfo<HTMLDivElement>
  swapButton: ElementInfo<HTMLButtonElement>
  ERROR: {
    block: ElementInfo<HTMLDivElement>
    message: ElementInfo<HTMLDivElement>
    button: ElementInfo<HTMLButtonElement>
  }
}

ids: {
  DEBUG: ElementInfo<HTMLInputElement>
  SHARED: {
    secretAsFileCheckbox: ElementInfo<HTMLInputElement>
  }
  LSB: {
    keyInputBlock: ElementInfo<HTMLDivElement>
    keyInputOutput: ElementInfo<HTMLInputElement>
  }
  ENCODE: {
    secretMessageInputBlock: ElementInfo<HTMLLabelElement>
    secretFileInputBlock: ElementInfo<HTMLLabelElement>
    secretMessageInput: ElementInfo<HTMLInputElement>
    secretFileInput: ElementInfo<HTMLInputElement>
  }
  DECODE: {
    secretMessageOutput: ElementInfo<HTMLInputElement>
    secretFileOutputButton: ElementInfo<HTMLButtonElement>
    secretMessageOutputBlock: ElementInfo<HTMLLabelElement>
    secretFileOutputButtonBlock: ElementInfo<HTMLDivElement>
  }
}
// // message: "message",
// }
// ids: Object.freeze({

```

```

        // })
    }

    interface LSBKey {
        StartX: number
        StartY: number
        EndX: number
        EndY: number
        GapX: number
        GapY: number
        ChannelsPerPixel: number
        Channels: string[]
    }

    type LSBKeyParams = keyof LSBKey

    interface State {
        activeMethod: Methods
        activeOperation: Operation

        debugMode: boolean

        originalImageFile: File | undefined
        resultImageFile: File | undefined

        secretAsFile: boolean
        secretMessage: string

        encodeSecretFile: File | undefined
        decodedSecretFile: File | undefined

        LSB: {
            key: LSBKey
        }

        errorMessage: string
    }

    type Assert = (condition: boolean, message: string) => asserts condition

    type ConstructorReturnType<T extends new (...args: any) => any> = T extends new (
        ...args: any
    ) => infer R
        ? R
        : any

    type LoadElement = <T extends HTMLElement>(elementInfo: ElementInfo<T>) => T
    type FileToByteArray = (file: File) => Promise<Uint8Array>

    type ErrorHandler = (err: unknown) => void
    type ShowError = (message: string) => void

    type IsRecord = (value: unknown) => value is Record<string, unknown>

    /**
     * Functions from WASM Golang
     */

    interface GolangError {
        ok: false
        message: string
    }

```

```

interface GolangOk<T> {
    ok: true
    data: T
}

//prettier-ignore
declare function goEncodeLSB(image: Uint8Array, imageType: string, secretMessage:
Uint8Array, key: string): GolangError | GolangOk<Uint8Array>
//prettier-ignore
declare function goDecodeLSB(image: Uint8Array, imageType: string, key: string):
GolangError | GolangOk<Uint8Array>
//prettier-ignore
declare function goEncodeBPCS(image: Uint8Array, imageType: string, secretMessage:
Uint8Array): GolangError | GolangOk<Uint8Array>
//prettier-ignore
declare function goDecodeBPCS(image: Uint8Array, imageType: string): GolangError |
GolangOk<Uint8Array>
//prettier-ignore
declare function goDebug(debugMode: boolean): void
//prettier-ignore
declare function goParseLSBKey(key: string): GolangError | GolangOk<LSBKey>

interface Array<T> {
    includes(searchElement: any, fromIndex?: number): searchElement is T
}

interface ObjectConstructor {
    typedKeys<T>(obj: T): Array<keyof T>
}

interface JSON {
    parse(
        text: string,
        reviver?: (this: any, key: string, value: any) => any
    ): unknown
}

```

```

=====

```

```

FILE: website\index.html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta
      name="viewport"
      content="width=device-width, initial-scale=1.0"
    />
    <title>Steganography</title>
    <link
      rel="stylesheet"
      href="./main.css"
    />
  </head>
  <body>
    <div class="layout">
      <div
        class="error"
        id="error"
        data-active="false"

```

```

>
    <div
        class="error--message"
        id="error-message"
    >
        TEST
    </div>
    <button
        class="error--close"
        id="error-close"
    >
        X
    </button>
</div>

<div class="layout--block layout--block_main layout--block_center">
    <div
        class="menu"
        id="menu"
    >
        <div
            class="menu-block menu-block_main"
            id="menu-methods"
        >
            <label class="menu-block--element">
                LSB
                <input
                    checked
                    type="radio"
                    name="menu-method"
                    value="lsb"
                />
            </label>
            <label class="menu-block--element">
                BPCS
                <input
                    type="radio"
                    name="menu-method"
                    value="bpcs"
                />
            </label>
        </div>

        <div
            id="menu-operation"
            class="menu-block"
        >
            <label class="menu-block--element">
                Encode
                <input
                    checked
                    type="radio"
                    name="menu-operation"
                    value="encode"
                />
            </label>
            <label class="menu-block--element">
                Decode
                <input
                    type="radio"
                    name="menu-operation"
                    value="decode"

```

```

        />
      </label>
    </div>
  </div>
</div>
<div class="content">
  <hr class="divider" />
  <div class="block">
    <h2 class="block--title">Options</h2>
    <label class="input-label">
      <h2 class="input-title">
        Secret message as file instead of
      </h2>
      <input
        type="checkbox"
        id="secret-as-file"
      />
    </label>
  </div>
  <div
    id="encode"
    class="block"
  >
    <h2 class="block--title block--
title_primary">Encode</h2>
    <div class="block--elements">
      <label
        class="input-label"
        id="encode-secret-message-input-
block"
      >
        <div class="input-title">Secret
        <input
          id="encode-secret-message-
input"
          type="text"
        />
      </label>
      <label
        class="input-label"
        id="encode-secret-file-input-
block"
      >
        <h2 class="input-title">Secret
        <input
          type="file"
          id="encode-secret-file-
input"
        />
      </label>
    </div>
  </div>
  <div
    id="decode"

```

```

class="block"
>
<h2 class="block--title block--
title_secondary">Decode</h2>

<label
  class="input-label"
  id="decode-secret-message-output-block"
>
  <h2 class="input-title">Decoded
  message</h2>
  <input
    type="text"
    id="decode-secret-message-output"
  />
</label>

<div id="decode-secret-file-output-button-
block">
  <button
    class="button"
    id="decode-secret-file-output-
button"
  >
    Download secret file
  </button>
</div>
<div id="lsb">
  <div
    class="block"
    id="lsb-secret-key-block"
  >
    <h2 class="block--title">LSB Key</h2>
    <div class="block--elements">
      <label class="input-label">
        <h2 class="input-
title">Key</h2>
        <input
          type="text"
          name="key"
          id="lsb-secret-key"
        />
      </label>
      <label class="input-label">
        <h2 class="input-
title">Start position of X</h2>
        <input
          value="0"
          type="number"
          name="StartX"
        />
      </label>
      <label class="input-label">
        <h2 class="input-
title">Start position of Y</h2>
        <input
          value="0"
          type="number"
          name="StartY"

```

is image end):

is image end):

beetween X:</h2>

beetween Y:</h2>

used per one pixel?

name="ChannelsPerPixel"

Example: "RBG", "RB", "G"

```

/>
</label>
<label class="input-label">
  <h2 class="input-title">
    End position of X (0

  </h2>
  <input
    value="0"
    type="number"
    name="EndX"
  />
</label>
<label class="input-label">
  <h2 class="input-title">
    End position of Y (0

  </h2>
  <input
    value="0"
    type="number"
    name="EndY"
  />
</label>
<label class="input-label">
  <h2 class="input-title">Gap

  <input
    value="0"
    type="number"
    name="GapX"
  />
</label>
<label class="input-label">
  <h2 class="input-title">Gap

  <input
    value="0"
    type="number"
    name="GapY"
  />
</label>
<label class="input-label">
  <h2 class="input-title">
    How much channels

  </h2>
  <input
    value="3"
    type="number"

  />
</label>
<label class="input-label">
  <h2 class="input-title">
    Used channels?

  </h2>
  <input
    value="RBG"
    type="text"

```

```

name="Channels"
/>
</label>
</div>
</div>
</div>
</div>
</div>
</div>
<div class="layout--block layout--block_secondary">
  <div class="images">
    <div class="block block_images">
      <h2 class="block--title">Original image</h2>
      
      <label
class="file-drop--label"
id="original-input-drop-zone"
>
        <h2 class="file-drop--title">
          Drop images here, or click to
upload.
        </h2>
        <input
class="file-drop--input"
type="file"
id="original-input"
name="Image container"
accept=".bmp,.png,.jpg"
/>
      </label>
    </div>
    <button
class="swap-button"
id="swap"
>
      <!-- SVG from:
https://www.svgrepo.com/svg/491145/swap -->
      <svg
class="swap-button--icon"
xmlns="http://www.w3.org/2000/svg"
viewBox="0 0 24 24"
>
        <path
stroke="currentColor"
stroke-linecap="round"
stroke-linejoin="round"
stroke-width="2"
d="M21 7.5H8m13 0L16.667 3M21 7.5
16.667 12M4 16.5h13m-13 0L8.333 21M4 16.5 8.333 12"
/>
      </svg>
    </button>
    <div class="block block_images">
      <h2 class="block--title">Result image</h2>
      
</div>
</div>

<div class="block">
    <h2 class="block--title">Run options</h2>

    <label class="input-label debug-label">
        <h2 class="input-title">Debug</h2>
        <input
            type="checkbox"
            id="debug"
        />
    </label>

    <button
        class="submit-button"
        id="submit-button"
    >
        Submit
    </button>
</div>
</div>

<script src="./wasm_exec.js"></script>
<script
    type="module"
    src="./index.js"
></script>
</body>
</html>

```

```
=====
```

```

FILE: website\index.js
Object.keys = Object.keys

/**
 * @type {Record<UIVariants, string>}
 */
const UIVariants = Object.freeze({
    LSB_ENCODE: 'LSB_ENCODE',
    LSB_DECODE: 'LSB_DECODE',
    BPCS_ENCODE: 'BPCS_ENCODE',
    BPCS_DECODE: 'BPCS_DECODE'
})

/**
 * @type LSBKeyParams[]
 */
const LSB_KEY_PARAMS = [
    'StartX',
    'StartY',
    'EndX',
    'EndY',
    'GapX',

```

```

        'GapY',
        'ChannelsPerPixel',
        'Channels'
    ]
    /**
     * @type Record<LSBKeyParams, string>
     */
    const LSB_KEY_PARSING_SCHEMA = {
        StartX: 'S',
        StartY: 'T',
        EndX: 'E',
        EndY: 'N',
        GapX: 'H',
        GapY: 'V',
        ChannelsPerPixel: 'P',
        Channels: 'C'
    }

    /**
     * @type {Config}
     */
    const config = {
        wasmUrl: './main.wasm',

        globalIds: {
            originalImageInput: { id: 'original-input', type: HTMLInputElement },
            originalImageInputDropZone: {
                id: 'original-input-drop-zone',
                type: HTMLLabelElement
            },
            originalImagePreview: { id: 'original-preview', type: HTMLImageElement },
            resultImagePreview: { id: 'result-preview', type: HTMLImageElement },
            submitButton: { id: 'submit-button', type: HTMLButtonElement }
        },

        menu: {
            name: {
                methods: 'menu-method',
                operation: 'menu-operation'
            },
            value: {
                methods: ['lsb', 'bpcs'],
                operation: ['encode', 'decode']
            }
        },

        // prettier-ignore
        UIids: {
            menu: {
                base: { id: "menu", type: HTMLDivElement },
                methods: { id: "menu-methods", type: HTMLDivElement },
            },
            lsbBlock: { id: "lsb", type: HTMLDivElement },
            encodeBlock: { id: "encode", type: HTMLDivElement },
            decodeBlock: { id: "decode", type: HTMLDivElement },
            swapButton: { id: "swap", type: HTMLButtonElement },

            ERROR: {
                block: { id: "error", type: HTMLDivElement },
                message: { id: "error-message", type: HTMLDivElement },
                button: { id: "error-close", type: HTMLButtonElement },
            }
        }
    }

```

```

    },

    // prettier-ignore
    ids: {
        DEBUG: { id: 'debug', type: HTMLInputElement },

        SHARED: {
            secretAsFileCheckbox: { id: 'secret-as-file', type:
HTMLInputElement },
        },
        ENCODE: {
            secretMessageInput: { id: 'encode-secret-message-input', type:
HTMLInputElement },
            secretFileInput: { id: 'encode-secret-file-input', type:
HTMLInputElement },
            secretMessageInputBlock: { id: 'encode-secret-message-input-block',
type: HTMLLabelElement },
            secretFileInputBlock: { id: 'encode-secret-file-input-block', type:
HTMLLabelElement },
        },
        DECODE: {
            secretMessageOutputBlock: { id: 'decode-secret-message-output-
block', type: HTMLLabelElement },
            secretFileOutputButtonBlock: { id: 'decode-secret-file-output-
button-block', type: HTMLDivElement },
            secretMessageOutput: { id: 'decode-secret-message-output', type:
HTMLInputElement },
            secretFileOutputButton: { id: 'decode-secret-file-output-button',
type: HTMLButtonElement },
        },
        LSB: {
            keyInputBlock: { id: "lsb-secret-key-block", type: HTMLDivElement
},
            keyInputOutput: { id: 'lsb-secret-key', type: HTMLInputElement },
        },
    },
}

// prettier-ignore
const GLOBAL = {
    originalImagePreview: loadElement(config.globalIds.originalImagePreview),
    originalImageInputDropZone:
loadElement(config.globalIds.originalImageInputDropZone),
    resultImagePreview: loadElement(config.globalIds.resultImagePreview),
    submitButton: loadElement(config.globalIds.submitButton),
    originalImageInput: loadElement(config.globalIds.originalImageInput),
}

const UI = {
    menu: {
        base: loadElement(config.UIids.menu.base),
        methods: loadElement(config.UIids.menu.methods)
    },
    lsbBlock: loadElement(config.UIids.lsbBlock),
    encodeBlock: loadElement(config.UIids.encodeBlock),
    decodeBlock: loadElement(config.UIids.decodeBlock),
    swapButton: loadElement(config.UIids.swapButton),
    ERROR: {
        block: loadElement(config.UIids.ERROR.block),
        message: loadElement(config.UIids.ERROR.message),
        button: loadElement(config.UIids.ERROR.button)
    }
}

```

```

}

const SHARED = {
  secretAsFileCheckbox: loadElement(config.ids.SHARED.secretAsFileCheckbox),
  encode: {
    secretMessageInput: loadElement(config.ids.ENCODE.secretMessageInput),
    secretFileInput: loadElement(config.ids.ENCODE.secretFileInput),
    secretMessageInputBlock: loadElement(
      config.ids.ENCODE.secretMessageInputBlock
    ),
    secretFileInputBlock: loadElement(config.ids.ENCODE.secretFileInputBlock)
  },
  decode: {
    secretMessageOutput: loadElement(config.ids.DECODE.secretMessageOutput),
    secretFileOutputButton: loadElement(
      config.ids.DECODE.secretFileOutputButton
    ),
    secretMessageOutputBlock: loadElement(
      config.ids.DECODE.secretMessageOutputBlock
    ),
    secretFileOutputButtonBlock: loadElement(
      config.ids.DECODE.secretFileOutputButtonBlock
    )
  }
}

// prettier-ignore
const LSB = {
  keyInputBlock: loadElement(config.ids.LSB.keyInputBlock),
  keyInputOutput: loadElement(config.ids.LSB.keyInputOutput),
}

const DEBUG = loadElement(config.ids.DEBUG)

/**
 * @type {State}
 */
const state = {
  activeMethod: 'LSB',
  activeOperation: 'ENCODE',
  debugMode: false,

  originalImageFile: undefined,
  resultImageFile: undefined,
  secretAsFile: false,
  secretMessage: '',
  encodeSecretFile: undefined,
  decodedSecretFile: undefined,

  LSB: {
    key: {
      startX: 0,
      startY: 0,
      endX: 0,
      endY: 0,
      gapX: 0,
      gapY: 0,
      channelsPerPixel: 3,
      channels: ['R', 'G', 'B']
    }
  },
},

```

```

        errorMessage: ''
    }
}

// --

window.addEventListener('drop', e => {
    if (
        e.dataTransfer &&
        Array.from(e.dataTransfer.items).some(item => item.kind === 'file')
    ) {
        e.preventDefault()
    }
})

window.addEventListener('dragover', e => {
    if (!e.target) return

    const fileItems =
        e.dataTransfer &&
        Array.from(e.dataTransfer.items).filter(item => item.kind === 'file')

    if (fileItems && fileItems.length > 0) {
        e.preventDefault()

        assert(e.target instanceof Node, 'e.target should be Node')

        if (!GLOBAL.originalImageInputDropZone.contains(e.target)) {
            e.dataTransfer.dropEffect = 'none'
        }
    }
})

GLOBAL.originalImageInputDropZone.addEventListener('dragover', e => {
    const fileItems =
        e.dataTransfer &&
        Array.from(e.dataTransfer.items).filter(item => item.kind === 'file')

    if (fileItems && fileItems.length > 0) {
        e.preventDefault()

        if (fileItems.some(item => item.type.startsWith('image/'))) {
            e.dataTransfer.dropEffect = 'copy'
        } else {
            e.dataTransfer.dropEffect = 'none'
        }
    }
})

GLOBAL.originalImageInputDropZone.addEventListener('drop', e => {
    e.preventDefault()

    const files =
        e.dataTransfer &&
        Array.from(e.dataTransfer.items)
            .map(item => item.getAsFile())
            .filter(file => file)

    const firstFile = files?.[0]

    if (!firstFile) {
        showError('No file selected. Please choose a file.')
    }
})

```

```

        state.originalImageFile = firstFile
        render()
    })

    UI.ERROR.button.addEventListener('click', e => {
        assert(
            e.target instanceof config.UIids.ERROR.button.type,
            'Event target on close error button should be equal to config type!'
        )

        state.errorMessage = ''
        render()
    })

    LSB.keyInputBlock.addEventListener('change', e => {
        assert(
            e.target instanceof HTMLInputElement,
            'Key input block change should be called only on input element!'
        )

        if (e.target.name == 'key') {
            if (e.target.value == '') return

            let result

            try {
                result = goParseLSBKey(e.target.value)
            } catch (err) {
                errorHandler(err)
                return
            }

            assert(
                result !== undefined,
                'Golang function result should be always defined'
            )

            if (result.ok == false) {
                showError(`Error! ${result.message}`)
                return
            }

            const parsedKey = result.data
            console.log(parsedKey)
            state.LSB.key = parsedKey

            render()
            return
        }

        assert(
            LSB_KEY_PARAMS.includes(e.target.name),
            'Key input block input name should be one of LSB key params'
        )

        if (e.target.name === 'Channels') {
            const value = e.target.value.split('')
            state.LSB.key[e.target.name] = value

            render()
            return
        }
    })

```

```

    }

    const value = Number(e.target.value)
    assert(Number.isNaN(value) === false, 'Input number value should not be NaN!')

    state.LSB.key[e.target.name] = value
    render()
  })

  SHARED.decode.secretFileOutputButton.addEventListener('click', e => {
    assert(
      e.target instanceof config.ids.DECODE.secretFileOutputButton.type,
      'Event target of lsb decode file output button should be equal to config
value'
    )

    assert(
      state.decodedSecretFile !== undefined,
      'Decoded secret file lsb should be defined on click on button'
    )

    const url = URL.createObjectURL(state.decodedSecretFile)

    const a = document.createElement('a')
    a.href = url
    a.download = state.decodedSecretFile.name
    document.body.appendChild(a)
    a.click()

    a.remove()
    URL.revokeObjectURL(url)
  })

  SHARED.encode.secretFileInput.addEventListener('change', e => {
    assert(
      e.target instanceof config.ids.ENCODE.secretFileInput.type,
      'Event target of lsb secret file input should be equal to config file'
    )
    assert(e.target.files !== null, 'LSB secret file input should have files')

    const file = e.target.files[0]
    if (!file) {
      showError('No file selected. Please choose a file.')
    }

    state.encodeSecretFile = file
  })

  SHARED.secretAsFileCheckbox.addEventListener('change', e => {
    assert(
      e.target instanceof config.ids.SHARED.secretAsFileCheckbox.type,
      'Event target on click lsb secret as file should be same type as config'
    )

    state.secretAsFile = e.target.checked

    render()
  })

  UI.swapButton.addEventListener('click', e => {
    assert(
      e.target instanceof config.UIids.swapButton.type,

```

```

        'Event target on click swap should be same type as config swap button'
    )

    if (state.originalImageFile === undefined) return
    if (state.resultImageFile === undefined) return

    const temp = state.originalImageFile
    state.originalImageFile = state.resultImageFile
    state.resultImageFile = temp

    render()
  })

  DEBUG.addEventListener('change', e => {
    assert(
      e.target instanceof config.ids.DEBUG.type,
      'Event target on change debug should be same type as debug'
    )

    state.debugMode = e.target.checked
    goDebug(state.debugMode)
  })

  GLOBAL.originalImageInput.addEventListener('change', e => {
    assert(
      e.target instanceof config.globalIds.originalImageInput.type,
      'Event target on change image input should be same type as type from
config'
    )
    assert(e.target.files !== null, 'Original image input should have files')

    const file = e.target.files[0]
    if (!file) {
      showError('No file selected. Please choose a file.')
    }

    state.originalImageFile = file
    render()
  })

  SHARED.encode.secretMessageInput.addEventListener('change', e => {
    assert(
      e.target !== null,
      'Secret message input on change event target is null!'
    )
    assert(
      e.target instanceof config.ids.ENCODE.secretMessageInput.type,
      'Secret message input on change event target should have the same type
from config!'
    )

    state.secretMessage = e.target.value
  })

  async function prepareSecretMessage() {
    if (state.secretAsFile === true) {
      if (state.encodeSecretFile === undefined) {
        showError('Secret file should exists to run LSB encoding')
      }

      return await fileToByteArray(state.encodeSecretFile)
    }
  }

```

```

    if (state.secretMessage === '') {
        showError('Secret message is empty!')
    }

    return new TextEncoder().encode(state.secretMessage)
}

async function submitLSBEncode() {
    if (state.originalImageFile === undefined) {
        showError('Image is not loaded!')
    }

    const message = await prepareSecretMessage()
    const originalImage = await fileToByteArray(state.originalImageFile)

    let result = undefined
    try {
        result = goEncodeLSB(
            originalImage,
            state.originalImageFile.type,
            message,
            generateLsbKey()
        )
    } catch (err) {
        errorHandler(err)
        return
    }

    assert(
        result !== undefined,
        'Golang function result should be always defined'
    )

    if (result.ok === false) {
        showError(`Error! ${result.message}`)
        return
    }

    const content = result.data
    console.log('JS result:', content, typeof content)

    let resultImageType = state.originalImageFile.type

    if (state.originalImageFile.type === 'image/jpeg') {
        resultImageType = 'image/png'
    }

    const blob = new Blob([content], { type: resultImageType })
    state.resultImageFile = new File([blob], `result.${blob.type}`, {
        type: blob.type
    })

    render()
}

async function submitLSBDecode() {
    if (state.originalImageFile === undefined) {
        showError('Image is not loaded!')
    }

    const originalImage = await fileToByteArray(state.originalImageFile)

```

```

let decodeImageType = state.originalImageFile.type

if (state.originalImageFile.type === 'image/jpeg') {
    decodeImageType = 'image/png'
}

let result

try {
    result = goDecodeLSB(originalImage, decodeImageType, generateLsbKey())
} catch (err) {
    errorHandler(err)
    return
}

assert(
    result !== undefined,
    'Golang function result should be always defined'
)

if (result.ok === false) {
    showError(`Error! ${result.message}`)
    return
}

const content = result.data

if (state.secretAsFile) {
    state.decodedSecretFile = new File([content], `result`)
} else {
    SHARED.decode.secretMessageOutput.value = content.toString()
}

console.log('Decoded message js:', content)

render()
}

async function submitBPCSEncode() {
    if (state.originalImageFile === undefined) {
        showError('Image is not loaded!')
    }

    const message = await prepareSecretMessage()
    const originalImage = await fileToByteArray(state.originalImageFile)

    let result
    try {
        result = goEncodeBPCS(originalImage, state.originalImageFile.type,
message)
    } catch (err) {
        errorHandler(err)
        return
    }

    assert(
        result !== undefined,
        'Golang function result should be always defined'
    )

    if (result.ok === false) {

```

```

        showError(`Error! ${result.message}`)
        return
    }

    const content = result.data

    let resultImageType = state.originalImageFile.type
    if (state.originalImageFile.type === 'image/jpeg') {
        resultImageType = 'image/png'
    }

    const blob = new Blob([content], { type: resultImageType })
    state.resultImageFile = new File([blob], `result.${blob.type}`, {
        type: blob.type
    })

    render()
}

async function submitBPCSDecode() {
    if (state.originalImageFile === undefined) {
        showError('Image is not loaded!')
    }

    const originalImage = await fileToByteArray(state.originalImageFile)

    let decodeImageType = state.originalImageFile.type
    if (state.originalImageFile.type === 'image/jpeg') {
        decodeImageType = 'image/png'
    }

    let result
    try {
        result = goDecodeBPCS(originalImage, decodeImageType)
    } catch (err) {
        errorHandler(err)
        return
    }

    assert(
        result !== undefined,
        'Golang function result should be always defined'
    )

    if (result.ok === false) {
        showError(`Error! ${result.message}`)
        return
    }

    const content = result.data

    if (state.secretAsFile) {
        state.decodedSecretFile = new File([content], `result`)
    } else {
        SHARED.decode.secretMessageOutput.value = content.toString()
    }

    console.log('[JS] BPCS decoded message:', content)

    render()
}

```

```

GLOBAL.submitButton.addEventListener('click', () => {
  state.errorMessage = ''
  render()

  if (state.activeMethod == 'LSB') {
    if (state.activeOperation == 'ENCODE') {
      submitLSBEncode()
    }

    if (state.activeOperation == 'DECODE') {
      submitLSBDecode()
    }
  }

  if (state.activeMethod == 'BPCS') {
    if (state.activeOperation == 'ENCODE') {
      submitBPCSEncode()
    }

    if (state.activeOperation == 'DECODE') {
      submitBPCSDecode()
    }
  }
})

UI.menu.base.addEventListener('change', e => {
  assert(
    e.target instanceof HTMLInputElement,
    'Change event in menu should be only on HTMLInputElement'
  )

  assert(
    e.target.name == config.menu.name.methods ||
    e.target.name == config.menu.name.operation,
    'Menu input name should be one of menu names'
  )

  assert(
    config.menu.value.methods.includes(e.target.value) ||
    config.menu.value.operation.includes(e.target.value),
    'Menu input value should be one from menu config'
  )

  if (e.target.name == config.menu.name.methods) {
    if (e.target.checked) {
      if (e.target.value == 'lsb') {
        state.activeMethod = 'LSB'
      }

      if (e.target.value == 'bpcs') {
        state.activeMethod = 'BPCS'
      }
    }
  }

  if (e.target.name == config.menu.name.operation) {
    if (e.target.checked) {
      if (e.target.value == 'encode') {
        state.activeOperation = 'ENCODE'
      }

      if (e.target.value == 'decode') {

```

```

        state.activeOperation = 'DECODE'
    }
}
render()
})

async function main() {
    render()

    const go = new Go()

    const wasmModule = await WebAssembly.instantiateStreaming(
        fetch(config.wasmUrl),
        go.importObject
    )

    go.run(wasmModule.instance)
}

main()

function render() {
    if (state.activeOperation === 'ENCODE') {
        UI.encodeBlock.classList.remove('hidden')
        UI.decodeBlock.classList.add('hidden')
    } else {
        UI.encodeBlock.classList.add('hidden')
        UI.decodeBlock.classList.remove('hidden')
    }

    if (state.activeMethod === 'LSB') {
        UI.lsbBlock.classList.remove('hidden')
    } else if (state.activeMethod === 'BPCS') {
        UI.lsbBlock.classList.add('hidden')
    }

    if (state.originalImageFile) {
        GLOBAL.originalImagePreview.src = URL.createObjectURL(
            state.originalImageFile
        )
        GLOBAL.originalImagePreview.classList.remove('hidden')
        GLOBAL.originalImageInputDropZone.classList.add('hidden')
    } else {
        GLOBAL.originalImagePreview.classList.add('hidden')
        GLOBAL.originalImageInputDropZone.classList.remove('hidden')
    }

    if (state.resultImageFile) {
        GLOBAL.resultImagePreview.src =
URL.createObjectURL(state.resultImageFile)
    }

    SHARED.secretAsFileCheckbox.checked = state.secretAsFile

    if (state.secretAsFile) {
        SHARED.encode.secretMessageInputBlock.classList.add('hidden')
        SHARED.decode.secretMessageOutputBlock.classList.add('hidden')
        SHARED.encode.secretFileInputBlock.classList.remove('hidden')
        SHARED.decode.secretFileOutputButtonBlock.classList.remove('hidden')
    } else {

```

```

        SHARED.encode.secretMessageInputBlock.classList.remove('hidden')
        SHARED.decode.secretMessageOutputBlock.classList.remove('hidden')
        SHARED.encode.secretFileInputBlock.classList.add('hidden')
        SHARED.decode.secretFileOutputButtonBlock.classList.add('hidden')
    }

    if (state.decodedSecretFile) {
        SHARED.decode.secretFileOutputButton.disabled = false
    } else {
        SHARED.decode.secretFileOutputButton.disabled = true
    }

    if (state.errorMessage !== '') {
        UI.ERROR.block.dataset['active'] = 'true'
        UI.ERROR.message.textContent = state.errorMessage
    } else {
        UI.ERROR.block.dataset['active'] = 'false'
        UI.ERROR.message.textContent = state.errorMessage
    }

    LSB.keyInputOutput.value = generateLsbKey()
    setLSBKeyFields()
}

function setLSBKeyFields() {
    for (const key of LSB_KEY_PARAMS) {
        const input = LSB.keyInputBlock.querySelector(`[name=${key}]`)
        assert(
            input instanceof HTMLInputElement,
            `LSB key field with name ${key} should be HTMLInputElement`
        )

        if (key === 'Channels') {
            input.value = state.LSB.key[key].join('')
            continue
        }

        input.value = String(state.LSB.key[key])
    }
}

function generateLsbKey() {
    let result = ''

    for (const key of LSB_KEY_PARAMS) {
        if (state.LSB.key[key] !== undefined) {
            if (key !== 'Channels') {
                result += LSB_KEY_PARSING_SCHEMA[key] + state.LSB.key[key]
            } else {
                result += state.LSB.key[key]
                    .map(e1 => LSB_KEY_PARSING_SCHEMA[key] + e1)
                    .join('')
            }
        }
    }

    return result
}

/**
 * @type {ErrorHandler}
 */

```

```

function errorHandler(err) {
  if (err instanceof WebAssembly.RuntimeError) {
    showError('Catch WebAssembly.RuntimeError! Check console for more
details!')
    return
  }

  console.log('Error', err)
  showError('Catch unknown error! Check console for more details!')
}

/**
 * @type {ShowError}
 */
function showError(message) {
  console.log(message)
  state.errorMessage = message
  render()
}

/**
 * @type {FileToByteArray}
 */
async function fileToByteArray(file) {
  return file.arrayBuffer().then(value => new Uint8Array(value))
}

/**
 * @type {LoadElement}
 */
function loadElement(elementInfo) {
  const element = document.querySelector('#' + elementInfo.id)
  assert(
    element !== null,
    `Element with id \`${elementInfo.id}\` should not be null!`
  )
  assert(
    element instanceof elementInfo.type,
    `Element with id \`${elementInfo.id}\` should be instance of
${elementInfo.type.name}!`
  )

  return element
}

/**
 * @type {Assert}
 */
function assert(condition, message) {
  if (!condition) {
    showError('[ASSERTION]: ' + message)
  }
}

/**
 * @type {IsRecord}
 */
function isRecord(value) {
  return typeof value === 'object' && value !== null && !Array.isArray(value)
}

```

```

=====

FILE: website\main.css
*,
*::before,
*::after {
    box-sizing: 'border-box';
    padding: 0;
    margin: 0;
}

:root {
    font-family: 'Inter', 'Roboto', sans-serif;

    /* Tailwind colors */
    --color-red-50: lab(96.5005% 4.18508 1.52328);
    --color-red-100: lab(92.243% 10.2865 3.83865);
    --color-red-200: lab(86.017% 19.8815 7.75869);
    --color-red-300: lab(76.5514% 36.422 15.5335);
    --color-red-400: lab(63.7053% 60.745 31.3109);
    --color-red-500: lab(55.4814% 75.0732 48.8528);
    --color-red-600: lab(48.4493% 77.4328 61.5452);
    --color-red-700: lab(40.4273% 67.2623 53.7441);
    --color-red-800: lab(33.7174% 55.8993 41.0293);
    --color-red-900: lab(28.5139% 44.5539 29.0463);
    --color-red-950: lab(13.003% 29.04 16.7519);
    --color-orange-50: lab(97.7008% 1.53735 5.90649);
    --color-orange-100: lab(94.7127% 3.58394 14.3151);
    --color-orange-200: lab(88.4871% 9.94918 28.8378);
    --color-orange-300: lab(80.8059% 21.7313 50.4455);
    --color-orange-400: lab(70.0429% 42.5156 75.8207);
    --color-orange-500: lab(64.272% 57.1788 90.3583);
    --color-orange-600: lab(57.1026% 64.2584 89.8886);
    --color-orange-700: lab(46.4615% 57.7275 70.8507);
    --color-orange-800: lab(37.1566% 46.6433 50.5562);
    --color-orange-900: lab(30.2951% 36.0434 37.671);
    --color-orange-950: lab(14.1747% 23.4515 19.4461);
    --color-amber-50: lab(98.6252% -0.635922 8.42309);
    --color-amber-100: lab(95.916% -1.21653 23.111);
    --color-amber-200: lab(91.7203% -0.505269 49.9084);
    --color-amber-300: lab(86.4156% 6.13147 78.3961);
    --color-amber-400: lab(80.1641% 16.6016 99.2089);
    --color-amber-500: lab(72.7183% 31.8672 97.9407);
    --color-amber-600: lab(60.3514% 40.5624 87.1228);
    --color-amber-700: lab(47.2709% 42.9082 69.2966);
    --color-amber-800: lab(37.8822% 37.1699 52.2718);
    --color-amber-900: lab(31.2288% 30.2627 40.0378);
    --color-amber-950: lab(15.8111% 20.9107 23.3752);
    --color-yellow-50: lab(98.6846% -1.79055 9.7766);
    --color-yellow-100: lab(97.3564% -4.51407 27.344);
    --color-yellow-200: lab(94.3433% -5.00429 52.9663);
    --color-yellow-300: lab(89.7033% -0.480294 84.4917);
    --color-yellow-400: lab(83.2664% 8.65132 106.895);
    --color-yellow-500: lab(76.3898% 14.5258 98.4589);
    --color-yellow-600: lab(62.7799% 22.4197 86.1544);
    --color-yellow-700: lab(47.8202% 25.2426 66.5015);
    --color-yellow-800: lab(38.7484% 23.5833 51.4916);
    --color-yellow-900: lab(32.3865% 21.1273 38.5959);
    --color-yellow-950: lab(16.8146% 15.7422 23.1133);
    --color-lime-50: lab(98.7039% -5.32573 10.2149);
    --color-lime-100: lab(96.8662% -11.7133 22.0854);
    --color-lime-200: lab(94.0718% -22.5338 42.5238);

```

```

--color-lime-300: lab(89.9218% -35.6546 68.5254);
--color-lime-400: lab(83.7876% -45.0447 88.4738);
--color-lime-500: lab(75.3197% -46.6547 86.1778);
--color-lime-600: lab(61.1055% -41.0235 73.1483);
--color-lime-700: lab(47.246% -32.2589 55.8249);
--color-lime-800: lab(37.7655% -25.1694 43.0683);
--color-lime-900: lab(31.9931% -20.7654 33.7379);
--color-lime-950: lab(16.5113% -15.1841 22.0145);
--color-green-50: lab(98.1563% -5.60117 2.75915);
--color-green-100: lab(96.1861% -13.8464 6.52365);
--color-green-200: lab(92.4222% -26.4702 12.9427);
--color-green-300: lab(86.9953% -47.2691 25.0054);
--color-green-400: lab(78.503% -64.9265 39.7492);
--color-green-500: lab(70.5521% -66.5147 45.8073);
--color-green-600: lab(59.0978% -58.6621 41.2579);
--color-green-700: lab(47.0329% -47.0239 31.4788);
--color-green-800: lab(37.4616% -36.7971 22.9692);
--color-green-900: lab(30.797% -29.6927 17.382);
--color-green-950: lab(15.6845% -20.4225 11.7249);
--color-emerald-50: lab(97.8462% -6.94966 1.85487);
--color-emerald-100: lab(94.9004% -17.0769 5.63836);
--color-emerald-200: lab(90.2247% -31.039 9.47084);
--color-emerald-300: lab(83.9203% -48.7124 13.8849);
--color-emerald-400: lab(75.0771% -60.7313 19.4147);
--color-emerald-500: lab(66.9756% -58.27 19.5419);
--color-emerald-600: lab(55.0481% -49.9246 15.93);
--color-emerald-700: lab(44.4871% -41.0396 11.0361);
--color-emerald-800: lab(35.3675% -33.1188 8.04002);
--color-emerald-900: lab(28.8637% -26.9249 5.45986);
--color-emerald-950: lab(15.0582% -17.9507 2.38369);
--color-teal-50: lab(98.3189% -4.74921 -0.111711);
--color-teal-100: lab(95.1845% -17.4212 -0.425422);
--color-teal-200: lab(90.7612% -33.1343 -0.542295);
--color-teal-300: lab(84.8977% -48.1516 -1.3321);
--color-teal-400: lab(76.0109% -53.3483 -2.27906);
--color-teal-500: lab(67.3859% -49.0983 -2.63511);
--color-teal-600: lab(55.0223% -41.0774 -3.90277);
--color-teal-700: lab(44.4134% -33.1436 -4.22149);
--color-teal-800: lab(35.5975% -26.6648 -4.34487);
--color-teal-900: lab(29.506% -21.4706 -3.59886);
--color-teal-950: lab(16.6371% -15.3183 -3.81732);
--color-cyan-50: lab(98.3304% -5.97432 -2.62108);
--color-cyan-100: lab(95.3146% -13.8285 -6.84732);
--color-cyan-200: lab(91.0821% -24.0435 -12.8306);
--color-cyan-300: lab(85.3886% -36.7636 -21.5716);
--color-cyan-400: lab(76.6045% -40.9406 -29.6231);
--color-cyan-500: lab(67.805% -35.3952 -30.2018);
--color-cyan-600: lab(55.1767% -26.7496 -30.5139);
--color-cyan-700: lab(44.7267% -21.5987 -26.118);
--color-cyan-800: lab(36.5114% -17.1989 -21.6292);
--color-cyan-900: lab(30.372% -13.1853 -18.7887);
--color-cyan-950: lab(19.1528% -9.68757 -15.5267);
--color-sky-50: lab(97.3623% -2.33802 -4.13098);
--color-sky-100: lab(94.3709% -4.56053 -8.23453);
--color-sky-200: lab(88.6983% -11.3978 -16.8488);
--color-sky-300: lab(80.3307% -20.2945 -31.385);
--color-sky-400: lab(70.687% -23.6078 -45.9483);
--color-sky-500: lab(63.3038% -18.433 -51.0407);
--color-sky-600: lab(51.7754% -11.4712 -49.8349);
--color-sky-700: lab(41.6013% -9.10804 -42.5647);
--color-sky-800: lab(35.164% -9.57692 -34.4068);
--color-sky-900: lab(29.1959% -8.34689 -28.2453);

```

```

--color-sky-950: lab(17.8299% -5.31271 -21.1584);
--color-blue-50: lab(96.492% -1.14644 -5.11479);
--color-blue-100: lab(92.0301% -2.24757 -11.6453);
--color-blue-200: lab(86.15% -4.04379 -21.0797);
--color-blue-300: lab(77.5052% -6.4629 -36.42);
--color-blue-400: lab(65.0361% -1.42065 -56.9802);
--color-blue-500: lab(54.1736% 13.3369 -74.6839);
--color-blue-600: lab(44.0605% 29.0279 -86.0352);
--color-blue-700: lab(36.9089% 35.0961 -85.6872);
--color-blue-800: lab(30.2514% 27.7853 -70.2699);
--color-blue-900: lab(26.1542% 15.7545 -51.5504);
--color-blue-950: lab(15.6723% 8.86232 -32.2945);
--color-indigo-50: lab(95.4818% 0.411302 -6.78529);
--color-indigo-100: lab(91.6577% 1.04591 -12.7199);
--color-indigo-200: lab(84.4329% 3.18977 -23.9688);
--color-indigo-300: lab(74.0235% 8.54138 -41.6075);
--color-indigo-400: lab(59.866% 22.4834 -64.4485);
--color-indigo-500: lab(48.295% 38.3129 -81.9673);
--color-indigo-600: lab(38.4009% 52.6132 -92.3857);
--color-indigo-700: lab(32.4486% 49.2217 -84.6695);
--color-indigo-800: lab(26.6645% 37.9804 -68.6402);
--color-indigo-900: lab(23.3911% 24.6978 -50.4718);
--color-indigo-950: lab(12.4853% 14.9672 -31.3418);
--color-violet-50: lab(96.2416% 2.28849 -5.51657);
--color-violet-100: lab(93.0838% 4.35197 -9.88284);
--color-violet-200: lab(87.0888% 8.53688 -19.4189);
--color-violet-300: lab(76.7419% 18.3911 -37.0706);
--color-violet-400: lab(62.8239% 34.9159 -60.0512);
--color-violet-500: lab(49.9355% 55.1776 -81.8963);
--color-violet-600: lab(41.088% 68.9966 -91.995);
--color-violet-700: lab(35.2783% 67.9912 -88.793);
--color-violet-800: lab(29.3188% 57.7986 -76.1493);
--color-violet-900: lab(24.3783% 45.7525 -61.4902);
--color-violet-950: lab(14.0706% 33.3353 -46.7553);
--color-purple-50: lab(97.1627% 2.99937 -4.13398);
--color-purple-100: lab(93.3333% 6.97437 -9.83434);
--color-purple-200: lab(87.8405% 13.4282 -18.7159);
--color-purple-300: lab(78.3298% 26.2195 -34.9499);
--color-purple-400: lab(63.6946% 47.6127 -59.2066);
--color-purple-500: lab(52.0183% 66.11 -78.2316);
--color-purple-600: lab(43.0295% 75.21 -86.5669);
--color-purple-700: lab(36.1758% 69.8525 -80.0381);
--color-purple-800: lab(30.6017% 56.7637 -64.4751);
--color-purple-900: lab(24.9401% 45.2703 -51.2728);
--color-purple-950: lab(14.8253% 38.9005 -44.5861);
--color-fuchsia-50: lab(97.1083% 4.46233 -4.09334);
--color-fuchsia-100: lab(93.9419% 9.57647 -9.08735);
--color-fuchsia-200: lab(87.7108% 19.9958 -18.2054);
--color-fuchsia-300: lab(78.5378% 39.3533 -32.9615);
--color-fuchsia-400: lab(66.1178% 66.0652 -52.4733);
--color-fuchsia-500: lab(56.4256% 83.132 -64.639);
--color-fuchsia-600: lab(47.5131% 83.4271 -63.0363);
--color-fuchsia-700: lab(39.787% 72.2653 -53.1244);
--color-fuchsia-800: lab(32.904% 60.2883 -43.6569);
--color-fuchsia-900: lab(27.755% 48.6174 -34.3553);
--color-fuchsia-950: lab(15.7348% 39.0235 -27.4073);
--color-pink-50: lab(96.4459% 4.53997 -1.49434);
--color-pink-100: lab(93.5864% 9.01193 -3.15079);
--color-pink-200: lab(87.4504% 19.6 -6.46662);
--color-pink-300: lab(77.8308% 38.525 -10.5394);
--color-pink-400: lab(64.5597% 64.3615 -12.7988);
--color-pink-500: lab(56.9303% 76.8162 -8.07021);

```

```

--color-pink-600: lab(49.5493% 79.8381 2.31768);
--color-pink-700: lab(42.1737% 71.8009 7.42233);
--color-pink-800: lab(34.9559% 60.2885 5.99639);
--color-pink-900: lab(29.4367% 49.3962 3.35757);
--color-pink-950: lab(15.6116% 35.2166 3.53979);
--color-rose-50: lab(96.2369% 4.94155 1.28011);
--color-rose-100: lab(92.8221% 9.86832 2.60075);
--color-rose-200: lab(86.806% 19.1909 4.07754);
--color-rose-300: lab(76.6339% 38.3549 9.68835);
--color-rose-400: lab(64.4125% 63.0291 19.2068);
--color-rose-500: lab(56.101% 79.4328 31.4532);
--color-rose-600: lab(49.1882% 81.577 36.0311);
--color-rose-700: lab(41.1651% 71.6251 30.3087);
--color-rose-800: lab(34.6481% 60.802 20.1957);
--color-rose-900: lab(29.7104% 51.514 12.6253);
--color-rose-950: lab(14.2323% 34.0086 9.80922);
--color-slate-50: lab(98.1434% -0.369519 -1.05966);
--color-slate-100: lab(96.286% -0.852436 -2.46847);
--color-slate-200: lab(91.7353% -0.998765 -4.76968);
--color-slate-300: lab(84.7652% -1.94535 -7.93337);
--color-slate-400: lab(65.5349% -2.25151 -14.5072);
--color-slate-500: lab(48.0876% -2.03595 -16.5814);
--color-slate-600: lab(35.5623% -1.74978 -15.4316);
--color-slate-700: lab(26.9569% -1.47016 -15.6993);
--color-slate-800: lab(16.132% -0.318035 -14.6672);
--color-slate-900: lab(7.78673% 1.82345 -15.0537);
--color-slate-950: lab(1.76974% 1.32743 -9.28855);
--color-gray-50: lab(98.2596% -0.247031 -0.706708);
--color-gray-100: lab(96.1596% -0.0823438 -1.13575);
--color-gray-200: lab(91.6229% -0.159115 -2.26791);
--color-gray-300: lab(85.1236% -0.612259 -3.7138);
--color-gray-400: lab(65.9269% -0.832707 -8.17473);
--color-gray-500: lab(47.7841% -0.393182 -10.0268);
--color-gray-600: lab(35.6337% -1.58697 -10.8425);
--color-gray-700: lab(27.1134% -0.956401 -12.3224);
--color-gray-800: lab(16.1051% -1.18239 -11.7533);
--color-gray-900: lab(8.11897% 0.811279 -12.254);
--color-gray-950: lab(1.90334% 0.278696 -5.48866);
--color-zinc-50: lab(98.26% 0 0);
--color-zinc-100: lab(96.1634% 0.0993311 -0.364041);
--color-zinc-200: lab(90.6853% 0.399232 -1.45452);
--color-zinc-300: lab(84.9837% 0.601262 -2.17986);
--color-zinc-400: lab(65.6464% 1.53497 -5.42429);
--color-zinc-500: lab(47.8878% 1.65477 -5.77283);
--color-zinc-600: lab(35.1166% 1.78212 -6.1173);
--color-zinc-700: lab(26.8019% 1.35387 -4.68303);
--color-zinc-800: lab(15.7305% 0.613764 -2.16959);
--color-zinc-900: lab(8.30603% 0.618205 -2.16572);
--color-zinc-950: lab(2.51107% 0.242703 -0.886115);
--color-neutral-50: lab(98.26% 0 0);
--color-neutral-100: lab(96.52% -0.0000298023 0.0000119209);
--color-neutral-200: lab(90.952% 0 -0.0000119209);
--color-neutral-300: lab(84.92% 0 -0.0000119209);
--color-neutral-400: lab(66.128% -0.0000298023 0.0000119209);
--color-neutral-500: lab(48.496% 0 0);
--color-neutral-600: lab(34.924% 0 0);
--color-neutral-700: lab(27.036% 0 0);
--color-neutral-800: lab(15.204% 0 -0.00000596046);
--color-neutral-900: lab(7.78201% -0.0000149012 0);
--color-neutral-950: lab(2.75381% 0 0);
--color-stone-50: lab(98.2686% -0.0991821 0.364304);
--color-stone-100: lab(96.5286% -0.0991821 0.364268);

```

```

--color-stone-200: lab(91.055% 0.663072 0.865579);
--color-stone-300: lab(84.7909% 0.928015 1.59738);
--color-stone-400: lab(66.2166% 1.88044 3.20326);
--color-stone-500: lab(48.1164% 2.35701 4.26852);
--color-stone-600: lab(35.5168% 1.08604 4.07829);
--color-stone-700: lab(27.3812% 1.32917 3.57789);
--color-stone-800: lab(15.0353% 1.96067 1.53427);
--color-stone-900: lab(9.03835% 1.15298 1.92955);
--color-stone-950: lab(2.86037% 0.455312 0.568903);
}

.hidden {
  /* TODO: hchange */
  display: none !important;
}

.divider {
  margin: 1rem;
  height: 1px;
  background: var(--color-sky-500);
  border: none;
}

/**
 * UI components
 */
label:hover {
  cursor: pointer;
}

.input-title {
  font-weight: normal;
  font-size: 1rem;
  color: var(--color-slate-700);
}

.input-label {
  display: flex;
  justify-content: space-between;
}

input[type='text'],
input[type='number'] {
  border: var(--color-slate-500) 1px solid;
  border-radius: 0.25rem;
  padding: 0.25rem 0.25rem;
}

/**
 * Layout
 */

.layout {
  max-width: 100%;
  max-height: 100%;
  display: grid;
  grid-template-columns: 1fr 1fr;
  margin: 3rem;
}

.layout--block {
  width: 100%;

```

```

}

.layout--block_center {
  display: flex;
  flex-direction: column;
  align-items: center;
}

/*
 * Menu
 */
.menu {
  padding-top: 2rem;
  max-width: 500px;
  display: flex;
  flex-direction: column;
  align-items: center;

  row-gap: 0.5rem;
}

.menu .menu-block {
  display: flex;
  column-gap: 1rem;
}

.menu .menu-block .menu-block--element {
  padding: 0.5rem;
  border-radius: 0.25rem;
  background: var(--color-slate-200);
}

.menu .menu-block .menu-block--element:hover {
  background: var(--color-slate-400);
}

.menu .menu-block .menu-block--element > input {
  display: none;
}

.menu .menu-block .menu-block--element:has(input:checked) {
  background: var(--color-violet-400);
}

.menu .menu-block.menu-block_main .menu-block--element:has(input:checked) {
  background: var(--color-pink-400);
}

.content {
  padding: 1rem;
  max-width: 500px;
  width: 100%;
}

.block {
  padding: 0.5rem 0;
  display: flex;
  flex-direction: column;
  row-gap: 0.5rem;
  width: 100%;
}

```

```

.block_images {
  max-width: 300px;
}

.block--title {
  color: var(--color-slate-800);
  font-size: 1.3rem;
}

.block .block--elements {
  display: flex;
  flex-direction: column;
  row-gap: 0.75rem;
}

.block .block--title.block--title_primary {
  color: var(--color-violet-600);
}

.block .block--title.block--title_secondary {
  color: var(--color-pink-400);
}

.images {
  display: flex;
  column-gap: 2rem;
  width: 100%;
}

.image-preview {
  aspect-ratio: 1;
  width: 100%;
  max-width: 300px;
  max-height: 300px;
  border: 1px solid var(--color-slate-500);
}

.swap-button {
  align-self: center;

  width: 100%;
  height: 100%;
  max-width: 2.5rem;
  max-height: 2.5rem;
  aspect-ratio: 1;

  border: none;
  background: var(--color-slate-200);
  border-radius: 0.5rem;
  padding: 0.5rem;

  color: var(--color-slate-800);
}

.swap-button:hover {
  color: var(--color-violet-500);
  background: var(--color-slate-100);
}

.swap-button--icon {
  pointer-events: none;
}

```

```

button:hover {
    cursor: pointer;
}

.button {
    width: 100%;
    height: 100%;

    border: none;
    background: var(--color-slate-200);
    border-radius: 0.5rem;
    padding: 0.5rem;

    color: var(--color-slate-800);
}

.button:disabled {
    background: var(--color-slate-300);
}

.button:hover:disabled {
    color: var(--color-slate-800);
    background: var(--color-slate-300);
    cursor: not-allowed;
}

.button:hover {
    color: var(--color-violet-500);
    background: var(--color-slate-100);
}

.submit-button {
    width: 100px;
    height: 50px;

    background: var(--color-emerald-400);
    border: 1px solid var(--color-emerald-600);
    border-radius: 0.25rem;
    font-size: 1.1rem;
    color: var(--color-slate-900);
}

.debug-label {
    max-width: 300px;
}

.error {
    position: absolute;
    top: 0;
    left: 0;
    height: 2.5rem;

    width: 100%;
    background: var(--color-red-500);
    color: white;

    display: flex;
    align-items: center;
    justify-content: center;

    column-gap: 1rem;

```

```

}

.error[data-active='false'] {
  top: -2.5rem;
}

.error--message {
  font-weight: 500;
}

.error--close {
  border: none;
  background: var(--color-red-500);
  color: white;
  padding: 0.25rem;
}

.error--close:hover {
  background: white;
  color: var(--color-red-500);
}

.file-drop--label {
  aspect-ratio: 1;

  width: 100%;
  height: 100%;
  max-width: 300px;
  max-height: 300px;
  border: 1px solid var(--color-slate-500);

  display: flex;
  align-items: center;
  justify-content: center;
  text-align: center;

  color: var(--color-slate-600);
  cursor: pointer;
}

.file-drop--input {
  display: none;
}

/* 1280px */
@media screen and (max-width: 80rem) {
  .layout {
    grid-template-columns: 1fr;
  }

  .images {
    justify-content: center;
  }

  .layout--block_secondary {
    order: 1;

    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    row-gap: 1rem;
  }
}

```

```

    }

    .layout--block_secondary > .block {
        width: auto;
        min-width: 500px;
    }

    .layout--block_main {
        order: 2;
    }
}

/* 768px */
@media screen and (max-width: 48rem) {
    .images {
        flex-direction: column;
        align-items: center;
    }

    .layout {
        margin: 2rem;
    }

    .layout--block_secondary > .block {
        min-width: auto;
        max-width: 300px;
        width: 100%;
    }
}

/* 768px */
@media screen and (max-width: 30rem) {
    .input-label {
        flex-direction: column;
        row-gap: 0.5rem;
    }

    .input-label:has(input[type='checkbox']) {
        flex-direction: row;
    }
}

```

=====

```

FILE: website\tsconfig.json
{
    "compilerOptions": {
        // Strict Checks
        "alwaysStrict": true,
        "noImplicitAny": true,
        "strictNullChecks": true,
        "useUnknownInCatchVariables": true,
        "strictPropertyInitialization": true,
        "strictFunctionTypes": true,
        "noImplicitThis": true,
        "strictBindCallApply": true,
        "noPropertyAccessFromIndexSignature": true,
        "noUncheckedIndexedAccess": true,
        "exactOptionalPropertyTypes": true,
        // Linter Checks
        "noImplicitReturns": true,

```

```

    "noImplicitOverride": true,
    "forceConsistentCasingInFileNames": true,
    // https://eslint.org/docs/rules/consistent-return ?
    "noFallthroughCasesInSwitch": true,
    // https://eslint.org/docs/rules/no-fallthrough
    "noUnusedLocals": true,
    // https://eslint.org/docs/rules/no-unused-vars
    "noUnusedParameters": true,
    // https://eslint.org/docs/rules/no-unused-vars#args
    "allowUnreachableCode": false,
    // https://eslint.org/docs/rules/no-unreachable ?
    "allowUnusedLabels": false,
    // https://eslint.org/docs/rules/no-unused-labels
    // Base Strict Checks
    "noImplicitUseStrict": false,
    "suppressExcessPropertyErrors": false,
    "suppressImplicitAnyIndexErrors": false,
    "noStrictGenericChecks": false,
    "lib": [
        "dom",
        "dom.iterable",
        "esnext"
    ],
    "allowJs": true,
    "checkJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true
  },
  "includes": [
    "*.d.ts",
    "*.js"
  ]
}

```

```
=====
```

```

FILE: website\wasm_exec.d.ts
/**
 * Go is the class as defined in the Golang `wasm_exec.js` distributable file required
 * for WebAssembly.
 * Golang WebAssembly wiki: https://github.com/golang/go/wiki/WebAssembly
 */
declare class Go {
  argv: string[];
  env: { [envKey: string]: string };
  exit: (code: number) => void;
  importObject: WebAssembly.Imports;
  exited: boolean;
  mem: DataView;
  run(instance: WebAssembly.Instance): Promise<void>;
}

```